

Diseño y desarrollo de una arquitectura multiagente
basada en LangGraph
para procesamiento de datos y detección de
anomalías industriales

Autor: pendiente

Curso académico: pendiente

Resumen

Este Trabajo Fin de Máster propone una arquitectura multiagente para el procesamiento, modelado y detección de anomalías en entornos industriales. El caso de uso inicial se centra en rodamientos de motores eléctricos monitorizados mediante señales de vibración. La solución combina agentes coordinados mediante LangGraph con ejecutores Python deterministas, de forma que las decisiones se representen mediante contratos estructurados y las transformaciones sobre datos sean reproducibles, audita-
bles y testeables.

Abstract

This Master's Thesis proposes a multi-agent architecture for data processing, modeling and anomaly detection in industrial environments. The initial use case focuses on electric motor bearings monitored through vibration signals. The solution combines agents coordinated with LangGraph and deterministic Python executors, so that decisions are represented through structured contracts and data transformations remain reproducible, auditable and testable.

Índice general

1. Introducción	1
1.1. Contexto	1
1.2. Motivacion	1
1.3. Problema	1
1.4. Objetivo general	2
1.5. Objetivos específicos	2
1.6. Alcance y limitaciones iniciales	2
2. Estado del arte	3
2.1. Mantenimiento predictivo y detección de anomalías	3
2.2. Modelos de detección de anomalías	3
2.3. Run-to-failure, indicadores de salud y modelos de reconstruccion	4
2.4. Sistemas multiagente aplicados a pipelines de datos	4
2.5. Datasets públicos de referencia	4
3. Arquitectura propuesta	5
3.1. Principios de diseño	5
3.2. Separación entre agentes y ejecutores	5
3.3. Nodos previstos	5
3.4. Estado global	6
3.5. Contratos estructurados	7
3.6. Trazabilidad	7
4. Metodología	8
4.1. Caso de uso industrial	8
4.2. Pipeline de datos	8
4.3. Principio rector agentico	8
4.4. Memoria agentica y RAG	9
4.5. Metodologia run-to-failure de Fase 9	9
4.6. Generalización multi-dataset	10
4.7. Control de reutilización técnica	11
4.8. Manifiesto del dataset	12
4.9. Extracción y perfilado	12
4.10. Estructuración temporal	12
4.11. Particiones	12
4.12. Métricas	13

5. Implementación	14
5.1. Estructura del repositorio	14
5.2. Estado Actual	14
5.3. Estado global implementado	14
5.4. Contratos Pydantic implementados	15
5.5. Artefactos previstos	15
5.6. Adaptadores de Entrada	15
5.7. Ejecutor de manifiesto	16
5.8. Ejecutor de perfilado	16
5.9. Ejecutor de Limpieza	16
5.10. Ejecutor de Estructuración Temporal	17
5.11. Ejecutor de Modelado Base	17
5.12. Ejecutor de Evaluación	17
5.13. Grafo LangGraph supervisado	18
5.14. Agente Limpiador LLM	19
5.15. Agente Estructurador LLM	19
5.16. Agente Modelador LLM	20
5.17. Agente Evaluador LLM	21
5.18. Agente Redactor LLM	22
5.19. Persistencia Local de Ejecuciones	23
5.20. Wrapper de Ejecucion Persistida	23
5.21. Registro Consultable de Ejecuciones	24
5.22. Protocolo Experimental Local	24
5.23. API Minima de Consulta	24
5.24. Contratos Iniciales Multi-Dataset	25
5.25. Prueba Agentica NASA IMS con Qwen	26
5.26. Benchmark NASA IMS Sintetico Etiquetado	27
5.27. Bucle de Reintento Agentico	27
5.28. API Controlada de Ejecucion	33
5.29. Dependencias	34
5.30. Verificación Actual	34
5.31. Siguiente Paso de Implementación	35
5.32. Fase 7: evidence pack e informe agentico	39
5.33. Fase 8: adaptacion agentica del perfil run-to-failure	47
5.34. Fase 9: cierre del perfil run-to-failure avanzado	52
6. Experimentos	54
6.1. Protocolo experimental	54
6.2. Plan CWRU Isolation Forest Threshold	54
6.3. Comparacion Agentica de Ventanas CWRU	55
6.4. Comparacion Agentica de Modelos CWRU	56
6.5. Prueba Smoke NASA IMS con Agentes Qwen	56
6.6. Benchmark NASA IMS Sintetico Etiquetado	57
6.7. Iteracion Agentica sobre NASA IMS Sintetico	57
6.8. Primer Experimento con Memoria Agentica	58
6.9. Memoria Transversal en una Run Completa CWRU	60
6.10. Validacion del Runner Comun Multi-dataset	61
6.11. Experimento de cierre de Fase 9	62

6.12. Validacion operativa de la aplicacion local	63
6.13. Experimentos posteriores	63
7. Resultados	65
7.1. Resultados cuantitativos	65
7.1.1. Baseline CWRU de Fase 2	65
7.1.2. Comparacion agentica de ventanas	65
7.1.3. Comparacion agentica de modelos	66
7.1.4. Benchmark NASA sintetico y reintentos	66
7.1.5. Primer resultado con memoria agentica	67
7.1.6. Resultado con memoria transversal	68
7.2. Resultados cualitativos	69
7.2.1. Resultado operativo de Fase 5	69
7.3. Resultados de Fase 9: run-to-failure avanzado	70
7.4. Limitaciones observadas	71
8. Conclusiones	72
8.1. Conclusiones principales	72
8.2. Limitaciones	73
8.3. Trabajo futuro	73

Índice de figuras

Índice de tablas

6.1.	Comparacion local de umbral con Isolation Forest sobre CWRU	55
6.2.	Comparacion agentica de ventanas sobre CWRU	55
6.3.	Comparacion agentica de modelos sobre CWRU	56
6.4.	Run agentica sobre benchmark NASA IMS sintetico etiquetado	57
6.5.	Reintentos agenticos sobre benchmark NASA IMS sintetico	57
6.6.	Reintento con memoria agentica sobre NASA IMS sintetico	58
6.7.	Cambio de familia guiado por memoria sobre NASA IMS sintetico	60
7.1.	Comparacion de umbral con Isolation Forest en el split de test CWRU.	65
7.2.	Comparacion agentica de tamanos de ventana en CWRU con 50 % de solape.	66
7.3.	Comparacion agentica de modelos sobre CWRU con ventana de 1024 muestras.	66
7.4.	Reintentos agenticos sobre benchmark NASA sintetico etiquetado. . . .	67
7.5.	Primera comparacion con memoria agentica sobre NASA sintetico. . . .	67
7.6.	Primer ciclo de memoria transversal en CWRU.	69
7.7.	Suite run-to-failure ligera con readiness para autoencoder.	70

Capítulo 1

Introducción

1.1. Contexto

La detección de anomalías en maquinaria industrial es una tarea relevante para mejorar la fiabilidad, reducir paradas no planificadas y apoyar estrategias de mantenimiento predictivo. En sistemas rotativos, como motores eléctricos, turbinas, bombas o rodamientos, las señales de vibración permiten observar cambios de comportamiento asociados a desgaste, desbalanceo, defectos mecánicos o degradación progresiva.

Este trabajo se centra inicialmente en rodamientos de motores eléctricos, usando como dataset principal CWRU Bearing Dataset [2]. El dataset NASA IMS Bearing se conserva como fuente futura para validar escenarios de degradación run-to-failure [7].

1.2. Motivación

El interés del proyecto no se limita a entrenar un modelo de detección de anomalías. El objetivo es construir una arquitectura capaz de coordinar el flujo completo de datos: ingestión, perfilado, limpieza, estructuración temporal, entrenamiento, evaluación, auditoría e informe técnico. Para evitar transformaciones opacas o no reproducibles, los agentes proponen decisiones estructuradas y los ejecutores deterministas aplican las operaciones reales.

1.3. Problema

El problema abordado es la detección automática de patrones anómalos en señales de vibración procedentes de rodamientos. En el MVP se formulará como una tarea binaria:

- clase normal: operación sin fallo;
- clase anómala: cualquier fallo de rodamiento disponible en el dataset.

La clasificación por tipo de fallo se mantiene como extensión posterior. Esta decisión reduce el alcance inicial y permite validar primero la trazabilidad del pipeline multiagente.

1.4. Objetivo general

Diseñar e implementar una arquitectura multiagente basada en LangGraph para procesar datos industriales de vibración y detectar anomalías de forma reproducible, auditable y extensible.

1.5. Objetivos específicos

1. Definir una estructura de proyecto que separe código, memoria académica y recursos.
2. Seleccionar un caso de uso industrial y datasets públicos adecuados.
3. Diseñar un pipeline de datos reproducible para señales de vibración.
4. Definir contratos estructurados para agentes y ejecutores.
5. Implementar ejecutores deterministas para perfilado, limpieza, estructuración temporal, modelado y evaluación.
6. Construir un MVP local antes de abordar Docker, SLURM o frontend.
7. Generar informes técnicos trazables a partir de métricas y artefactos.

1.6. Alcance y limitaciones iniciales

El alcance inicial queda limitado a ejecución local, ficheros crudos en disco y un dataset principal. No se abordarán en el MVP la ejecución distribuida, streaming industrial, integración SLURM, frontend ni persistencia avanzada. NASA IMS se mantiene como extensión experimental una vez validado el flujo sobre CWRU.

Capítulo 2

Estado del arte

2.1. Mantenimiento predictivo y detección de anomalías

Esta sección desarrollará el contexto de mantenimiento predictivo, monitorización de condición y detección de anomalías en maquinaria rotativa.

Aspectos a cubrir:

- diferencias entre mantenimiento correctivo, preventivo y predictivo;
- papel de las señales de vibración en rodamientos y motores;
- fallos comunes: pista interior, pista exterior, bola y desbalanceo;
- diferencias entre detección de anomalías, diagnóstico y pronóstico.

2.2. Modelos de detección de anomalías

El MVP comparará primero modelos clásicos y reproducibles:

- Isolation Forest;
- One-Class SVM;
- Local Outlier Factor;
- PCA con error de reconstrucción.

Los autoencoders densos y modelos recurrentes se considerarán en fases posteriores, cuando el pipeline de tensores esté validado.

En la extensión run-to-failure, la lectura de estos modelos cambia. Un detector no se evalúa solo por separar ventanas normales y anomalías, sino por producir una trayectoria de score que anticipe degradación, mantenga falsas alarmas nominales controladas y permita construir un indicador de salud interpretable. Por ello, PCA con error de reconstrucción, Isolation Forest, One-Class SVM y autoencoders densos se tratan como familias comparables dentro de un mismo protocolo temporal, no como modelos aislados optimizados por F1.

2.3. Run-to-failure, indicadores de salud y modelos de reconstrucción

Los datasets run-to-failure representan una trayectoria histórica de un activo hasta fallo. En este tipo de problema conviene distinguir cuatro conceptos: anomalía puntual, degradación sostenida, indicador de salud y pronóstico de vida útil restante. Una alerta aislada puede deberse a ruido o transitorios; la degradación defendible requiere persistencia temporal, tendencia del score y coherencia con el tramo nominal.

Los Health Indicators son habituales en mantenimiento predictivo porque condensan la evolución del activo en una curva de salud. En un sistema experimental local, este indicador debe declararse como evidencia auxiliar y no como predicción industrial cerrada. Su calidad puede evaluarse mediante caída entre tramo inicial y final, monotonía, robustez frente al ruido, volatilidad nominal y fuerza de tendencia durante la degradación.

Los autoencoders densos aportan una extensión natural para este escenario, porque aprenden a reconstruir normalidad y convierten el error de reconstrucción en un score de anomalía. Sin embargo, su uso exige suficientes ventanas nominales de entrenamiento, control de fuga temporal, arquitectura cerrada, evaluación comparable y guardarrailes que impidan presentar un modelo más complejo como mejora automática. En este TFM se incorporan como familia PyTorch reproducible sobre features tabulares, dejando los autoencoders recurrentes y el RUL completo como líneas futuras.

2.4. Sistemas multiagente aplicados a pipelines de datos

Esta sección describirá el uso de arquitecturas multiagente para dividir responsabilidades entre nodos especializados. El enfoque del proyecto separa:

- decisiones de alto nivel tomadas por agentes;
- validación de contratos mediante esquemas estrictos;
- ejecución determinista de transformaciones sobre datos;
- auditoría de rutas, configuraciones, métricas y artefactos.

2.5. Datasets públicos de referencia

El dataset principal será CWRU Bearing Dataset [2, 3, 1]. NASA IMS Bearing Dataset se reserva como extensión para escenarios de degradación temporal [7, 8].

Capítulo 3

Arquitectura propuesta

3.1. Principios de diseño

La arquitectura se basa en cuatro principios:

1. separar código, memoria académica y recursos;
2. mantener los datos crudos inmutables;
3. representar decisiones mediante contratos estructurados;
4. ejecutar transformaciones mediante módulos Python deterministas.

3.2. Separación entre agentes y ejecutores

Los agentes no escriben ni ejecutan código arbitrario sobre los datos. Su papel es proponer configuraciones, interpretar perfiles y decidir la siguiente fase del flujo. Los ejecutores leen artefactos desde disco, aplican operaciones deterministas y devuelven rutas, métricas y logs.

Para que agentes basados en LLM locales puedan actuar como expertos sin depender de conocimiento interno sobre datasets de nicho, la arquitectura introduce el patrón de expediente experto. Antes de cada decisión relevante, un ejecutor determinista resume la evidencia disponible, calcula diagnósticos, enumera opciones soportadas y explicita advertencias o bloqueos. El agente conserva libertad para escoger, ordenar alternativas, justificar su criterio o solicitar una capacidad no implementada, pero su respuesta sigue validándose mediante un contrato Pydantic.

Como extensión futura, se contempla un agente investigador controlado. Su papel no sería ejecutar transformaciones, sino responder solicitudes estructuradas de otros agentes mediante evidencia local o fuentes autorizadas, registrando citas, fecha, confianza y limitaciones. De este modo la arquitectura puede incorporar búsqueda documental o RAG sin conceder navegación libre ni capacidad de ejecutar acciones fuera de los contratos.

3.3. Nodos previstos

El grafo mínimo previsto contiene:

- supervisor;
- agente limpiador;
- ejecutor de limpieza;
- agente estructurador;
- ejecutor de estructuración;
- agente modelador;
- ejecutor de modelado;
- evaluador;
- redactor;
- revisión humana para trabajos costosos.

3.4. Estado global

El estado global de LangGraph no almacenará datasets completos. Debe contener rutas, resúmenes y artefactos:

- identificador de ejecución;
- contexto del proyecto;
- rutas de datos crudos, perfiles, datos limpios, tensores e informes;
- configuraciones de limpieza, estructuración y modelado;
- métricas y evaluación;
- errores, aprobaciones humanas y artefactos generados.

En la implementación inicial se ha definido un modelo canónico ‘TFMStateModel’ validado con Pydantic y una vista ‘TFMState’ basada en ‘TypedDict’ para su uso en LangGraph. Esta separación permite validar contratos de forma estricta antes de entregar el estado al grafo y, al mismo tiempo, mantener una representación ligera y serializable.

El estado inicial del caso CWRU queda fijado en la fase ‘dataset_manifest’, con el nodo siguiente ‘manifest_executor’. El directorio crudo se registra como `codigo/data/raw/cwru_bearing/mat`, el canal principal es ‘DE_time’ y la frecuencia objetivo es 12 kHz. Las rutas de artefactos posteriores permanecen como valores nulos hasta que los ejecutores correspondientes las generen.

3.5. Contratos estructurados

Los contratos iniciales incluyen:

- ‘ProjectContext’, que define dominio, máquina, dataset, objetivo, canal principal y frecuencia objetivo;
- ‘DatasetManifestRow’ y ‘DatasetManifest’, que formalizan el inventario de ficheros crudos y sus etiquetas;
- ‘DatasetProfileSummary’, que resume el perfil sin almacenar señales;
- ‘CleaningConfig’, ‘StructuringConfig’ y ‘ModelingConfig’, que fijan las configuraciones que podrán proponer los agentes;
- ‘SupervisorDecision’, ‘CleaningDecision’, ‘StructuringDecision’, ‘ModelingDecision’, ‘EvaluationDecision’ y ‘ReportDecision’, que estructuran las respuestas de los agentes;
- ‘ExecutorResult’ y resultados especializados, que normalizan las salidas de los ejecutores deterministas;
- ‘MetricsReport’ y ‘EvaluationResult’, que estructuran la evaluación;
- ‘PipelineError’, ‘HumanApproval’ y ‘ArtifactRef’, que mantienen trazabilidad de errores, revisiones y artefactos.

Estos esquemas aplican validación estricta y rechazan campos no declarados. Por ejemplo, una fila normal del manifiesto debe declarar los campos de fallo como nulos, mientras que una fila con fallo debe incluir tipo de fallo y diámetro. De forma análoga, un resultado de ejecutor con estado fallido debe incluir un error estructurado, y una decisión no terminal del supervisor debe indicar el siguiente nodo del grafo.

3.6. Trazabilidad

Cada ejecución debe permitir responder:

- qué datos se usaron;
- qué configuración fue validada;
- que ejecutor produjo cada artefacto;
- qué métricas se obtuvieron;
- qué limitaciones permanecen.

Capítulo 4

Metodología

4.1. Caso de uso industrial

El caso de uso seleccionado es la detección de anomalías en rodamientos de motores eléctricos mediante señales de vibración. La elección permite trabajar con datos industriales públicos, etiquetas interpretables y un flujo temporal adecuado para ventanas, features y tensores.

4.2. Pipeline de datos

El flujo metodológico para el MVP queda definido como:

```
datos .mat crudos
-> manifiesto del dataset
-> extracción de señales
-> normalización de frecuencia de muestreo
-> perfilado estadístico
-> limpieza determinista
-> ventanas temporales
-> features y tensores
-> particiones train/validation/test
-> modelos de anomalías
-> evaluación
-> informe técnico
```

4.3. Principio rector agentico

El objeto central de investigación de este TFM no es solo construir un pipeline clásico de detección de anomalías, sino estudiar hasta donde puede llegar una arquitectura multiagente basada en LLM locales, especialmente la familia Qwen, en un contexto industrial de monitorización de activos. Por tanto, los agentes mantienen el protagonismo metodológico: formulan hipótesis, seleccionan configuraciones, interpretan evidencias, proponen alternativas, debaten resultados, redactan conclusiones y dejan trazabilidad de sus decisiones.

Los componentes deterministas no sustituyen ese papel. Su función es dar a los agentes un entorno industrialmente defendible: contratos Pydantic, herramientas de

observacion, calculo reproducible de metricas, ejecucion segura de transformaciones y persistencia auditable de artefactos. En otras palabras, la parte determinista actua como instrumentacion, guardarrail y brazo ejecutor, mientras que la linea de investigacion evalua si los agentes pequenos pueden tomar decisiones utiles, consistentes y trazables sobre esa evidencia.

La eleccion de modelos LLM locales y relativamente pequenos es deliberada. No se plantea como una solucion comodamente sobredimensionada, sino como una hipotesis de viabilidad para entornos industriales reales, donde el coste, la latencia, la privacidad, la conectividad y la posibilidad de despliegue local son restricciones relevantes. Si el sistema llega a sus limites, esos limites formaran parte del resultado de investigacion; hasta entonces, la metodologia debe intentar llevar la linea agentica tan lejos como sea razonablemente posible sin abandonar seguridad, reproducibilidad ni trazabilidad.

4.4. Memoria agentica y RAG

La memoria se trata como una capacidad metodologica central, no como un simple historial de ejecuciones. Cada recuerdo debe representar una decision, una evidencia, una advertencia o una leccion reutilizable; debe poder recuperarse por un agente, citarse dentro de una decision estructurada y auditarse despues. Por ello se separan cuatro responsabilidades: los agentes deciden si usan la memoria, los contratos limitan que pueden declarar, los ejecutores persisten e indexan artefactos reproducibles y la interfaz debe mostrar el ciclo completo del recuerdo.

El baseline del TFM conserva una memoria vectorial local basada en JSON y embeddings Qwen cuando Ollama esta disponible. Esta eleccion facilita reproducibilidad, inspeccion manual y pruebas locales, pero se declara explicitamente como punto de partida, no como equivalente a una base vectorial industrial. La evolucion metodologica prevista compara ese baseline con tecnicas actuales de RAG: observabilidad de consultas, evaluacion de retrieval, reranking, busqueda hibrida, consolidacion de episodios y eventualmente una base vectorial externa. En todos los casos, la memoria debe informar a Qwen sin sustituir el protagonismo deliberativo de los agentes.

4.5. Metodologia run-to-failure de Fase 9

La Fase 9 fija una metodologia especifica para elevar el perfil `run_to_failure_degradation` desde una ejecucion temporal defendible hacia un caso local de mantenimiento predictivo agentico. La unidad de analisis deja de ser la ventana aislada y pasa a ser la trayectoria completa del activo. Por ello, el criterio principal no es F1, sino la deteccion de degradacion confirmada antes de fallo, el `lead time` persistente, las falsas alarmas nominales, la tendencia del score y la calidad del Health Indicator.

Esta metodologia introduce politicas versionadas y citables. La politica de alerta distingue pico aislado, vigilancia y aviso sostenido; la politica de Health Indicator produce una curva de salud causal y metricas de caida, monotonicidad y robustez; y la politica de readiness decide si existen datos suficientes para modelos temporales avanzados. Los agentes pueden consultar estas herramientas y citarlas en sus decisiones, pero las herramientas no aprueban runs, no eligen modelos y no estiman RUL.

El principio agentico se mantiene: Qwen/LLM conserva el protagonismo como decisor. El sistema le ofrece familias de modelo ejecutables y evidencias temporales, pero

no fuerza una elección. En particular, el autoencoder PyTorch denso se incorpora como candidato profesional y reproducible, no como reemplazo del agente ni como mejora garantizada. Para seleccionarlo, el modelador debe trabajar dentro del perfil run-to-failure, citar la herramienta de readiness, usar hiperparámetros cerrados y aceptar que el evaluador juzgara el resultado con las mismas métricas temporales que PCA, Isolation Forest y One-Class SVM.

También se refuerza la trazabilidad de fallos del LLM. Una salida que no cumple un contrato no debe aparecer como una caída opaca a fallback si puede repararse mediante una segunda emisión estructurada. Por ello, las violaciones de contrato se registran como correcciones de guardarrail, mientras que el fallback queda reservado para fallos técnicos del proveedor o JSON irrecuperable. Esta distinción es importante en un TFM centrado en agentes: una corrección de guardarrail demuestra control metodológico; un fallback técnico indica que el LLM no pudo participar.

4.6. Generalización multi-dataset

Tras consolidar el MVP sobre CWRU, la siguiente extensión metodológica consiste en separar explícitamente tres niveles: el dataset crudo, el adaptador determinista de dataset y el pipeline común. Esta separación evita que los ejecutores dependan de nombres de fichero o estructuras particulares de un benchmark concreto.

La generalización se plantea mediante un descriptor común de dataset, un manifiesto común con una fila por unidad cruda procesable y un registro de adaptadores. CWRU se mantendrá como benchmark de regresión, mientras que NASA IMS se contempla como primer candidato para validar el sistema con series más largas, múltiples canales y condiciones de degradación menos triviales.

La inspección local de NASA IMS muestra que el paquete se distribuye como un contenedor anidado ('zip', '7z' y 'rar') y que la aplicación deberá distinguir entre detección de formato, selección de adaptador y generación efectiva de manifiesto. En una futura interfaz, el usuario debería ver una previsualización del descriptor del dataset, los formatos detectados, las limitaciones de extracción y si el manifiesto puede generarse directamente o requiere preparar una carpeta preextraída.

Como primer paso ejecutable, el manifiesto NASA IMS se restringe a carpetas ya preextraídas. Cada snapshot temporal se representa como una fila común con etiqueta inicial desconocida, fallo final conocido a nivel de set y metadatos de frecuencia, canales y variante local. Esta decisión permite validar la generalización del pipeline sin incorporar todavía dependencias de extracción ni una política de etiquetado temporal.

El perfilado se adapta para leer tanto el manifiesto histórico de CWRU como el manifiesto común de NASA IMS. En este segundo caso, los snapshots ASCII sin extensión se tratan como tablas numéricas y se convierten en canales normalizados ('channel_1', 'channel_2', etc.), lo que permite obtener estadísticos ligeros sin cargar señales completas en el estado del grafo.

La limpieza se generaliza de forma acotada mediante una selección explícita de canal. El contrato 'CleaningConfig' permite indicar 'selected_channel' y el ejecutor valida esa elección contra los canales declarados en el manifiesto común. Así, NASA IMS puede limpiarse sobre un canal concreto sin habilitar transformaciones libres ni pasar todavía a estructuración o modelado.

Los agentes no interpretarán directamente carpetas crudas. Recibirán descriptores, resúmenes de manifiesto y diagnósticos de calidad generados por código determinista.

De este modo pueden aumentar su capacidad de decisión –por ejemplo, recomendando canal, ventana, familia de features o modelo– sin ejecutar código arbitrario ni saltarse los contratos del sistema.

Esta elección responde a una limitación metodológica importante: los LLM locales empleados no deben considerarse fuentes autosuficientes de conocimiento sobre datasets industriales específicos. Su viabilidad se apoya en externalizar el conocimiento de dominio en expedientes verificables: métricas, candidatos, opciones ejecutables, advertencias y capacidades no soportadas. Así, el agente actúa como experto deliberativo sobre evidencia proporcionada por el sistema, no como una caja negra que recuerda parámetros de memoria.

El mismo criterio se aplica a la estructuración temporal. En lugar de pedir al LLM que invente un tamaño de ventana, el sistema calcula alternativas soportadas, estima coste, número de ventanas y riesgos de fuga de información. El estructurador puede elegir entre configuraciones válidas y justificar el compromiso entre resolución temporal, coste y robustez experimental.

Para disponer de métricas supervisadas multi-dataset sin forzar etiquetas sobre NASA IMS real, se introduce además un benchmark sintético NASA IMS-like. Este benchmark conserva formato temporal, canales y frecuencia de muestreo compatibles con NASA IMS Set 2, pero genera etiquetas ‘normal’ y ‘fault’ por construcción. Su papel metodológico es probar decisiones agénticas, persistencia y evaluación sobre un escenario no trivial; no se interpreta como resultado oficial del dataset NASA IMS.

Sobre este benchmark se incorpora además una iteración agéntica acotada. Cuando una ejecución no supera los umbrales, el sistema genera un expediente de fallo con matriz de confusión, falsos negativos, falsos positivos, umbral efectivo y convención de puntuación. El agente modelador recibe ese expediente y decide, mediante un contrato estructurado, si propone un único reintento o si detiene el proceso. El número máximo de intentos queda fijado antes de la ejecución, de modo que el agente puede aprender de sus errores sin abrir una búsqueda ilimitada de hiperparámetros.

La revisión humana se plantea en esta fase como human-on-the-loop, no como una aprobación manual de resultados. Al finalizar una ejecución, el sistema puede presentar al revisor la hipótesis del agente, la evidencia usada, el cambio aplicado y el resultado observado. El humano etiqueta la calidad del razonamiento, por ejemplo como correcto, parcialmente correcto, incorrecto, inseguro o insuficiente. Esta etiqueta no modifica las métricas; se conserva como memoria supervisada para futuros prompts, RAG local o filtrado de comportamientos erróneos repetidos.

4.7. Control de reutilización técnica

Para evitar que la ampliación incremental genere contratos, scripts o servicios duplicados, se adopta una regla metodológica adicional: antes de implementar una nueva capacidad debe realizarse un inventario explícito de piezas existentes. El proceso consiste en describir la capacidad buscada, buscar en código, documentación y tests, identificar el propietario canónico de esa responsabilidad y decidir si se reutiliza, adapta, extiende o crea una pieza nueva.

Esta regla es especialmente relevante a partir de la Fase 4, donde conviven runners multi-dataset, memoria agéntica, API, revisión humana y políticas temporales por dataset. El objetivo no es frenar el desarrollo, sino preservar la trazabilidad arquitectónica: una capacidad debe vivir en una frontera clara y ser consumida por CLI,

API o futura interfaz sin duplicar lógica. Por ejemplo, la API no debe reimplementar la planificación multi-dataset, sino delegar en el runner común; del mismo modo, la revisión humana debe reutilizar los contratos existentes de configuración y aprobación.

4.8. Manifiesto del dataset

El primer artefacto metodológico será un manifiesto con una fila por fichero crudo. Este manifiesto conectará los ‘.mat’ originales con el resto del pipeline. Las columnas previstas son:

Campo	Descripción
file_id	identificador del fichero original
dataset	nombre del dataset
source_path	ruta del fichero crudo
label	clase ‘normal’ o ‘fault’
fault_type	tipo de fallo si existe
fault_diameter_inch	diámetro del defecto
load_hp	carga del motor
rpm	régimen de giro
sensor_channel	canal de sensor usado
source_sample_rate_hz	frecuencia de muestreo original
target_sample_rate_hz	frecuencia objetivo del MVP
source_format	formato original
notes	observaciones

4.9. Extracción y perfilado

El canal principal inicial será ‘DE_time’, asociado al acelerómetro del lado drive-end. El perfilado generará resúmenes estadísticos por fichero y por etiqueta, incluyendo número de muestras, canales detectados, valores no finitos, media, desviación típica, RMS, curtosis, factor de cresta y energía.

4.10. Estructuración temporal

La frecuencia objetivo inicial será 12 kHz. Las ventanas candidatas serán de 2048 o 4096 muestras con solapamiento del 50 %. Esta decisión permite comparar features tabulares con modelos clásicos y conservar ventanas crudas para autoencoders posteriores.

4.11. Particiones

Para evitar fuga de información entre ventanas, no se usará un split aleatorio ingenuo. La estrategia inicial será:

- entrenamiento con ventanas normales;

- validación con ventanas normales retenidas;
- test con ventanas normales retenidas y ventanas con fallo.

4.12. Métricas

Las métricas iniciales serán precision, recall, F1-score, ROC-AUC, PR-AUC, tasa de falsos positivos y matriz de confusión. Desde el punto de vista industrial, el criterio principal será mantener alta sensibilidad ante fallos sin disparar en exceso los falsos positivos.

Capítulo 5

Implementación

5.1. Estructura del repositorio

La implementación se organiza en tres áreas:

- ‘codigo/’: aplicación, tests, datos, modelos, informes y documentación técnica;
- ‘memoria/’: documento LaTeX del TFM;
- ‘recursos/’: PDFs, referencias y diagramas auxiliares.

5.2. Estado Actual

Hasta este punto se ha creado la estructura inicial de directorios, se han descargado los datasets crudos, se ha definido el flujo de datos del MVP y se han implementado el contrato inicial del estado global del grafo y los esquemas Pydantic de entradas, decisiones y resultados. Los datos crudos se conservan en ‘codigo/data/raw/’, junto con un README de trazabilidad.

5.3. Estado global implementado

El estado global se ha dividido en dos capas:

- ‘codigo/app/schemas/state.py’: modelos Pydantic estrictos para validar el estado, configuraciones, métricas, errores y artefactos;
- ‘codigo/app/graph/state.py’: ‘TypedDict’ usado por LangGraph, validador de estado e inicializador del caso CWRU.

El inicializador ‘create_initial_cwru_state’ crea una ejecución preparada para generar el manifiesto del dataset. El estado resultante no contiene señales ni arrays, solo rutas, contexto experimental y campos reservados para artefactos futuros.

5.4. Contratos Pydantic implementados

Se han añadido tres grupos de contratos:

- `‘codigo/app/schemas/dataset.py‘`: manifiesto del dataset, canales y metadatos de fallo;
- `‘codigo/app/schemas/agent_decisions.py‘`: decisiones estructuradas del supervisor, limpiador, estructurador, modelador, evaluador y redactor;
- `‘codigo/app/schemas/executor_results.py‘`: resultados comunes y especializados de ejecutores deterministas.

Estos contratos consolidan la regla arquitectónica de que los agentes producen configuraciones o decisiones validables, mientras que los ejecutores devuelven rutas, métricas, errores y referencias a artefactos, pero no datos pesados.

5.5. Artefactos previstos

- `‘codigo/data/interim/cwru_bearing/manifest.csv‘`;
- `‘codigo/data/interim/cwru_bearing/profile.json‘`;
- `‘codigo/data/processed/cwru_bearing/clean_signals/‘`;
- `‘codigo/data/tensors/cwru_bearing/windows_features.csv‘`;
- `‘codigo/data/tensors/cwru_bearing/windows_raw.npz‘`;
- `‘codigo/data/tensors/cwru_bearing/splits.json‘`;
- `‘codigo/reports/cwru_bearing/‘`;
- `‘codigo/models/cwru_bearing/‘`.

5.6. Adaptadores de Entrada

Para reducir el acoplamiento al formato CWRU se ha introducido una capa ligera de adaptadores en `‘codigo/app/services/signal_adapters.py‘`. Esta capa expone dos operaciones comunes: leer todos los canales numéricos de un fichero y cargar un canal concreto como vector. Los ejecutores de perfilado y limpieza usan ahora esta frontera en lugar de leer directamente ficheros `‘.mat‘`.

La implementación inicial soporta `‘.mat‘`, `‘.csv‘`, `‘.txt‘`, `‘.tsv‘` y `‘.npz‘`. En el caso de CWRU, el adaptador reconoce los canales `‘DE_time‘`, `‘FE_time‘`, `‘BA_time‘` y `‘RPM‘`; en formatos tabulares, interpreta columnas numéricas como canales. Para incorporar un nuevo origen, como NASA IMS, el objetivo será crear o ampliar un adaptador de entrada y mantener estable el resto del pipeline: perfilado, limpieza, estructuración, modelado y evaluación.

5.7. Ejecutor de manifiesto

Se ha implementado el primer ejecutor determinista del pipeline, el módulo ‘data-set_manifest.py’. Su responsabilidad es generar el artefacto:

```
codigo/data/interim/cwru_bearing/manifest.csv
```

a partir de los ficheros ‘.mat’ crudos de CWRU.

El ejecutor no carga señales completas. Solo inspecciona los nombres de fichero y aplica un mapeo controlado de metadatos CWRU: carga, RPM, tipo de fallo, diámetro y posición del fallo exterior cuando corresponde. El manifiesto generado conserva la frecuencia original de cada fichero y fija una frecuencia objetivo de 12 kHz para el MVP.

La ejecución real sobre los datos descargados ha generado 64 registros: 4 ficheros normales y 60 ficheros con fallo. El fichero resultante contiene 65 líneas incluyendo la cabecera.

5.8. Ejecutor de perfilado

Se ha implementado el perfilador determinista ‘data_profiler.py’. Este ejecutor lee el manifiesto, inspecciona los ficheros ‘.mat’ y genera:

```
codigo/data/interim/cwru_bearing/profile.json
```

El perfil contiene metadatos y estadísticas por canal: número de muestras, valores no finitos, mínimo, máximo, media, desviación típica, RMS, curtosis y energía. No almacena arrays ni señales completas.

La ejecución real ha perfilado 64 ficheros. El resumen agregado contiene 4 ficheros normales, 60 ficheros con fallo, frecuencias de muestreo de 48 kHz y 12 kHz, y los canales detectados ‘BA_time’, ‘DE_time’, ‘FE_time’ y ‘RPM’.

5.9. Ejecutor de Limpieza

Se ha implementado el ejecutor determinista ‘cleaning.py’. Este ejecutor lee el manifiesto y el perfil, extrae el canal principal configurado, elimina valores no finitos, remuestrea la señal a 12 kHz cuando procede y guarda una señal limpia comprimida por fichero:

```
codigo/data/processed/cwru_bearing/clean_signals/
```

También genera un resumen de auditoría en:

```
codigo/data/processed/cwru_bearing/cleaning_summary.json
```

La ejecución real ha producido 64 ficheros ‘.npz’ comprimidos. Los cuatro ficheros normales se han remuestreado de 48 kHz a 12 kHz; los ficheros de fallo ya estaban a 12 kHz. El estado del grafo conserva únicamente la ruta del directorio limpio y las referencias a artefactos, no las señales.

5.10. Ejecutor de Estructuración Temporal

Se ha implementado el ejecutor determinista ‘structuring.py’. Este ejecutor lee las señales limpias, valida que el canal y la frecuencia coincidan con la configuración, genera ventanas temporales con tamaño 2048 y solapamiento del 50 %, calcula features temporales iniciales y crea particiones reproducibles a nivel de fichero.

```
codigo/data/tensors/cwru_bearing/windows_features.csv
codigo/data/tensors/cwru_bearing/windows_raw.npz
codigo/data/tensors/cwru_bearing/splits.json
```

La estrategia de partición evita mezclar ventanas de un mismo fichero entre subconjuntos. Las ventanas normales se reparten entre entrenamiento, validación y test, mientras que las ventanas con fallo se reservan para test. La ejecución real ha generado 7460 ventanas de 2048 muestras: 175 de entrenamiento, 117 de validación y 7168 de test. El tensor crudo tiene forma ‘(7460, 2048)’ y el directorio de tensores ocupa aproximadamente 22 MB.

5.11. Ejecutor de Modelado Base

Se ha implementado el ejecutor determinista ‘modeling.py’ con el primer modelo base de detección de anomalías: Isolation Forest. El modelo se entrena únicamente con ventanas normales del subconjunto de entrenamiento y consume las features temporales generadas por la fase anterior.

```
codigo/models/cwru_bearing/isolation_forest.joblib
codigo/models/cwru_bearing/predictions.csv
codigo/models/cwru_bearing/modeling_summary.json
```

El artefacto ‘joblib’ conserva el modelo entrenado, las columnas usadas y el umbral de decisión. Las predicciones guardan, por cada ventana, identificador, fichero original, partición, etiqueta real, puntuación de anomalía, umbral y predicción binaria. Las métricas finales no se calculan en este ejecutor, sino que se reservan para la fase de evaluación.

La ejecución real ha entrenado el modelo con 175 ventanas normales y ha generado 7460 predicciones. El umbral estimado ha sido 0.5992 y el resumen de modelado registra 398 ventanas predichas como normales y 7062 como anómalas. Estos conteos son una salida preliminar del modelo, no una evaluación final de rendimiento.

5.12. Ejecutor de Evaluación

Se ha implementado el ejecutor determinista ‘evaluation.py’. Este módulo lee las predicciones del modelo, compara la etiqueta binaria real con la predicción de anomalía y calcula métricas técnicas por partición. La partición principal para el MVP es ‘test’.

```
codigo/reports/cwru_bearing/evaluation/metrics.json
codigo/reports/cwru_bearing/evaluation/evaluation_summary.md
```

La ejecución real sobre el split de test ha obtenido precision 0.9991, recall 1.0000, F1-score 0.9996, ROC-AUC 1.0000, PR-AUC 1.0000 y una tasa de falsos positivos de 0.0513. La matriz de confusión ha sido: 111 verdaderos negativos, 6 falsos positivos, 0 falsos negativos y 7051 verdaderos positivos.

Estos resultados deben interpretarse como validación del pipeline y del baseline inicial sobre un benchmark controlado. No constituyen todavía una garantía de generalización industrial, que se deberá analizar con validaciones más exigentes y, posteriormente, con otros datasets como NASA IMS.

5.13. Grafo LangGraph supervisado

Se ha implementado el primer grafo LangGraph del MVP en ‘pipeline.py’ y se ha incorporado un supervisor en el modulo:

```
codigo/app/agents/supervisor.py
```

La version actual ya contempla dos modos: un supervisor LLM que emite una ‘SupervisorDecision’ validada por Pydantic y una politica determinista de respaldo para pruebas, ejecuciones locales sin modelo y recuperacion ante respuestas invalidas.

El grafo ya no avanza directamente de un ejecutor al siguiente. Todas las transiciones vuelven al supervisor:

```
START
-> supervisor
-> manifest_executor
-> supervisor
-> profiler_executor
-> supervisor
-> cleaner_agent
-> supervisor
-> cleaning_executor
-> supervisor
-> structuring_agent
-> supervisor
-> structuring_executor
-> supervisor
-> modeling_agent
-> supervisor
-> modeling_executor
-> supervisor
-> evaluator
-> supervisor
-> evaluation_agent
-> supervisor
-> report_writer
-> supervisor
-> END
```

El grafo valida el estado entrante y saliente con ‘TFMStateModel’, registra mensajes de trazabilidad, acumula referencias a artefactos y detiene la ejecucion de forma

controlada si un ejecutor devuelve estado ‘failed’. Cada decision del supervisor se almacena en ‘messages’ con rol ‘supervisor’ y contenido JSON serializado. El servicio LLM comun vive en:

```
codigo/app/services/llm.py
```

La integracion inicial usa un cliente JSON para Ollama configurable por variables de entorno. Si se activa ‘TFM_SUPERVISOR_MODE=llm’, el supervisor solicita al modelo una decision estructurada; si no hay modelo disponible o la respuesta no valida, se conserva la decision determinista como fallback.

Incluso cuando decide el LLM, la decision queda acotada por validaciones posteriores: la fase actual debe coincidir con el estado real, no se aceptan nodos fuera del plan del MVP, la revision humana permanece desactivada en esta fase local y cualquier salto invalido fuerza el fallback. De este modo se introduce toma de decisiones agentica sin permitir que el modelo ejecute codigo ni manipule datos directamente.

5.14. Agente Limpiador LLM

Se ha implementado el primer agente LLM especializado en:

```
codigo/app/agents/cleaner.py
```

El agente limpiador lee un resumen ligero del perfil del dataset y devuelve una ‘CleaningDecision’ validada por Pydantic. Su decision queda guardada como ‘cleaning_config’ en el estado, pero la transformacion real de las senales sigue delegada exclusivamente en ‘codigo/app/executors/cleaning.py’.

La fase de limpieza queda dividida en decision y ejecucion:

```
supervisor
-> cleaner_agent
-> supervisor
-> cleaning_executor
```

Con Ollama local y el modelo ‘qwen3.5:4b’, se han validado respuestas reales para el supervisor y para el limpiador. El limpiador ha propuesto ‘strategy_id = cwru_clean_llm_v1’, eliminacion de no finitos, remuestreo a 12 kHz y normalizacion ‘none’, cumpliendo el contrato ‘CleaningDecision’.

5.15. Agente Estructurador LLM

Se ha implementado el agente estructurador LLM en:

```
codigo/app/agents/structurer.py
```

Este agente decide la estructuracion temporal del dataset: tamano de ventana, solapamiento, canal principal, frecuencia objetivo, modo de etiquetas, features temporales y rutas esperadas de salida. Su salida principal es una ‘StructuringDecision’ validada por Pydantic. Igual que en el resto de fases, el agente no genera ventanas ni tensores; la ejecucion real permanece en ‘codigo/app/executors/structuring.py’.

La fase de estructuracion queda dividida en decision y ejecucion:

```
supervisor
-> structuring_agent
-> supervisor
-> structuring_executor
```

Con ‘qwen3.5:4b’, el estructurador ha generado una decision valida con ventanas de 2048 muestras, solapamiento del 50 %, canal ‘DE_time’, frecuencia de 12 kHz, modo ‘binary_anomaly’ y features temporales trazables como RMS, curtosis, crest factor y energia.

En la Fase 3 se ha ampliado ‘StructuringDecision’ con ‘comparison_candidates’. Este campo permite que el agente no solo proponga una configuracion final, sino tambien alternativas comparables con su justificacion y efecto esperado. La aplicacion no inventa esas alternativas: si el agente no las devuelve, el plan comparativo no se construye.

Esta ampliacion se materializa en el script:

```
python -m codigo.scripts.run_cwru_agentic_window_comparison
```

El script solicita a Qwen una decision de estructuracion con alternativas y, solo despues, convierte esas propuestas en un plan experimental ejecutable con el protocolo local. De este modo se conserva la separacion entre decision agentica y ejecucion reproducible, pero se evita que una regla interna suplante la deliberacion del agente.

Durante la integracion se ha detectado que ‘checkpoint_id’ es un nombre reservado por LangGraph 1.1. Por ello se ha retirado de la vista ‘TypedDict’ que circula por el grafo. La persistencia futura debera apoyarse en los checkpoints propios de LangGraph o en un campo de aplicacion que no colisione con nombres reservados.

5.16. Agente Modelador LLM

Se ha implementado el agente modelador LLM en:

```
codigo/app/agents/modeler.py
```

Este agente decide el modelo base y sus hiperparametros a partir del estado ligero del pipeline, la configuracion de estructuracion y un resumen de la tabla de features. Su salida principal es una ‘ModelingDecision’ validada por Pydantic. El agente no entrena modelos ni genera predicciones; la ejecucion real se mantiene en ‘codigo/app/executors/modeling.py’.

La fase de modelado queda dividida en decision y ejecucion:

```
supervisor
-> modeling_agent
-> supervisor
-> modeling_executor
```

Aunque el contrato ‘ModelingConfig’ contempla algoritmos futuros como One-Class SVM, LOF o autoencoders, la implementacion solo permite al agente seleccionar modelos con ejecutor reproducible. Inicialmente se incluyo ‘isolation_forest’; en Fase 3 se ha anadido PCA con error de reconstruccion como segundo modelo soportado. Tambien se acotan la semilla (‘random_state = 42’), el split de entrenamiento (‘train’),

el uso del split de validacion para el umbral y los hiperparametros permitidos, evitando que el LLM seleccione opciones que la aplicacion aun no puede ejecutar de forma reproducible.

Con ‘qwen3.5:4b’, el modelador ha generado una decision valida con ‘model_name = isolation_forest’, ‘random_state = 42’, entrenamiento sobre el split ‘train’, validacion sobre ‘validation’, ‘n_estimators = 200’, ‘n_jobs = 1’ y ‘threshold_quantile = 0.99’.

El contrato de decision del modelador incorpora ahora una lista de alternativas comparables. Este campo permite que el agente proponga otros modelos o umbrales, que despues se convierten en planes experimentales si estan soportados por el ejecutor. Con ello se aumenta la capacidad de decision del agente sin permitir modelos no implementados.

Durante esta validacion se ha ajustado el cliente de Ollama para usar ‘think=false’ por defecto en llamadas JSON estrictas, ya que este modelo puede devolver el razonamiento en el campo ‘thinking’ y dejar vacio ‘message.content’ si no se desactiva ese modo. Esta opcion no queda fijada de forma rigida: puede activarse u omitirse mediante:

```
TFM_LLM_THINK=true
TFM_LLM_THINK=auto
```

5.17. Agente Evaluador LLM

Se ha implementado el agente evaluador LLM en:

```
codigo/app/agents/evaluator.py
```

Este agente interpreta las metricas calculadas por el ejecutor determinista de evaluacion y devuelve una ‘EvaluationDecision’ validada por Pydantic. La separacion es deliberada: ‘codigo/app/executors/evaluation.py’ calcula precision, recall, F1, ROC-AUC, PR-AUC, tasa de falsos positivos y matriz de confusion; el agente solo emite el juicio tecnico, la aprobacion y las limitaciones del experimento.

La fase de evaluacion queda dividida en calculo y juicio agentico:

```
supervisor
-> evaluator
-> supervisor
-> evaluation_agent
-> supervisor
```

Para el MVP CWRU se han fijado umbrales conservadores y explicitos:

```
min_recall_required = 0.90
max_false_positive_rate = 0.10
```

El LLM puede redactar el resumen tecnico y las limitaciones, pero no puede aprobar una ejecucion que incumpla esos umbrales. Si la respuesta no valida o contradice las metricas, se activa un fallback determinista.

Tras esta decision, el supervisor no interpreta una metrica baja como fallo de infraestructura. Si existen metricas, enruta la fase final hacia el agente redactor y el

informe se genera igualmente. La aprobacion experimental queda codificada en ‘evaluation.approved’: una run puede terminar como ‘completed’ y al mismo tiempo quedar ‘approved=false’ con limitaciones explicitas. El estado ‘failed’ se reserva para errores de contratos, artefactos o ejecutores. El fichero de metricas se genera en:

```
codigo/reports/cwru_bearing/evaluation/metrics.json
```

5.18. Agente Redactor LLM

Se ha implementado el agente redactor LLM en:

```
codigo/app/agents/report_writer.py
```

Este agente decide la estructura del informe tecnico final a partir del estado ligero del pipeline, las metricas, la evaluacion y las referencias a artefactos. Su salida principal es una ‘ReportDecision’ validada por Pydantic. El agente no escribe el informe directamente ni modifica resultados previos: la generacion material del documento se delega en el ejecutor determinista:

```
codigo/app/executors/reporting.py
```

La fase final queda dividida en decision y materializacion del informe:

```
supervisor  
-> report_writer  
-> supervisor  
-> END
```

El contrato acota la salida a formato Markdown y, en Fase 3, calcula la ruta a partir del dataset y del identificador de ejecucion para evitar sobrescrituras entre runs:

```
codigo/reports/<dataset>/<run_id>/final_report.md
```

Tambien se exigen secciones minimas: resumen ejecutivo, contexto y datos, configuraciones del pipeline, metricas y evaluacion, artefactos generados y limitaciones. Las rutas fuente propuestas por el LLM deben existir previamente en el estado como manifiesto, perfil, datos procesados, tensores, metricas o artefactos registrados. Si la respuesta no valida, se aplica un fallback determinista que genera un informe reproducible.

El resultado del ejecutor es un ‘ReportExecutorResult’ y un artefacto de tipo ‘report’. Con ello el pipeline cierra el ciclo completo del MVP: datos crudos, perfilado, limpieza, estructuracion, modelado, evaluacion, juicio agentic e informe tecnico trazable.

5.19. Persistencia Local de Ejecuciones

Como primer hito de la segunda fase se ha implementado una persistencia local de ejecuciones en:

```
codigo/app/services/run_persistence.py
```

El objetivo es conservar una evidencia ligera y auditable de cada ejecucion sin introducir todavia una base de datos externa. El servicio recibe un ‘TFMStateModel’ final y genera una carpeta por ‘run_id’ con el estado final, las decisiones extraidas de los mensajes del grafo, los artefactos, las metricas, la evaluacion, un resumen Markdown y una metadata del snapshot.

```
codigo/reports/runs/<run_id>/state_final.json
codigo/reports/runs/<run_id>/decisions.json
codigo/reports/runs/<run_id>/artifacts.json
codigo/reports/runs/<run_id>/metrics.json
codigo/reports/runs/<run_id>/evaluation.json
codigo/reports/runs/<run_id>/summary.md
codigo/reports/runs/<run_id>/snapshot.json
```

Ademas, se mantiene un indice local:

```
codigo/reports/runs/index.json
```

Esta decision mantiene separado el pipeline puro de la persistencia. La funcion ‘run_cwru_pipeline’ no se modifica; la integracion automatica se realiza mediante un wrapper de ejecucion persistida. De este modo se conserva la estabilidad del grafo ya validado y se prepara la comparacion de ejecuciones para los experimentos de la Fase 2.

5.20. Wrapper de Ejecucion Persistida

El segundo hito de la Fase 2 incorpora el wrapper:

```
run_and_persist_cwru_pipeline(...)
```

Esta funcion ejecuta primero el grafo CWRU con ‘run_cwru_pipeline’, valida el estado final y despues invoca ‘save_run_snapshot’. El resultado se devuelve como un objeto ‘PersistedPipelineRun’, compuesto por el estado final y la metadata del snapshot persistido.

La decision de implementarlo como wrapper, y no como nodo adicional del grafo, mantiene dos rutas claramente separadas: una ejecucion pura del pipeline y una ejecucion con persistencia automatica. Esto evita cambiar el comportamiento ya validado del MVP y permite que los tests sigan inyectando ejecutores y agentes simulados.

La validacion real con CWRU genera una carpeta en:

```
codigo/reports/runs/<run_id>/
```

con 13 artefactos, 17 decisiones y ningun error para una ejecucion completada. Este punto deja preparada la siguiente fase tecnica: construir un registro consultable y comparaciones entre ejecuciones persistidas.

5.21. Registro Consultable de Ejecuciones

El tercer hito de la Fase 2 introduce una capa de consulta sobre las ejecuciones persistentes:

```
codigo/app/services/run_registry.py
```

Este servicio lee el índice local de ejecuciones y los snapshots generados por la persistencia. Permite listar runs, filtrar por dataset, estado final o aprobación, cargar la metadata de una ejecución, recuperar sus artefactos y comparar métricas principales entre dos o más ejecuciones.

La comparación se devuelve mediante contratos Pydantic propios: ‘RunComparison’, ‘RunComparisonRow’ y ‘MetricComparison’. Para precisión, recall y F1-score se considera mejor el valor más alto, mientras que para la tasa de falsos positivos se considera mejor el valor más bajo. Esta distinción permite comparar experimentos con criterio técnico coherente con el dominio industrial.

El registro no recalcula métricas ni vuelve a ejecutar el pipeline; solo consulta evidencia ya persistente. Esta separación prepara la futura API mínima, que podrá exponer consultas sobre ejecuciones sin acoplarse a LangGraph ni a los ejecutores.

5.22. Protocolo Experimental Local

El cuarto hito de la Fase 2 incorpora un servicio para ejecutar planes experimentales locales:

```
codigo/app/services/experiment_protocol.py
```

Este servicio define contratos ‘ExperimentPlan’, ‘ExperimentSpec’, ‘ExperimentRunSummary’ y ‘ExperimentPlanResult’. A partir de un plan validado, ejecuta varios runs mediante el wrapper persistente, guarda los artefactos de modelado, evaluación e informe en carpetas separadas por experimento y genera una comparación reutilizando ‘run_registry.py’.

El primer plan real, ‘cwru_iforest_threshold_v1’, compara el baseline de Isolation Forest con una variante que solo modifica ‘threshold_quantile’ de 0.99 a 1.00. Los resultados se guardan en:

```
codigo/experiments/cwru_local/cwru_iforest_threshold_v1/
```

La salida incluye ‘experiment_plan.json’, ‘comparison.json’ y ‘results_table.md’. Esta estructura permite llevar resultados comparables a la memoria sin recalcular métricas ni mezclar artefactos de diferentes ejecuciones.

5.23. API Mínima de Consulta

El sexto hito de la Fase 2 incorpora una primera API FastAPI sobre la persistencia local:

```
codigo/app/api/app.py
codigo/app/api/routes.py
```

La API se crea mediante `'create_app(runs_dir=...)'`, de modo que en ejecución real consulta `'codigo/reports/runs/'` y en tests puede trabajar contra un directorio temporal. Esta primera versión es deliberadamente de lectura: permite consultar runs persistidos, recuperar su metadata, listar artefactos y servir el informe Markdown asociado. También expone la comparación de ejecuciones ya persistidas, reutilizando el registro consultable sin recalcular métricas ni relanzar el pipeline.

Los endpoints iniciales son:

```
GET /health
GET /runs
GET /runs/compare
GET /runs/{run_id}
GET /runs/{run_id}/artifacts
GET /runs/{run_id}/report
```

El endpoint `'POST /runs'` queda pospuesto hasta definir políticas de ejecución, aprobación y control de coste. Esta decisión mantiene separada la capa de consulta de la lógica que podría lanzar trabajos de modelado o experimentos múltiples.

5.24. Contratos Iniciales Multi-Dataset

Como primer paso de la Fase 3 se han incorporado contratos comunes para desacoplar el pipeline del caso CWRU sin modificar todavía los ejecutores ya validados. En `'codigo/app/schemas/dataset.py'` se han añadido `'DatasetDescriptor'`, `'DatasetAdapterInfo'` y `'CommonManifestRecord'`. Estos modelos permiten describir datasets heterogéneos, registrar adaptadores y representar filas de manifiesto comunes sin almacenar señales completas.

También se ha creado el servicio `'codigo/app/services/dataset_adapters.py'`, que expone un registro local de adaptadores. La primera versión incluye adaptadores para CWRU, NASA IMS y señales tabulares genéricas. Esta capa solo inspecciona rutas y cabeceras ligeras; no modifica datos crudos ni lanza transformaciones costosas.

La decisión mantiene el flujo CWRU existente como benchmark de regresión. El adaptador `'cwru_bearing'` se ha integrado como envoltorio del manifiesto actual mediante una función común de generación de manifiestos, delegando internamente en el ejecutor CWRU ya verificado. Los tests comprueban que el CSV generado por esta ruta común coincide con el manifiesto producido por la ruta histórica.

Para NASA IMS, el adaptador reconoce rutas o ficheros con patrón temporal compatible, infiere canales desde ficheros sintéticos pequeños y registra advertencias cuando detecta recuentos de canales inconsistentes. Tras la inspección local, se ha implementado `'build_manifest(...)'` solo para carpetas preextraídas. Esta función genera un manifiesto común con una fila por snapshot, `'label = unknown'`, fallo final conocido a nivel de set, frecuencia de 20 kHz, canales serializados como JSON y metadatos escalares sobre puntos por fichero, duración nominal, carga, régimen y variante local.

La inspección local ha confirmado que el paquete disponible se organiza como `'zip ->7z ->rar'`, con tres archivos RAR de experimentos y un README interno. El README describe tres ensayos test-to-failure con snapshots ASCII de un segundo, 20.480 puntos por fichero y frecuencia de muestreo de 20 kHz. También se ha detectado una discrepancia local en el tercer RAR, que lista más ficheros y una ruta interna distinta a la descrita por el README. Por ello, la primera implementación de manifiesto

NASA IMS trabaja con carpetas preextraídas y sintéticas antes de procesar el paquete completo. La ingestión directa del contenedor anidado queda pospuesta hasta decidir dependencias y experiencia de interfaz.

El perfilador `'data_profiler.py'` se ha generalizado para detectar si el CSV de entrada sigue el formato histórico CWRU o el contrato común. Para NASA IMS, propaga `'record_id'`, `'run_id'`, `'condition_id'`, `'primary_channel'`, canales de manifiesto y metadatos JSON. La capa `'signal_adapters.py'` también reconoce ficheros IMS con nombre temporal y sin extensión, expone `'read_signal_frame'` para leerlos como tablas numéricas con `'pandas'` y asigna canales normalizados `'channel_1'`, `'channel_2'`, etc. Esta adaptación permite generar perfiles estadísticos sobre estructuras NASA preextraídas.

El perfil incluye además un bloque `'decision_summary'` orientado al agente limpiador. Este bloque clasifica si el perfil está limpio, requiere remuestreo, requiere seleccionar canal o presenta calidad insuficiente. También enumera acciones requeridas, canales candidatos, opciones de limpieza soportadas y advertencias bloqueantes o no bloqueantes. De este modo, el LLM local no necesita conocer de memoria el dataset industrial: recibe un expediente experto calculado por el sistema y decide dentro de un espacio de acción validado.

El ejecutor `'cleaning.py'` también se ha generalizado para consumir manifiestos comunes. El contrato `'CleaningConfig'` incorpora `'selected_channel'`; si se declara, el ejecutor carga ese canal mediante la capa de adaptadores de señal y lo valida contra `'channel_names'` en los manifiestos comunes. Si no se declara, mantiene compatibilidad con `'sensor_channel'` para CWRU y con `'primary_channel'` para NASA IMS. La salida `'.npz'` conserva metadatos ligeros como `'record_id'`, `'dataset'`, `'run_id'`, `'condition_id'` y `'label_detail'`, manteniendo la estructura común necesaria para estructuración posterior.

El ejecutor `'structuring.py'` incorpora un resumen experto para el agente estructurador. A partir de las señales limpias estima configuraciones soportadas de ventana y solape, número aproximado de ventanas, coste relativo, conjuntos de features temporales disponibles y advertencias de fuga de información por fichero, run o secuencia temporal. El agente puede elegir entre estas configuraciones, pero la validación mantiene los valores dentro de rangos controlados y las features dentro de las implementadas por el ejecutor.

5.25. Prueba Agentica NASA IMS con Qwen

Para validar el comportamiento de los agentes locales ante un dataset distinto de CWRU se ha añadido un script de prueba controlada:

```
python -m codigo.scripts.run_nasa_ims_qwen_smoke \
  --model qwen3.5:4b \
  --run-id nasa-ims-qwen-smoke-fase3-qwen
```

La prueba crea una carpeta NASA IMS sintética preextraída, ejecuta el grafo completo con agentes Qwen/Ollama y persiste el snapshot en:

```
codigo/reports/runs/nasa-ims-qwen-smoke-fase3-qwen/
```

El objetivo de esta ejecución no es entrenar un modelo NASA IMS, sino comprobar si los agentes usan los expedientes expertos. El limpiador selecciona `'channel_1'` como

canal principal, el estructurador eligio ventanas de 1024 muestras con solape del 50 % y features temporales soportadas, y el modelador propuso ‘isolation_forest’, coherente con el unico ejecutor de modelado disponible en ese momento.

La ejecucion genero manifiesto, perfil, señales limpias, features, tensores y splits. Sin embargo, el flujo termino en estado ‘failed’ por una barrera metodologica deliberada: NASA IMS no dispone todavia de etiquetas por ventana ni de una politica de particion temporal defendible. Esta prueba confirma que los agentes locales pueden tomar decisiones utiles con contexto experto, pero tambien que debe formalizarse una decision de capacidad no soportada antes de permitir el paso al modelador en datasets run-to-failure.

5.26. Benchmark NASA IMS Sintetico Etiquetado

Para obtener metricas supervisadas sin atribuir etiquetas por ventana al NASA IMS real, se ha añadido un generador sintetico controlado:

```
codigo/app/services/synthetic_nasa_ims.py
codigo/scripts/run_nasa_ims_synthetic_temporal_benchmark.py
```

El generador crea una estructura compatible con NASA IMS Set 2: ficheros con nombre temporal, cuatro canales tabulares, frecuencia de 20 kHz y un manifiesto comun con etiquetas ‘normal’ y ‘fault’ generadas por construccion. Esta distincion es importante: el benchmark valida el flujo multi-dataset y las decisiones agenticas, pero no representa anotaciones oficiales del dataset NASA IMS.

La ejecucion canonica con Qwen se lanza mediante:

```
python -m codigo.scripts.run_nasa_ims_synthetic_temporal_benchmark \
--model qwen3.5:4b \
--run-id nasa-ims-synth-agentic-qwen-fase3
```

La run deja snapshot persistido en ‘codigo/reports/runs/nasa-ims-synth-agentic-qwen-fase3/'. Qwen selecciono ‘channel_1’, eligio ventanas de 2048 muestras con 50 % de solape, propuso Isolation Forest y, finalmente, el agente evaluador rechazo la ejecucion por recall insuficiente y FPR superior al umbral local. Esta salida es metodologicamente valiosa porque introduce un caso no perfecto y obliga a una iteracion posterior de decisiones, en lugar de cerrar todos los experimentos con recall 1.0000.

5.27. Bucle de Reintento Agentico

Para convertir una ejecucion rechazada en aprendizaje trazable se ha añadido un servicio de analisis de fallo y un contrato especifico de reintento:

```
codigo/app/services/iteration_analysis.py
codigo/scripts/run_nasa_ims_agentic_retry.py
ModelingRetryDecision
```

El servicio ‘iteration_analysis.py’ resume la ejecucion fallida sin exponer el dataset completo al LLM: calcula matriz de confusion, falsos negativos, falsos positivos, estadisticos de puntuacion, umbral efectivo y reintentos restantes. El agente modelador recibe ese expediente y debe devolver JSON validable con ‘should_retry’, ‘retry_config’, ‘learning_summary’, ‘expected_effect’ y ‘stop_reason’ cuando decida parar.

La logica de ejecucion sigue siendo determinista. El agente no entrena modelos ni modifica predicciones; solo propone una nueva ‘ModelingConfig’ soportada. El script ejecuta como maximo dos intentos, persiste cada decision y, si el ultimo intento sigue rechazado, fuerza ‘next_action = stop’ en la evaluacion efectiva para impedir bucles infinitos. Esta parada no sustituye al juicio del agente: documenta el limite operativo pactado para la ejecucion.

Para auditar si el agente razono de forma consistente, se ha añadido una capa de post-mortem:

```
codigo/app/services/reasoning_audit.py
codigo/app/schemas/reasoning.py
codigo/scripts/create_agentic_reasoning_audit.py
```

El post-mortem compara hipotesis, evidencia, accion, efecto esperado y metricas antes/despues. Ademas puede generar una solicitud de revision humana y una plantilla JSON de veredicto. Esta revision humana no altera la ejecucion; sirve para decidir si el razonamiento debe reutilizarse como contexto, descartarse o tratarse como caso frontera.

Como inicio de la Fase 4 se han añadido los contratos de memoria agentica supervisada. Estos contratos no implementan todavia una base vectorial, pero fijan la semantica que debera cumplir el futuro RAG: ‘HumanReviewSettings’ mantiene Human-in-the-loop apagado por defecto, ‘ReasoningMemoryRecord’ distingue ejemplos positivos, negativos, advertencias y casos frontera, y ‘RetrievedMemoryContext’ valida que el contexto recuperado pertenece al agente consultado. Un razonamiento ‘overcorrected’ o ‘unsafe’ no puede convertirse en ejemplo positivo; como mucho puede reutilizarse como advertencia o caso frontera si la politica de revision humana lo permite.

Sobre esos contratos se ha implementado el primer backend local de memoria vectorial. ‘LocalHashEmbeddingModel’ genera embeddings deterministas por hashing y ‘LocalJsonVectorMemoryStore’ persiste colecciones por agente en JSON, calcula similitud coseno y devuelve un ‘RetrievedMemoryContext’ validado. Esta eleccion evita dependencias externas en el primer hito de Fase 4 y permite probar la semantica de RAG antes de adoptar una base vectorial mas especializada como FAISS o Chroma. La memoria queda acotada por agente: una consulta del modelador puede recuperar ‘modeler_memory’ y memoria metodologica compartida, pero no recuerdos propios del limpiador.

Para uso real se ha definido un proveedor ‘OllamaEmbeddingProvider’, que llama a ‘/api/embed’ y registra modelo, version y dimension del vector en cada recuerdo. El modelo por defecto seleccionado es ‘qwen3-embedding:0.6b’, por su equilibrio entre coherencia con los agentes Qwen, tamano moderado, soporte multilingue y ventana de contexto amplia. El proveedor determinista por hashing queda reservado para tests y reconstrucciones sin dependencia externa; la memoria operacional debe reconstruirse si se cambia de modelo de embeddings, ya que los vectores de modelos distintos no comparten necesariamente el mismo espacio semantico.

Para convertir los razonamientos auditados en memoria se ha anadido un servicio de indexacion y un comando de reconstruccion reproducible. El indexador recorre los post-mortems persistidos, busca una revision final humana junto al artefacto original y, por defecto, solo indexa recuerdos reutilizables cuando existe dicha validacion humana. Esta politica mantiene el modo Human-in-the-loop apagado para la ejecucion normal, pero exige revision cuando se desea alimentar la memoria agentica. Los post-mortems sin revision quedan registrados como pendientes; si se activa una opcion explicita de

inclusión, solo pueden entrar como advertencia o caso frontera, no como ejemplo positivo. El modelo ‘qwen3-embedding:0.6b’ se ha instalado en Ollama y se ha verificado con una llamada real a ‘/api/embed’, obteniendo vectores de dimensión 1024 para la memoria operacional.

El reintento agéntico del modelador ya puede recibir memoria de forma opcional. El contrato ‘ModelingRetryDecision’ declara ahora si el agente usó memoria recuperada, el identificador del contexto consultado y los recuerdos concretos citados. Durante la construcción del prompt se genera una consulta ‘AgentMemoryQuery’ para ‘modeler_memory’, se recuperan hasta tres recuerdos y se pasan al agente como contexto compacto con similitud, veredicto humano, resumen, métricas y ruta de origen. La validación posterior rechaza cualquier decisión que cite un recuerdo no recuperado, por lo que el agente conserva libertad argumentativa pero no puede inventar memoria. Esta funcionalidad queda apagada por defecto; el script de reintento la activa solo con una opción explícita y persiste el contexto en ‘retrieved_memory_context.json’.

Para aumentar la evidencia de investigación, el contrato del modelador exige ahora una declaración más detallada cuando usa memoria: resumen de uso, tipo de relación con cada recuerdo (‘followed’, ‘adapted’, ‘contradicted’ o ‘ignored’), influencia concreta y mitigación del riesgo si el recuerdo era un caso frontera o una advertencia. Sobre esa declaración se ha implementado ‘memory_usage_audit.py’, que compara lo que el agente dijo hacer con la memoria recuperada y las métricas finales. La auditoría genera ‘memory_usage_audit.json’ y ‘memory_usage_audit.md’, y puede clasificar una decisión como ‘memory_repeated_boundary_failure’ si el agente cita un caso frontera pero repite el mismo tipo de fallo experimental.

El aprendizaje por memoria se ha generalizado para no depender solo del agente modelador ni de post-mortems finales. El tipo de fuente ‘memory_usage_audit’ permite transformar una auditoría de uso de memoria en un nuevo ‘ReasoningMemoryRecord’. La conversión recibe explícitamente el ‘target_agent’, de modo que el mismo mecanismo puede alimentar ‘cleaner_memory’, ‘structurer_memory’, ‘evaluator_memory’, ‘report_writer_memory’, ‘researcher_memory’ o ‘shared_methodology_memory’. En el caso actual, la auditoría donde el modelador citó un recuerdo de frontera pero repitió FPR 1.0 se incorpora como ‘warning’, no como ejemplo positivo.

Como primera carga canónica de memoria, se ha revisado el post-mortem del segundo reintento NASA sintético. El veredicto humano registrado es ‘partially_correct’: la decisión del agente acertó la dirección del cambio al reducir falsos negativos, pero sobre corrigió la magnitud del umbral y elevó la tasa de falsos positivos hasta 1.0. El caso queda por tanto como ‘boundary_case’, reutilizable como advertencia para el modelador, no como ejemplo positivo. Al reconstruir la memoria con ‘qwen3-embedding:0.6b’, se ha generado un vector de dimensión 1024 en ‘modeler_memory’; el informe de indexación conserva además que otro post-mortem sigue pendiente de revisión final. Una consulta de humo sobre sobrecorrección de umbral recupera este recuerdo como ‘boundary_context’ con una similitud aproximada de 0.6868.

Tras incorporar las auditorías de uso como fuente opcional, se reconstruyó la memoria canónica con ‘-include-memory-usage-audits’. El índice contiene ahora dos recuerdos: el post-mortem revisado por humano como ‘boundary_case’ y la auditoría de uso como ‘warning’. Una consulta de humo sobre repetición de fallo por umbral recupera primero la advertencia derivada de la auditoría (‘negative_warning’, similitud aproximada 0.6852) y después el caso frontera revisado (‘boundary_context’, similitud aproximada 0.6559). Esto cierra un primer ciclo de aprendizaje controlado: revisión

humana, memoria de frontera, uso agentico, auditoria del uso y nueva memoria de advertencia.

Para que esta memoria no quede limitada a parametros concretos, se ha introducido un contrato general de episodios de decision. ‘DecisionEpisode’ describe una decision agentica completa: agente, tipo de decision, contexto, alternativas consideradas, accion elegida, efecto esperado, evidencia usada, resultado de ejecucion, metricas antes/despues, trade-offs observados, modos de fallo, leccion aprendida y condiciones de reutilizacion. Este contrato sirve tanto para el modelador como para el limpiador, estructurador, evaluador, redactor o futuro investigador.

Sobre el episodio se define ‘MemoryCandidate’, que es la version destilada y lista para RAG. Un candidato debe indicar ‘target_agent’, ‘memory_role’, ‘when_to_reuse’, ‘when_not_to_reuse’ y ‘risk_if_misused’. La politica de seguridad se mantiene: un ejemplo positivo requiere veredicto humano correcto, un candidato reutilizable debe declarar cuando se puede usar, y una advertencia o ejemplo negativo debe aportar condiciones de no reutilizacion o riesgo. El indexador de memoria recoge automaticamente ‘memory_candidate.json’ y lo convierte en ‘ReasoningMemoryRecord’, generando despues el embedding local y almacenandolo en la coleccion vectorial del agente correspondiente.

El contrato anterior ya se ha conectado al flujo real de reintentos. El servicio ‘decision_memory.py’ transforma el post-mortem del modelador en ‘decision_episode.json’ y ‘memory_candidate.json’, junto con sus versiones Markdown. El episodio conserva la configuracion elegida como alternativa seleccionada, las alternativas comparables si existen, los recuerdos recuperados, la evaluacion final, los trade-offs y las condiciones de reutilizacion. El candidato queda marcado como reutilizable cuando no existe una revision humana bloqueante; si se solicita revision, se conserva como evidencia pero no debe incorporarse automaticamente al contexto recuperable hasta recibir validacion.

Para que estos episodios influyan en decisiones mas amplias que un simple ajuste de umbral, el prompt de reintento del modelador expone ahora el ‘source_type’ de cada recuerdo recuperado y reconoce etiquetas de comparacion de familia de modelo tras una mejora parcial. Cuando aparece este tipo de memoria, el agente recibe una instruccion explicita de considerar familias de modelo soportadas, no solo mover ‘threshold_quantile’. El formato esperado de ‘ModelingRetryDecision’ incluye tambien candidatos comparables, por ejemplo una alternativa PCA y otra variante moderada de Isolation Forest. Estos candidatos no se ejecutan de forma automatica; documentan la deliberacion del agente, mientras que la ejecucion real sigue limitada a una unica ‘retry_config’ validada.

La auditoria de uso de memoria se ha ampliado con una comprobacion de coherencia del razonamiento. Ademas de detectar repeticiones de fallos de frontera, ahora puede clasificar una decision como ‘memory_reasoning_inconsistent’ cuando la explicacion del agente contiene una comparacion numerica imposible, por ejemplo afirmar que ‘0.95’ es mayor que ‘0.99’, o cuando describe un cuantil ‘0.99’ como bajo. Esta comprobacion no cambia la decision del agente ni las metricas, pero deja evidencia trazable de errores en su razonamiento declarado. La auditoria resultante puede incorporarse posteriormente a ‘modeler_memory’ como ‘warning’, de forma que el siguiente agente vea no solo el resultado experimental, sino tambien la calidad de la explicacion usada para llegar a el.

Para transformar la comparacion de familia en una accion ejecutable, se ha anadido ademas una guia derivada de memoria al prompt del modelador. Esta guia, ‘me-

memory_retry_guidance', resume si los recuerdos recuperados indican que los ajustes de umbral sobre Isolation Forest ya han producido solo mejoras parciales rechazadas. En ese caso marca como preferente una configuracion 'pca_reconstruction_error' y advierte que no debe repetirse otro reintento basado solo en 'threshold_quantile' de Isolation Forest. La guia no ejecuta nada por si misma: sigue siendo el agente quien debe devolver 'retry_config' o 'should_retry=false', y la decision vuelve a pasar por los contratos Pydantic. La primera ejecucion real con esta guia consiguio que Qwen eligiera PCA como modelo ejecutado.

Tras obtener esta evidencia, la implementacion se ha reorientado para no concentrar la Fase 4 en ajustes de umbral o en una unica familia de modelos. El servicio 'agent_memory.py' centraliza ahora la preparacion de memoria recuperada para prompts, la plantilla comun de declaracion de uso y la validacion de que un agente solo cite recuerdos realmente recuperados. Sobre esta base, 'StructuringDecision' y 'EvaluationDecision' incorporan los mismos campos de memoria que el reintento del modelador: contexto recuperado, recuerdos citados, resumen de uso y declaracion por recuerdo.

El agente estructurador dispone de 'build_structurer_memory_query' y 'retrieve_structurer_memory_context', orientados a recuperar experiencias sobre ventanas, solape, features, frecuencia de muestreo y particion temporal. La memoria se inyecta en el prompt como contexto historico, pero la validacion sigue impidiendo ventanas, canales, solapes o features no soportadas. De forma analoga, el agente evaluador dispone de 'build_evaluator_memory_query' y 'retrieve_evaluator_memory_context'; sus recuerdos pueden mejorar la explicacion y las limitaciones, pero no pueden aprobar metricas que incumplen el protocolo local.

Tambien se han ampliado los episodios de decision para estos dos agentes. 'decision_memory.py' puede construir episodios y candidatos de memoria para 'structurer_memory' y 'evaluator_memory', no solo para 'modeler_memory'. Esto convierte la memoria en un mecanismo transversal de aprendizaje gradual: cada ejecucion futura podra dejar evidencia sobre decisiones de estructuracion, juicios de evaluacion y condiciones de reutilizacion, sin forzar errores artificiales ni depender exclusivamente del ajuste de hiperparametros.

La preparacion anterior se ha conectado despues al grafo completo mediante 'PipelineMemoryConfig'. Cuando se activa un 'VectorMemoryStore', el nodo del estructurador y el nodo del evaluador recuperan memoria de su propia coleccion, persisten el contexto recuperado bajo una ruta agentica de la ejecucion y lo pasan al agente antes de validar su decision:

```
codigo/reports/<dataset>/<run_id>/agent_memory/<agent>/
  retrieved_memory_context.json
  decision_episode.json
  decision_episode.md
  memory_candidate.json
  memory_candidate.md
```

La configuracion queda apagada por defecto, de modo que las pruebas canonicas y el pipeline historico conservan el mismo comportamiento si no se proporciona memoria.

Para cerrar el circuito operacional se ha añadido un script especifico:

```
codigo/scripts/run_cwru_pipeline_with_memory.py
```

Este script ejecuta el pipeline CWRU persistido con ‘PipelineMemoryConfig’, permite elegir proveedor de embeddings, puede activar Qwen con ‘-use-llm’ y reconstruye el indice RAG al finalizar si se usa ‘-rebuild-index’. Es deliberadamente especifico de CWRU: no selecciona entre CWRU y NASA IMS, sino que crea el estado con ‘create_initial_cwru_state’ y usa los ejecutores CWRU por defecto. Para NASA IMS se mantienen scripts separados, porque la politica temporal y los guardarrailes metodologicos son distintos. Ademas, el script rechaza rutas ‘-raw-path’ que parezcan NASA/IMS para evitar una ejecucion mal etiquetada. En una primera run determinista se generaron los primeros candidatos de ‘structurer_memory’ y ‘evaluator_memory’; en una segunda run con Qwen, ambos agentes recuperaron esos recuerdos y declararon ‘used_memory_context=true’. El indice resultante contiene veinte registros: dieciseis del modelador, dos del estructurador y dos del evaluador.

Tras esa ejecucion se ha incorporado una auditoria cualitativa transversal en ‘transversal_memory_audit.py’, expuesta mediante ‘audit_transversal_memory_run.py’. Esta auditoria no recalcula metricas ni modifica la decision; comprueba que los recuerdos citados fueron recuperados, que cada recuerdo citado tiene declaracion ‘memory_record_uses’, que el dataset y el agente coinciden, y que el evaluador mantiene respaldo metrico actual. En la run ‘cwru-memory-transversal-fase4-002-llm’ el resultado fue ‘memory_aligned’ para estructurador y evaluador. Los candidatos generados pueden reutilizarse como contexto, pero quedan marcados como necesitados de revision humana antes de usarse como evidencia academica principal.

Para anticipar una futura interfaz de seleccion de dataset, se ha añadido una capa comun de planificacion multi-dataset:

```
codigo/app/schemas/pipeline_run.py
codigo/app/services/pipeline_runner.py
codigo/scripts/run_dataset_pipeline_with_memory.py
```

La interfaz prevista no deberia llamar directamente a scripts especificos ‘cwru_’ o ‘nasa_’. Debe construir un ‘PipelineRunRequest’, resolver el adaptador registrado, generar un ‘DatasetPipelinePlan’ y mostrar la politica de capacidades antes de ejecutar. CWRU permite el pipeline supervisado completo. NASA IMS real permite manifesto, perfilado, limpieza y estructuracion, pero bloquea modelado y evaluacion supervisados mientras no exista una politica temporal y de etiquetas defendible. Si NASA IMS se usa como benchmark sintetico controlado, la peticion debe declararlo mediante ‘allow_synthetic_labels’. Si se desea ejecutar NASA IMS como secuencia run-to-failure con etiquetas proxy, la peticion debe declarar una politica versionada mediante ‘dataset_policy_id’. La primera politica implementada es ‘nasa_ims_temporal_v1’, documentada en ‘codigo/docs/33_politica_temporal_nasa_ims.md’. Los nombres historicos del grafo se mantienen por compatibilidad, pero se han añadido alias neutrales (‘run_pipeline’, ‘run_and_persist_pipeline’) para evitar acoplar la futura API al nombre CWRU.

La politica ‘nasa_ims_temporal_v1’ se aplica despues de generar el manifesto comun NASA IMS y antes del perfilado. El servicio ‘nasa_ims_temporal_policy.py’ escribe un nuevo manifesto derivado, ‘manifest_temporal_policy_v1.csv’, conserva el manifesto original y marca cada registro con ‘label_source=temporal_proxy_v1’ y ‘official_nasa_labels=false’. El tramo inicial de cada ‘run_id’ se etiqueta como ‘normal’, el tramo posterior como ‘degradation’ y ‘metadata_json’ incluye ‘split_hint’, ‘split_source’, ‘temporal_order_index’ y ‘temporal_phase’. La estructuracion temporal reconoce estos ‘split_hint’ solo cuando todos los registros limpios los declaran;

en caso contrario, mantiene la estrategia historica por etiquetas normales/fallo. Asi, CWRU no cambia y NASA IMS no depende de una inferencia implicita dentro del ejecutor.

Como pulido adicional de trazabilidad multi-dataset, los artefactos comunes ya no usan nombres internos heredados de CWRU cuando se ejecutan sobre NASA IMS. Las rutas pueden seguir apuntando a ‘cwru_bearing’ en el benchmark historico, pero los identificadores de artefacto pasan a ser neutrales: ‘windows_features’, ‘model_predictions’, ‘modeling_summary’, ‘evaluation_metrics’ y ‘evaluation_summary’. Esto reduce el acoplamiento visual del sistema a CWRU y hace mas clara la lectura de snapshots multi-dataset.

5.28. API Controlada de Ejecucion

Tras consolidar el runner comun, se ha iniciado el Hito 8 de Fase 4 con una primera version de ‘POST /runs’. La API conserva los endpoints de lectura sobre runs persistidos y anade un contrato especifico:

```
codigo/app/schemas/api_runs.py
codigo/app/services/api_run_jobs.py
codigo/app/api/routes.py
```

La solicitud de ejecucion API extiende el contrato multi-dataset con el campo ‘dry_run’, que por defecto vale ‘true’. En ese modo, la API no ejecuta el pipeline: construye un plan de ejecucion y devuelve si las fases solicitadas estan permitidas, que adaptador se usara, que rutas se derivan y que bloqueos existen. Si el cliente envia ‘dry_run=false’, la API intenta ejecutar el runner comun, pero solo despues de aplicar guardarrailes.

Los guardarrailes iniciales son deliberadamente conservadores. ‘raw_path’ debe estar dentro de una raiz permitida por ‘create_app’, siendo ‘codigo/data/raw’ la raiz por defecto. La ejecucion falla con respuesta HTTP ‘409’ si la politica multi-dataset bloquea modelado o evaluacion. Ademas, esta primera version no permite ejecutar con ‘use_llm=true’ ni con memoria RAG desde API; esas capacidades se reservaran para una politica posterior con Human-in-the-loop, coste y auditoria mas explicitos. Finalmente, si el ‘run_id’ ya existe en el registro local, la API responde ‘409’ para evitar sobrescrituras accidentales.

La primera pieza de Human-in-the-loop se ha conectado a esta puerta de API sin crear contratos duplicados. La solicitud reutiliza los contratos de revision ya existentes y la logica comun se centraliza en un servicio dedicado. En modo ‘off’ no interviene; en modo ‘passive’ devuelve razones de revision sin bloquear; en modo ‘required’ impide ejecutar salvo que llegue una aprobacion explicita. Cuando la aprobacion existe, se inserta en el estado inicial y queda persistida en el snapshot de la run. Esta implementacion cubre la puerta minima de revision, pero no introduce todavia colas, usuarios, notificaciones ni una interfaz de aprobacion completa.

La respuesta ‘ApiRunResponse’ devuelve siempre el plan y, cuando hay ejecucion, tambien el snapshot persistido, la fase final, el estado ‘approved’, la ruta del informe, las metricas y los errores. Esto permite que una interfaz futura distinga entre ejecucion tecnica completada y resultado experimental no aprobado por metricas.

Como siguiente paso incremental se ha anadido una ejecucion en segundo plano local, activada mediante ‘background=true’ junto con ‘dry_run=false’. Esta opcion no

introduce un segundo runner: el job invoca el runner comun y conserva la persistencia de snapshots como fuente de verdad cuando la ejecucion termina. El estado intermedio se mantiene en memoria del proceso mediante un registro de jobs y se consulta con un endpoint especifico. Por tanto, esta ampliacion sirve para una futura UI local, pero no pretende ser todavia una cola persistida con workers externos, cancelacion o tolerancia a reinicios.

5.29. Dependencias

Se ha creado ‘requirements.txt’ como punto inicial para construir el entorno de Miniconda ‘tfm_v2’, fijado sobre Python 3.11. Incluye dependencias de contratos y grafo, procesamiento de datos, modelos clásicos, API local con FastAPI, visualización técnica y verificación.

5.30. Verificación Actual

La verificación de los contratos se ha realizado con tests unitarios en el entorno objetivo:

```
conda run -n tfm_v2 python -m unittest discover codigo/tests
```

Las pruebas validan la creación del estado inicial CWRU, serialización JSON, rechazo de campos no declarados, restricciones de configuraciones y métricas, coherencia del manifiesto, decisiones de agentes, resultados de ejecutores y generación del manifiesto, perfilado, limpieza, estructuración temporal y modelado base y evaluación CWRU. También validan los adaptadores de entrada para formatos ‘.mat’ y ‘.csv’, y la orquestacion del grafo minimo con ejecutores simulados. Tambien validan el supervisor LLM con cliente fake, el agente limpiador LLM, el agente estructurador LLM, el agente modelador LLM, el agente evaluador LLM, el agente redactor LLM, el ejecutor de informes, el parser JSON, la persistencia local de ejecuciones, el registro consultable, el protocolo experimental local, la API minima de consulta y ejecucion controlada, la configuracion de razonamiento de Ollama, los contratos multi-dataset iniciales, el registro de adaptadores de dataset, el manifiesto comun NASA IMS sobre una carpeta sintetica preextraida, el rechazo de paquetes comprimidos para ese manifiesto, el perfilado del manifiesto comun NASA IMS, la lectura tabular de snapshots IMS sin extension, la limpieza comun con seleccion controlada de canal, el rechazo de canales no declarados, el rechazo de transiciones invalidas y el fallback determinista contextualizado por dataset. Tambien validan el ‘decision_summary’ del perfilador, su entrega al agente limpiador, el expediente experto del estructurador, las alternativas comparables propuestas por el estructurador y la generacion de planes de ventana y modelo a partir de dichas alternativas, el ejecutor PCA, el diagnostico no supervisado de degradacion, el benchmark NASA sintetico, la iteracion agentica acotada, la auditoria del razonamiento y la preparacion de revision humana. Tambien validan los contratos iniciales de memoria agentica supervisada, incluyendo modo Human-in-the-loop apagado por defecto, exclusion de recuerdos no reutilizables y recuperacion acotada por agente. Tambien validan el primer backend vectorial local con embeddings deterministas, persistencia JSON, reconstruccion reproducible y filtrado por agente, junto con el proveedor Ollama de

embeddings reales y su seleccion por defecto de ‘qwen3-embedding:0.6b’, y el indexador que convierte post-mortems revisados en memoria vectorial sin admitir ejemplos positivos no validados. Tambien validan el retrieval opcional del modelador, la inclusion de memoria en el prompt de reintento, la declaracion explicita de recuerdos usados y el rechazo de identificadores de memoria no recuperados. Tambien validan la nueva auditoria de uso de memoria, incluyendo la deteccion de repeticiones de fallos en recuerdos de tipo frontera, su conversion posterior en memoria de advertencia y la separacion de colecciones para otros agentes como el evaluador. Tambien validan el nuevo contrato de episodios de decision, la destilacion en ‘MemoryCandidate’, su conversion a memoria vectorial y la indexacion automatica de ‘memory_candidate.json’. Tambien validan la generacion automatica de episodios y candidatos a partir de post-mortems de reintento. La ampliacion mas reciente valida ademas el retrieval opcional del estructurador y del evaluador, la imposibilidad de citar recuerdos no recuperados, y la generacion de episodios y candidatos para ‘structurer_memory’ y ‘evaluator_memory’ dentro de una ejecucion completa del grafo. Tambien valida la auditoria cualitativa de memoria transversal para estructurador y evaluador, incluyendo la deteccion de recuerdos citados pero no recuperados. La ultima ampliacion cubre el runner comun multi-dataset, la politica temporal NASA IMS, ‘POST /runs’ con planificacion segura y la puerta inicial de revision humana ‘off/passive/required’ desde API. La extension actual anade jobs API locales en memoria para ejecuciones ‘background=true’. En la verificacion actual se han ejecutado 241 tests correctamente.

5.31. Siguiete Paso de Implementación

Con la Fase 4 cerrada, el siguiente bloque de trabajo pasa a ser la aplicacion local completa. La nueva Fase 5 tendra como objetivo construir una interfaz backend/frontend capaz de seleccionar o recibir datasets, ejecutar runs mediante la API controlada, seguir jobs en segundo plano, visualizar metricas, informes y artefactos, y consultar la memoria local generada por los agentes. Esta fase mantendra las mismas reglas metodologicas: reutilizacion de componentes existentes, documentacion tecnica paralela y actualizacion de la memoria academica.

Como primer avance de esta fase se ha creado un esqueleto de frontend local en React, TypeScript y Vite. La pantalla inicial consulta el estado de la API, lista las ejecuciones persistidas y permite enviar una solicitud ‘POST /runs’ con ‘dry_run=true’, mostrando el plan devuelto por el backend sin ejecutar el pipeline. Esta decision mantiene la logica de planificacion y las politicas de dataset en FastAPI y en los servicios Python existentes, evitando duplicar contratos en la interfaz. La verificacion incluyo la compilacion del frontend y una prueba HTTP del proxy local hacia ‘GET /health’ y ‘POST /runs’.

El segundo avance de Fase 5 sustituye el catalogo local de la interfaz por una consulta al backend. Se han incorporado los endpoints ‘GET /datasets/adapters’ y ‘POST /datasets/describe’, ambos delegando en el registro existente de adaptadores de dataset. La descripcion de un dataset valida que la ruta pertenezca a una raiz permitida, infiere o comprueba el adaptador y devuelve un descriptor compacto sin cargar el dataset completo ni generar manifiestos. Asimismo, la aplicacion crea un area de staging bajo ‘codigo/data/raw/uploads/’ para futuras ingestas locales controladas. Esta capa mantiene ‘POST /runs’ con ‘dry_run=true’ como preflight canonico de capacidades antes de ejecutar. La verificacion incluyo tests de catalogo, rechazo de

rutas fuera de raiz, rechazo de adaptadores incompatibles y una nueva compilacion del frontend.

El tercer avance conecta la interfaz con la ejecucion real en segundo plano ya disponible en la API. La vista mantiene un boton de planificacion basado en `'dry_run=true'` y habilita la accion de ejecucion solamente cuando el plan devuelto por el backend es ejecutable. La ejecucion envia `'POST /runs'` con `'dry_run=false'` y `'background=true'`, recibe un identificador de job local y consulta periodicamente `'GET /run-jobs/{job_id}'`. Cuando el job termina, la interfaz obtiene el snapshot persistido mediante `'GET /runs/{run_id}'` y refresca el registro de ejecuciones. Si el job falla, se muestra el detalle tecnico devuelto por el backend, preservando la trazabilidad del error sin confundirlo con un resultado metodologicamente no aprobado. Como prueba de humo se lanzo una run CWRU en segundo plano, que completo con snapshot persistido, informe tecnico y artefactos registrados.

El cuarto avance convierte el registro de ejecuciones en una vista operativa. La interfaz consume los endpoints de lectura ya existentes: `'GET /runs'` para listar con filtros, `'GET /runs/{run_id}'` para cargar el snapshot, `'GET /runs/{run_id}/artifacts'` para inspeccionar artefactos, `'GET /runs/{run_id}/report'` para previsualizar el informe Markdown y `'GET /runs/compare'` para comparar metricas entre varias ejecuciones. De este modo, la aplicacion no recalcula resultados ni lee ficheros directamente desde el navegador, sino que reutiliza el registro local persistido y los contratos `'RunIndexEntry'`, `'RunSnapshot'` y `'RunComparison'`. La vista distingue el fallo de infraestructura de una ejecucion completada pero no aprobada por metricas, manteniendo la semantica metodologica del evaluador.

El quinto avance inicia la observabilidad agentica en tiempo real. Se ha definido el contrato `'AgentRuntimeEvent'` y un `'AgentRuntimeRecorder'` que permite al job en segundo plano acumular eventos de supervisor, agentes, recuperacion de memoria, ejecutores, estado del job y errores. El grafo LangGraph emite estos eventos de forma opcional y no bloqueante, de modo que la telemetria no altera la ejecucion ni sustituye los snapshots persistidos. La API expone los eventos en `'GET /run-jobs/{job_id}'` y en `'GET /run-jobs/{job_id}/events'`. En el frontend se ha anadido la pestana `'Agentes'`, que representa la jerarquia supervisor-agentes, un timeline de comunicacion y un panel de detalle con `'decision_id'`, confianza, `'rationale'`, memoria citada y payload JSON observable. Esta vista muestra decisiones y justificaciones estructuradas, no cadena de pensamiento privada del modelo.

El siguiente incremento completa la visibilidad de memoria persistida. Se ha anadido una capa API de solo lectura sobre `'LocalJsonVectorMemoryStore'`, sin crear otro indice ni modificar la logica de recuperacion RAG de los agentes. La API expone `'GET /memory/collections'` para resumir colecciones canonicas, `'GET /memory/records'` para listar recuerdos filtrables por agente, dataset, rol, reutilizacion y texto, y `'GET /memory/records/{memory_record_id}'` para abrir el `'ReasoningMemoryRecord'` completo. En el frontend, la pestana `'Agentes'` muestra para el agente seleccionado la coleccion persistida, numero de recuerdos, recuerdos reutilizables, busqueda textual y detalle de fuente, veredicto humano, `'run_id'`, `'decision_id'` y contenido. La memoria se presenta como evidencia trazable y no como mecanismo de aprobacion automatica.

El sexto avance hace operativa la revision humana minima desde la interfaz. Se reutilizan los contratos existentes `'HumanReviewSettings'` y `'HumanApproval'`, asi como las razones `'human_review_reasons'` calculadas por la API durante el preflight. La UI permite seleccionar el modo `'off'`, `'passive'` o `'required'`, indicar el revisor, marcar los

puntos de revision explicita y, cuando procede, registrar una aprobacion simple con motivo y fecha. En modo ‘passive’, las razones se muestran como advertencia sin bloquear la ejecucion. En modo ‘required’, la accion de ejecutar queda bloqueada hasta que exista aprobacion explicita, manteniendo ademas la validacion del backend como fuente de verdad. Este incremento no introduce un sistema multiusuario de aprobaciones, sino una puerta local y trazable coherente con el alcance de Fase 5.

El septimo avance consolida el frontend como herramienta operativa completa. La interfaz queda organizada en una vista ‘Pipeline’, que agrupa seleccion de dataset, preflight, revision humana, ejecucion en segundo plano, seguimiento de job, registro de runs, detalle, metricas, informe, artefactos y comparacion, y una vista ‘Agentes’, que agrupa observabilidad runtime, jerarquia de agentes, timeline, memoria recuperada y memoria persistida. Para reforzar la trazabilidad durante la operacion se incorpora una banda de contexto fija con ‘dataset_id’, ‘run_id’, modo de ejecucion, politica de dataset, numero de fases, estado del plan y estado del job. Esta consolidacion no introduce nuevos endpoints ni runners, sino que integra visualmente los contratos y servicios ya validados en los hitos anteriores.

Como ampliacion visual de la aplicacion, se ha incorporado una lectura humana determinista de los eventos agenticos. Junto al ‘payload’ JSON de cada evento, la interfaz genera una frase que resume el agente responsable, la fase, la confianza, la siguiente accion propuesta y los campos clave del JSON. Esta traduccion no pretende revelar cadena de pensamiento privada, sino convertir la telemetria estructurada en una explicacion operativa comprensible.

Tambien se ha anadido una pestana ‘Visualizacion’ para analizar resultados de runs persistentes. El backend expone ‘GET /runs/{run_id}/visualization’, que lee los artefactos ‘windows_features.csv’ y ‘predictions.csv’, une ambos por ‘window_id’, escala las features numericas y calcula una proyeccion PCA en dos dimensiones. La interfaz muestra barras de metricas y un diagrama de dispersion con ventanas normales, anomalias detectadas y una frontera visual aproximada. Se declara explicitamente que esta frontera es una ayuda grafica en la proyeccion 2D, no la frontera exacta del modelo en el espacio original de características.

El siguiente ajuste conecta la aplicacion web con los agentes Qwen/Ollama reales. El contrato ‘PipelineRunRequest’ ya contenia el campo ‘use_llm’, pero la API lo mantenia bloqueado por seguridad durante la primera ejecucion controlada. En Fase 5 se sustituye ese bloqueo por una comprobacion explicita de disponibilidad: ‘GET /llm/status’ consulta Ollama local, informa del modelo configurado y verifica que ‘qwen3.5:4b’ este instalado. Si la interfaz activa ‘Agentes LLM’, ‘POST /runs’ con ‘use_llm=true’ construye un conjunto de agentes con ‘OllamaJSONClient’ e inyecta ese cliente en supervisor, limpiador, estructurador, modelador, evaluador y redactor. La ejecucion conserva los contratos JSON/Pydantic y los fallbacks deterministas ante respuestas invalidas, por lo que el LLM participa en la decision sin adquirir capacidad de ejecutar codigo ni modificar directamente los datos.

Finalmente, se ha refactorizado la interfaz de Fase 5 desde una pantalla tecnica acumulativa hacia un dashboard local de operacion. La aplicacion se estructura con un shell principal, navegacion lateral, cabecera de estado, resumen de salud y areas de trabajo separadas para pipeline, agentes y visualizacion. Dentro de ‘Pipeline’, la configuracion de la run, el preflight, la ejecucion y el historial de runs se presentan como bloques diferenciados, mientras que rutas largas, parametros avanzados y detalles tecnicos quedan en paneles plegables. Este cambio no altera endpoints ni contratos

del backend: reorganiza visualmente las mismas capacidades para mejorar jerarquia, legibilidad y mantenibilidad de la aplicacion local. Como ultimo pulido de interfaz, se evita renderizar rutas locales de ficheros en la pagina web y el panel de nueva ejecucion deja de comportarse como un bloque pegajoso durante el desplazamiento vertical.

Con estos elementos, la Fase 5 queda cerrada como aplicacion local avanzada y defendible. El sistema ya no depende solo de scripts para demostrar el flujo: permite operar la API desde una interfaz web, ejecutar runs controladas, seguir jobs, consultar resultados, observar agentes, revisar memoria y analizar visualizaciones. Las limitaciones que permanecen son deliberadas: no se ha introducido autenticacion multiusuario, cola persistida, despliegue cloud ni empaquetado Docker. La siguiente fase se centra precisamente en la dockerizacion, la reproducibilidad local y la preparacion de una demo final repetible, manteniendo la misma metodologia de reutilizacion, contratos estrictos y separacion entre razonamiento generativo y ejecucion determinista.

El primer paso de la Fase 6 no consiste aun en crear imagenes, sino en fijar los supuestos de entorno. Se documentan los puertos canonicos del sistema local ('8010' para FastAPI, '5173' para Vite y '11434' para Ollama), las variables de entorno necesarias para Qwen y embeddings, y los directorios que deberan montarse como volúmenes persistentes ('codigo/data', 'codigo/reports', 'codigo/models' y 'codigo/experiments'). Tambien se decide que Ollama quedara externo al primer 'docker-compose', de forma que los modelos locales no se descarguen ni empaqueten dentro de las imagenes iniciales.

El segundo paso prepara la configuracion reproducible sin crear todavia contenedores. Se añade una plantilla '.env.example' en 'codigo/docker/' con las variables de backend, frontend, LLM, embeddings y volúmenes previstos, y se documenta el uso esperado de la carpeta Docker. Ademas, el proxy de desarrollo de Vite se hace configurable mediante 'VITE_DEV_PROXY_TARGET', conservando 'http://127.0.0.1:8010' como valor por defecto para el desarrollo local.

El tercer paso define la primera imagen Docker del backend. Se añade un '.dockerignore' para excluir caches, dependencias frontend, datasets, reports, modelos, experimentos y recursos academicos del contexto de build, y se crea un 'backend.Dockerfile' basado en 'python:3.11-slim'. La imagen instala 'requirements.txt', copia unicamente el paquete 'codigo.app', crea los directorios base que despues se montaran como volúmenes y arranca la API con Uvicorn en el puerto '8010'. La verificacion estatica del paquete Python se ejecuta correctamente. Tras activar Docker Desktop, la imagen 'tfm-backend:fase6-hito3' se construye y se valida con un contenedor publicado en '8015:8010' para no interferir con la API local ya activa en '8010'. Los endpoints 'GET /health', 'GET /datasets/adapters' y 'GET /llm/status' responden desde el contenedor; este ultimo confirma la comunicacion con Ollama externo a traves de 'host.docker.internal:11434' y disponibilidad del modelo 'qwen3.5:4b'.

El cuarto paso empaqueta el frontend como una imagen estatica reproducible. Se opta por no ejecutar Vite en modo desarrollo dentro de Docker: la imagen usa 'node:22-alpine' para instalar dependencias con 'npm ci' y generar el build React/Vite, y una segunda etapa 'nginx:1.27-alpine' para servir los artefactos compilados. La variable 'VITE_API_BASE_URL' mantiene '/'api' como frontera publica del navegador, mientras que una plantilla Nginx redirige ese prefijo a 'TFM_BACKEND_URL', que en compose apuntara al servicio 'backend' y en pruebas manuales puede apuntar al backend local mediante 'host.docker.internal:8010'. La verificacion confirma que 'npm run build' pasa en local, que la imagen 'tfm-frontend:fase6-hito4' se construye correctamente y

que un contenedor publicado en '5174:5173' sirve el HTML del dashboard, responde a '/healthz' y comunica con la API mediante '/api/health' y '/api/datasets/adapters'. El healthcheck Docker queda en estado 'healthy'.

El quinto paso integra ambas imagenes en un 'docker-compose.yml' local. El servicio 'backend' construye la imagen FastAPI, monta 'codigo/data', 'codigo/reports', 'codigo/models' y 'codigo/experiments' como bind mounts y conserva Ollama como dependencia externa mediante 'host.docker.internal:11434'. El servicio 'frontend' construye la imagen web, espera a que el backend este saludable y redirige su proxy '/api' al servicio interno 'backend:8010'. Los puertos publicados en el host quedan parametrizados, de modo que la validacion se ejecuta en '8016' y '5175' sin interferir con la API local ya activa en '8010'. La comprobacion de compose confirma que ambos servicios quedan 'healthy', que el backend responde a '/health' y '/llm/status', que el frontend sirve el HTML compilado y que el proxy Nginx devuelve correctamente '/api/health' y '/api/datasets/adapters'.

Con este resultado, la Fase 6 queda cerrada operativamente en el Hito 5. La decision es no automatizar todavia las pruebas de humo ni preparar la demo final cerrada, porque el siguiente bloque de trabajo volvera a la aplicacion para pulir funcionalidad, ampliar capacidades, revisar fronteras y mejorar la experiencia de usuario. Docker queda como base reproducible minima: a partir de ahora, cada cambio relevante de backend, frontend, dependencias o variables se desarrollara primero en local y se trasladara despues al stack dockerizado mediante reconstruccion y validacion manual. Esta decision evita fijar una secuencia de pruebas sobre una superficie que todavia se espera que evolucione.

5.32. Fase 7: evidence pack e informe agentico

Tras cerrar la base dockerizada minima, la Fase 7 vuelve al nucleo funcional del TFM: agentes de IA, deteccion autonoma de anomalias y reporte auditable para analistas humanos. El primer hito se centra en dos necesidades: conservar una evidencia estructurada de cada ejecucion y hacer que el agente redactor tenga un papel narrativo real en el informe final, sin romper la separacion entre razonamiento generativo y ejecucion determinista.

En esta fase se fija ademas un criterio de implementacion para evitar que la instruccion determinista oculte el objetivo investigador. Los agentes Qwen/LLM conservan el protagonismo como decisores e interpretes del sistema: el supervisor coordina, el limpiador y estructurador formulan configuraciones, el modelador plantea hipotesis, el evaluador juzga resultados, el redactor y el verificador producen y revisan conclusiones. Los ejecutores, herramientas y visualizaciones deterministas son el soporte que permite que esas decisiones sean seguras, reproducibles y auditables. Por ello, cada nueva metrica, vista o herramienta de Fase 7 debe entenderse como una ampliacion del campo de evidencia de los agentes, no como una sustitucion de la investigacion agentica.

La decision de trabajar con LLM locales relativamente pequenos, como Qwen sobre Ollama, se conserva como hipotesis metodologica. Su lentitud en una maquina local no invalida la linea: permite estudiar que ocurre cuando se intenta acercar un sistema agentico auditable a restricciones industriales de coste, privacidad, despliegue local y latencia. Si la arquitectura alcanza limites practicos, esos limites se documentaran como resultado experimental; mientras tanto, la implementacion seguira intentando llevar la participacion agentica tan lejos como permitan los contratos estrictos y los

guardarrailes de seguridad.

La persistencia local de runs se ha ampliado en `run_persistence.py`. Cada snapshot nuevo puede generar ahora dos artefactos adicionales:

```
codigo/reports/runs/<run_id>/evidence_pack.json
codigo/reports/runs/<run_id>/evidence_pack.md
```

El primer fichero es el paquete de evidencia auditable. Incluye identificacion de la run, contexto experimental, configuraciones, metricas, evaluacion, revision humana si existe, decisiones agenticas normalizadas, errores y artefactos con estado de existencia y checksum SHA-256 cuando el fichero esta disponible. El segundo fichero es una vista Markdown resumida para inspeccion humana rapida. Esta vista no sustituye al informe tecnico final, sino que ayuda a auditar de donde salen las conclusiones.

Tambien se ha ampliado `pipeline_runner.py` para registrar, antes de la ejecucion, la solicitud exacta y el plan preflight aplicado:

```
codigo/reports/<dataset>/<run_id>/evidence/pipeline_request.json
codigo/reports/<dataset>/<run_id>/evidence/pipeline_plan.json
```

Estos ficheros son artefactos de tipo `config` y permiten revisar que se pidio, que politica de dataset se aplico, que fases eran efectivas y si habia bloqueos metodologicos antes de transformar datos.

En paralelo, el contrato `ReportSection` se ha extendido con campos de contenido humano: cuerpo narrativo, hallazgos principales, recomendaciones y referencias de evidencia. El agente `report_writer` sigue devolviendo un `ReportDecision` validado por Pydantic, pero ahora puede redactar las secciones del informe en castellano tecnico. El JSON sigue siendo un contrato interno de seguridad y validacion; el documento final para el analista se materializa como Markdown mediante el ejecutor determinista `reporting.py`.

La verificacion del hito se ha realizado con las pruebas de agente redactor, ejecutor de reporting, persistencia de runs y runner comun. La bateria ejecutada cubre la redaccion narrativa validada, el renderizado del informe, la creacion de evidence pack y la inclusion de request/plan como artefactos de evidencia. Con ello, el sistema queda mejor preparado para una defensa auditable del TFM: no solo produce metricas, sino que conserva la ruta entre peticion, plan, decisiones agenticas, artefactos y conclusiones humanas.

La interfaz web se ha ajustado para reflejar esta nueva jerarquia. En el detalle de una run, el informe ya no aparece como un bloque tecnico al final de la lista de artefactos, sino como la seccion principal de cierre analitico. El Markdown servido por la API se transforma en una vista de documento con titulo, metadatos clave, secciones, parrafos, hallazgos y recomendaciones. La evidencia tecnica queda debajo, separada visualmente, como soporte auditable del informe. Ademias, la previsualizacion de la UI filtra rutas locales y bloques de fuentes tecnicas para mantener la decision de no exponer rutas de fichero en la pagina web, sin alterar el artefacto Markdown persistido en disco.

Como segundo hito de Fase 7 se ha separado explicitamente el informe final del agente redactor de una nueva auditoria de ejecucion. Esta auditoria se genera de forma determinista a partir del *evidence pack* y se persiste como:

```
codigo/reports/runs/<run_id>/execution_audit.md
```

El objetivo de este documento es que un evaluador humano pueda entender, de un vistazo y con lenguaje claro, que ocurrió durante toda la run. La auditoria incluye lectura rapida, solicitud y plan aplicado, cronologia de agentes, resultado tecnico, metricas, artefactos, checksums abreviados, errores, bloqueos y consideraciones de revision humana. La API expone este documento mediante ‘GET /runs/{run_id}/audit-report’, manteniendo separado el endpoint del informe final ‘GET /runs/{run_id}/report’.

En frontend, el detalle de run queda organizado en tres niveles: primero el informe final de conclusiones generado por ‘report_writer’, despues la auditoria de ejecucion como explicacion operativa del proceso, y finalmente la evidencia tecnica como lista de artefactos. Esta separacion evita que el analista confunda el documento narrativo de conclusiones con la traza determinista de lo que ha ocurrido durante la ejecucion.

El tercer hito introduce una primera capa de verificacion agentica del informe final. Se ha creado el agente **report_verifier**, cuya funcion no es exigir una redaccion literal ni imponer un estilo concreto, sino detectar afirmaciones factuales o metodologicas que no esten respaldadas por la evidencia disponible. El agente recibe el informe Markdown y un catalogo cerrado de evidencias derivado del estado de la run: dataset, modo de etiquetas, configuraciones, metricas, evaluacion, limitaciones, artefactos y politicas metodologicas relevantes. Con esa informacion devuelve una decision **Pydantic ReportVerificationDecision** con estado de verificacion, incidencias, severidad, referencias de evidencia y correcciones requeridas.

La verificacion mantiene una tolerancia explicita sobre orden, enfasis y formulacion del texto. Lo que se penaliza son afirmaciones no soportadas, como presentar una ejecucion local como validacion industrial, declarar etiquetas oficiales en NASA IMS cuando solo existe politica proxy o sintetica, omitir que una run no fue aprobada o esconder limitaciones persistidas por el evaluador. Si el LLM no esta disponible, un fallback determinista detecta solo riesgos evidentes para no convertir esta capa en un script rigido de comparacion literal.

Tras generarse **final_report.md**, el grafo invoca el verificador y persiste dos artefactos bajo el directorio de evidencias del informe:

```
<directorio_del_informe>/evidence/report_verification.json
<directorio_del_informe>/evidence/report_verification.md
```

El primer fichero conserva la decision estructurada y el segundo ofrece una lectura humana de incidencias y correcciones. La auditoria de ejecucion incorpora ademas un bloque especifico de verificacion del informe.

El cuarto hito implementa ya ese debate controlado entre **report_writer** y **report_verifier**. Se ha anadido el contrato **ReportRevisionDecision** para que el redactor responda a las incidencias del verificador sin conversacion libre, indicando que problemas acepta o rechaza, que cambios aplica y que evidencias conserva. En paralelo, los contratos **ReportDebateTurn** y **ReportDebateRecord** registran una vista ordenada del intercambio: borrador inicial, verificacion, revision, nueva verificacion y cierre del debate.

La version implementada mantiene una sola ronda de revision. Si el informe inicial es aceptado, el debate termina como **approved_without_revision**. Si el verificador solicita cambios, el redactor genera una version revisada y el verificador la evalua de nuevo. Si tras esa ronda quedan incidencias, el resultado pasa a **needs_human_review** o **blocked**, evitando aprobar silenciosamente un informe con afirmaciones no soportadas.

El coordinador del debate se ha aislado en **report_debate.py**. Este servicio no toma decisiones agenticas nuevas; solo compone los turnos y persiste los artefactos humanos y estructurados:

```
<directorio_del_informe>/evidence/report_debate.json  
<directorio_del_informe>/evidence/report_debate.md  
<directorio_del_informe>/evidence/final_report_initial.md  
<directorio_del_informe>/evidence/final_report_revision_001.md
```

La API expone el debate mediante GET `/runs/{run_id}/report-debate`. La interfaz web carga ese Markdown y lo muestra como seccion propia despues del informe final y la auditoria de ejecucion. Tambien se ha anadido `report_verifier` a la vista de agentes y se han mejorado las traducciones humanas de eventos para que campos como estado de verificacion, ronda, resumen, correcciones y cambios sean legibles sin abrir el JSON completo.

Sobre esa base se ha incorporado una conversacion agentica en la pestana de agentes. Esta vista toma los eventos runtime de supervisor y agentes, descarta ruido de job y ejecutores, y los presenta como mensajes ordenados con nombre, rol, accion resumida, fase y pequenos indicadores de confianza, memoria o veredicto. Cada mensaje permite seleccionar el agente correspondiente para ver el detalle tecnico cuando sea necesario. Asi, el evaluador humano puede seguir de un vistazo el hilo real de trabajo: supervisor, limpiador, estructurador, modelador, evaluador, redactor y verificador.

La validacion posterior con una run completa ha permitido refinar la tolerancia del verificador. En particular, se ha evitado que frases de acotacion como “no equivale a validacion industrial final” se interpreten como afirmaciones industriales no soportadas. Tambien se ha saneado la caida a fallback cuando el LLM devuelve un JSON con nombres de campo no canonicos: el sistema normaliza alias frecuentes antes de validar con Pydantic y, si aun asi debe usar el verificador determinista, no expone errores de validacion crudos en la interfaz.

El siguiente avance inicia la expansion controlada de herramientas para los agentes. Se han definido contratos Pydantic de especificacion, solicitud y observacion de herramienta, que permiten declarar herramientas elegibles por los LLM sin que estas sustituyan su decision. La primera herramienta es `evidence_lookup`, de solo lectura, orientada a consultar evidencia de una ejecucion: contexto del proyecto, configuraciones, metricas, evaluacion, artefactos, errores y politica del dataset. Su salida es una observacion estructurada con referencias citables como `metric:recall`, `evaluation:approved` o `artifact:evaluation_metrics`. La herramienta no modifica datos ni artefactos; solo amplia el espacio de observacion disponible para futuras decisiones agenticas. Ademias, el verificador del informe reutiliza el mismo catalogo cerrado de evidencias, reduciendo duplicacion y preparando una frontera comun para herramientas posteriores.

La segunda herramienta del catalogo es `threshold_analysis`. Esta herramienta lee el artefacto de predicciones registrado en el estado y calcula un diagnostico de sensibilidad: metricas por cuantiles de score, matriz de confusion, resumen de distribucion de scores y ventanas cercanas al umbral actual. Su salida incluye un guardarrail explicito: el analisis no selecciona un umbral ni sustituye la comparacion entre modelos. Para reforzar esta idea, la decision del modelador se ha ampliado con una estrategia declarada (`ModelingDecisionStrategy`), donde el agente debe explicitar si su hipotesis es seleccion de familia de modelo, calibracion de umbral, ajuste entre features y modelo, riesgo de particion, conservacion de baseline o falta de evidencia. Cuando la estrategia declarada es calibracion de umbral, el contrato exige mantener al menos una alternativa de otra familia de modelo en `comparison_candidates`. Asi el umbral se trata como una evidencia de diagnostico, no como la unica via de mejora del agente.

Como tercera ampliacion del espacio de modelado se ha incorporado One-Class

SVM, expuesto en la aplicacion como `one_class_svm`. Este modelo mantiene la frontera metodologica no supervisada del pipeline: se entrena solo con ventanas normales del `split train` y produce un `anomaly_score` comparable con el resto de detectores, donde valores mas altos indican mayor anomalia. La implementacion usa `StandardScaler` y `sklearn.svm.OneClassSVM`, valida hiperparametros como `kernel`, `nu`, `gamma`, `tol`, `shrinking` y `max_iter`, y persiste el modelo, las predicciones y el resumen de modelado igual que las familias anteriores. El agente modelador puede ahora comparar `isolation_forest`, `pca_reconstruction_error` y `one_class_svm`. En cambio, XGBoost se ha dejado fuera de esta etapa porque representa un baseline supervisado que solo debe introducirse cuando exista un dataset y una politica de etiquetas adecuados.

Como ajuste intermedio antes de continuar ampliando el modelador, se ha estabilizado la ejecucion NASA IMS desde la aplicacion. El formulario web usa ahora por defecto la muestra preextraida local, el modo `full` y la politica `nasa_ims_temporal_v1`, de modo que el preflight permite modelado, evaluacion y reporte sin presentar las etiquetas proxy como oficiales. Ademas, el endpoint de visualizacion puede construir una proyeccion PCA diagnostica cuando existen features persistidas pero aun no hay predicciones. En ese caso los puntos se muestran sin puntuacion de anomalia ni frontera, con una advertencia explicita para distinguir una run diagnostica de una run evaluada. La validacion local de NASA IMS confirma que completa el flujo, genera informe, metricas, predicciones y visualizacion; las metricas no se aprueban por $FPR=1.0$, lo que se conserva como limitacion metodologica en lugar de ocultarlo como fallo tecnico.

El siguiente paso de Fase 7 introduce perfiles explicitos de supervision para separar datasets binarios con etiquetas directas de secuencias run-to-failure. Las referencias revisadas sobre NASA IMS, la libreria de metricas pronosticas de NASA, XJTU-SY y PRONOSTIA/IEEE PHM 2012 justifican que los ensayos de degradacion se modelen como trayectorias temporales hasta fallo, no como clasificacion binaria por ventana salvo que exista una politica proxy declarada [4, 6, 9, 5]. Por ello se han anadido `supervision_profile`, `label_source` y `label_granularity` a `DatasetDescriptor` y `ProjectContext`. La declaracion inicial queda fijada asi:

CWRU:

```
supervision_profile = binary_fault_classification
label_source = official
label_granularity = file
```

NASA IMS:

```
supervision_profile = run_to_failure_degradation
label_source = none | temporal_proxy | synthetic
label_granularity = event | proxy_temporal | file
```

En NASA IMS, el valor `temporal_proxy` solo aparece cuando se usa la politica `nasa_ims_temporal_v1`. La aplicacion web muestra estos campos en el preflight para que el usuario vea antes de lanzar una run que tipo de evidencia y supervision se esta usando.

El segundo paso del mismo hito extiende el manifiesto comun sin crear otro formato paralelo. La lectura y escritura CSV de `CommonManifestRecord` se ha centralizado en `common_manifest.py`, eliminando duplicacion entre adaptadores, politica temporal NASA IMS y benchmarks sinteticos. El manifiesto NASA IMS base conserva ahora los timestamps por snapshot y anade en `metadata_json` los metadatos run-to-failure generales:

```
failure_event_time
failure_mode
end_of_life_policy
official_window_labels = false
label_source = none
label_granularity = event
```

Cuando se aplica `nasa_ims_temporal_v1`, esos metadatos se preservan y solo se declara una nueva capa de supervision proxy:

```
label_source = temporal_proxy
label_granularity = proxy_temporal
label_policy_id = nasa_ims_temporal_v1
```

Asi, el pipeline dispone de informacion temporal/fallo suficiente para las siguientes fases, pero sigue sin presentar etiquetas por ventana como oficiales.

El tercer paso propaga esa informacion hasta el nivel de ventana. En lugar de crear un estructurador especifico para NASA IMS, se ha extendido la cadena existente. El ejecutor `cleaning.py` conserva en cada fichero limpio:

```
source_path
condition_id
asset_id
run_id
timestamp_start
timestamp_end
```

Despues, `structuring.py` transforma esos campos en columnas por ventana dentro de `windows_features.csv`. Las nuevas columnas son:

```
run_id
asset_id
condition_id
timestamp_start
timestamp_end
time_since_start_seconds
time_to_failure_seconds
relative_life
```

La vida relativa solo se calcula cuando el run tiene timestamps y evento de fallo declarado. Para evitar fugas, `modeling.py` y `run_visualization.py` tratan estas columnas como metadatos y no como features numericas de entrenamiento o proyeccion. El ejecutor de modelado las preserva en `predictions.csv`, preparando el siguiente paso de evaluacion temporal de degradacion.

El cuarto paso incorpora esa evaluacion temporal sin crear un evaluador paralelo. Se ha extendido `evaluation.py`, que sigue siendo el propietario canonico de las metricas de prediccion, para que detecte de forma generica si `predictions.csv` contiene `relative_life`, `anomaly_score` y `predicted_anomaly`. Cuando esos campos estan disponibles, el artefacto `metrics.json` declara dos familias: `binary_classification` y `run_to_failure_degradation`. La primera mantiene las metricas previas de precision, recall, F1, ROC-AUC, PR-AUC, FPR y matriz de confusion; la segunda calcula

metricas por trayectoria, incluyendo primera alerta, tiempo hasta deteccion, `lead_time` hasta fallo, falsas alarmas en tramo nominal, persistencia de alertas, correlacion Spearman del score con la vida relativa, pendiente, separacion inicial-final, monotonicidad y fallos no detectados.

Esta separacion evita presentar una politica proxy temporal como si fuera una etiqueta oficial por ventana. Si el origen de etiqueta no viaja dentro del CSV de predicciones, el evaluador deja una advertencia en el contexto binario y el resumen Markdown muestra una seccion diferenciada de degradacion. A su vez, el grafo copia al estado solo un resumen plano de esta familia en `MetricsReport.extra`, manteniendo compatible el contrato existente y permitiendo que los agentes posteriores razonen sobre `lead_time`, falsas alarmas nominales y tendencia sin mezclar estructuras anidadas en el estado global.

El quinto paso traslada esta informacion a la aplicacion visual sin crear un endpoint paralelo. El contrato `RunVisualizationData` se ha extendido con un bloque opcional `temporal_series`, formado por `TemporalSeriesData`, `TemporalRunSeries` y `TemporalSeriesPoint`. El servicio `run_visualization.py` construye dicho bloque directamente desde `predictions.csv`; selecciona como eje temporal, por prioridad, `relative_life`, `time_since_start_seconds` o `window_index`. Para cada trayectoria conserva el score de anomalia, el umbral, la primera alerta, el tiempo restante hasta fallo cuando existe, un marcador de fallo estimado y una muestra compacta de puntos para no saturar la API.

La interfaz web mantiene la misma pestana de visualizacion, pero anade una vista principal de degradacion antes de la proyeccion PCA. Esta vista dibuja la curva del `anomaly_score`, la linea de umbral, el primer aviso y el fallo estimado si los metadatos lo permiten. Si las predicciones no contienen metadatos temporales suficientes, el backend devuelve `temporal_series.available = false` con advertencias legibles y la PCA sigue funcionando como vista secundaria. De esta forma, NASA IMS y futuros datasets run-to-failure pueden inspeccionarse como trayectorias temporales sin perder compatibilidad con las runs binarias de CWRU.

El sexto paso hace que los agentes posteriores sean conscientes del perfil de supervision. En `modeler.py`, el contexto entregado al LLM y al fallback incluye el perfil de supervision, la fuente de etiquetas y su granularidad. Para el perfil de degradacion temporal, el fallback selecciona `pca_reconstruction_error` como indicador temporal interpretable y declara que la hipotesis se evaluara por tendencia, primera alerta y falsas alarmas nominales, no por F1 como objetivo principal. Ademas, se rechaza una decision cuyo `strategy_type` principal sea `threshold_calibration` en este perfil, manteniendo el umbral como elemento auxiliar para disparar alertas.

En `evaluator.py`, la politica binaria previa se conserva para CWRU y perfiles equivalentes: recall y tasa de falsos positivos siguen siendo los umbrales deterministas. En cambio, cuando el contexto declara `run_to_failure_degradation`, la aprobacion se calcula con metricas temporales: deteccion antes del fallo, falsas alarmas en tramo nominal y tendencia Spearman del score. En ese caso, `min_recall_required` y `max_false_positive_rate` quedan a `null` en la decision del agente para no presentar el criterio binario como vinculante. La ausencia de etiquetas oficiales por ventana no invalida por si sola la run si existen metricas temporales, aunque se conserva como limitacion metodologica.

Por ultimo, `report_writer.py` y `report_verifier.py` incorporan la misma frontera. El redactor presenta lead time, falsas alarmas nominales y tendencia como me-

tricas principales en informes run-to-failure, dejando F1 solo como apoyo si procede de una politica proxy. El verificador bloquea cualquier afirmacion que atribuya etiquetas oficiales o *ground truth* oficial cuando `label_source` no es `official`. La herramienta `evidence_lookup` expone referencias cerradas para `supervision_profile`, `label_source`, `label_granularity` y `metric_extra:*`, de forma que los agentes puedan citar evidencia temporal sin inventar campos.

La comprobacion posterior se realizo con la run `fase7-paso7-nasa-common-long-v2`, generada sobre una secuencia NASA-like local de 24 snapshots mediante el runner comun y la politica `nasa_ims_temporal_v1`. La ejecucion produjo 168 ventanas con `label_source=temporal_proxy` y `label_granularity=proxy_temporal` preservados hasta `predictions.csv`; `metrics.json` declaro la familia `run_to_failure_degradation`, el informe fue aprobado por el verificador y la visualizacion temporal quedo disponible. Durante esta validacion se ajusto el verificador para no bloquear frases negadas como “las etiquetas no son oficiales”, manteniendo el bloqueo para afirmaciones positivas de oficialidad sin evidencia.

El siguiente refinamiento convierte esa evaluacion temporal en una herramienta comparativa dentro de la aplicacion. La ruta existente `GET /runs/compare`, implementada sobre `run_registry.py`, se ha extendido para que cada fila de comparacion incluya el perfil de supervision, la fuente y granularidad de etiquetas, el modelo usado y las familias metricas disponibles. Cuando una run contiene metricas `run_to_failure_degradation`, la respuesta anade un bloque especifico con comparaciones de deteccion antes de fallo, `lead_time` medio, falsa alarma nominal media, tendencia Spearman del score, fallos perdidos y separacion inicial-final si esta disponible. La interfaz web muestra este bloque antes de las metricas binarias, de forma que el analista compare modelos temporales por comportamiento de degradacion y no por F1 como criterio principal. Esta decision mantiene la compatibilidad con CWRU y prepara el panel de investigacion para comparar familias como `pca_reconstruction_error`, `isolation_forest` y `one_class_svm` sobre el mismo protocolo run-to-failure.

A partir de esa comparacion se ha incorporado una capa de monitorizacion de estado de salud. La serie temporal ya no se limita a mostrar el score bruto: `run_visualization.py` deriva para cada ventana un `risk_index`, un `health_index`, un `health_state` y una razon textual del estado. Si existe umbral, el riesgo se calcula a partir de la relacion entre `anomaly_score` y `threshold`; si no existe, se usa una normalizacion interna del score como aproximacion diagnostica. Los estados resultantes son `nominal`, `watch`, `warning` y `critical`. La trayectoria completa conserva tambien el estado actual, el riesgo y salud actuales, el numero de puntos en alerta y el tiempo restante hasta fallo solo cuando este aparece como metadato de un replay historico. De esta forma, la aplicacion se aproxima a una monitorizacion industrial de activos sin afirmar una prediccion RUL real cuando el modelo aun no la produce.

El siguiente ajuste traslada esa informacion a una lectura mas humana dentro de la pestana `Visualizacion`. El contrato `RunVisualizationData` declara ahora el perfil de supervision, fuente y granularidad de etiquetas, modelo usado, familias metricas, metricas primarias y metricas auxiliares. Para `run_to_failure_degradation`, las metricas principales pasan a ser deteccion antes de fallo, `lead_time`, falsas alarmas nominales, tendencia del score, fallos perdidos y separacion inicio-final. Precision, recall, F1, FPR, ROC-AUC y PR-AUC se conservan como metricas binarias auxiliares cuando proceden de una politica proxy.

En la interfaz, la misma pestana se vuelve sensible al perfil. Si la run es run-to-

failure, se muestra un panel de control del motor con estado actual, salud, riesgo, primer aviso, `lead_time`, alertas, puntos criticos, una tira de ventanas coloreada por estado y un flujo compacto `ventanas ->detector ->estado ->agente`. La curva temporal existente se mantiene como lectura tecnica, pero incorpora bandas de estado para facilitar la interpretacion. La proyeccion PCA se conserva como diagnostico 2D de features; la interfaz aclara que la elipse dibujada es una aproximacion visual y no la frontera real del detector ni la metrica principal del perfil temporal.

Tras revisar la interpretacion visual, se ha separado explicitamente un pico puntual de una alerta sostenida. Cada ventana se evalua localmente, por lo que un score critico aislado puede ir seguido de una ventana nominal sin implicar que el motor haya fallado y se haya recuperado. Para evitar esa lectura, la visualizacion distingue entre **primer pico** y **aviso sostenido**, definido como una racha minima de ventanas consecutivas en **warning** o **critical**. Tambien muestra picos aislados, episodios de alerta y racha maxima. El fallo mostrado sigue siendo un marcador historico del replay cuando existe `time_to_failure_seconds`; no se presenta como una prediccion RUL real del modelo.

5.33. Fase 8: adaptacion agentica del perfil run-to-failure

Con la base temporal y visual del perfil `run_to_failure_degradation` preparada, la siguiente fase desplaza el centro de gravedad hacia los agentes Qwen/LLM. El objetivo ya no es solo que el pipeline calcule metricas temporales y las muestre en la interfaz, sino que los agentes gobiernen la investigacion dentro de contratos estrictos: observar evidencia temporal, formular hipotesis, elegir herramientas, seleccionar modelo y politica de alerta, debatir resultados, recomendar acciones y generar memoria reutilizable.

La metodologia mantiene la separacion de seguridad: los LLM no ejecutan codigo arbitrario ni transforman datos directamente. Sus decisiones se validan mediante contratos Pydantic y los ejecutores deterministas aplican las acciones materiales. La diferencia respecto a la fase anterior es de protagonismo: `run_to_failure_degradation` pasa a ser el caso principal para estudiar si agentes locales pequenos pueden conducir una monitorizacion industrial compleja con evidencia, debate y auditoria. Histeresis avanzada, RUL experimental y autoencoders se trataran despues de cerrar esta base agentica, para que esas ampliaciones tambien dependan de decisiones de agentes y no solo de reglas fijas.

El primer hito de esta fase introduce un paquete de evidencia temporal para agentes. La herramienta `evidence_lookup` incorpora una seccion **temporal** que resume, sin cargar CSV completos en el estado, el perfil `run_to_failure_degradation`: estado actual de la trayectoria, primer pico, primer aviso sostenido, episodios de alerta, picos aislados, racha maxima, referencia de fallo historico, metricas de degradacion y advertencias sobre etiquetas proxy y RUL no estimado. Esta evidencia se deriva de la misma serie temporal que alimenta el panel visual, evitando una interpretacion paralela para agentes.

Ademas, se refuerza el papel del agente **cleaner**. Su prompt LLM recibe contexto temporal especifico para razonar sobre canal, frecuencia, valores no finitos y normalizacion como decisiones que condicionan la continuidad temporal y la estabilidad del score posterior. El agente no decide fallos ni estima RUL; mantiene su frontera de calidad

de senal, pero con protagonismo dentro del perfil temporal que gobiernan los agentes Qwen/LLM.

El segundo hito convierte esa evidencia agregada en herramientas agenticas especificas. Se incorporan `temporal_health_lookup`, que devuelve estado de salud, primer pico, aviso sostenido, episodios, picos aislados, racha maxima y referencia de fallo historico, y `degradation_metrics_lookup`, que devuelve metricas temporales primarias y contexto de etiquetas. Ambas herramientas son de solo lectura y generan referencias citables; no aprueban ejecuciones, no eligen modelos y no estiman RUL. Con ello, los agentes ganan capacidad de eleccion informada sin romper la arquitectura: `cleaner`, `structurer`, `modeler` y `evaluator` siguen tomando decisiones en contratos propios, mientras que los ejecutores deterministas siguen aplicando las acciones materiales.

El tercer hito especializa al agente `modeler` como estrategia run-to-failure. En lugar de introducir un contrato paralelo, se extiende `ModelingDecisionStrategy` con tres campos compatibles: `tool_names`, `optimization_targets` y `alert_policy`. Para el perfil `run_to_failure_degradation`, la decision del modelador debe declarar el uso de `temporal_health_lookup` y `degradation_metrics_lookup`, citar referencias temporales o metricas de degradacion, optimizar deteccion antes de fallo, lead time, falsas alarmas nominales y tendencia del score, y describir una politica que separe pico aislado, aviso sostenido y umbral auxiliar. F1 queda restringida a metrica auxiliar. El fallback seguro tambien se adapta a este marco: conserva `pca_reconstruction_error` como score interpretable, pero deja herramientas, objetivos, evidencia y alternativas comparables trazadas.

El cuarto hito extiende al agente `evaluator` como auditor operacional del perfil temporal. `EvaluationDecision` incorpora campos compatibles para registrar herramientas, referencias de evidencia, una valoracion operacional, puntos de debate temporal y comprobaciones de guardarrail. En `run_to_failure_degradation`, el evaluador debe citar `temporal_health_lookup` y `degradation_metrics_lookup`, debatir pico aislado frente a aviso sostenido, declarar que RUL no esta estimado, mantener F1 como metrica auxiliar y advertir que las etiquetas proxy no son oficiales por ventana. La base del debate temporal queda integrada en la decision del evaluador y se conserva en la memoria de decisiones para futuras runs, sin crear todavia un contrato separado de debate.

El quinto hito traslada esa decision agentic a al panel principal de visualizacion. El contrato `RunVisualizationData` incorpora `AgentOperationalRecommendation`, derivado de los mensajes JSON persistidos de `evaluator` y contextualizado con la ultima estrategia de `modeler`. La interfaz muestra la recomendacion operacional, confianza, siguiente accion, evidencia citada, herramientas usadas, guardarrails, cautelas y puntos de debate. Si una ejecucion esta aprobada pero mantiene limitaciones metodologicas, la recomendacion se presenta como cautela y no como aprobacion limpia. De este modo, el panel no sustituye al agente por reglas de interfaz: hace visible la interpretacion Qwen/LLM y conserva su trazabilidad dentro de la monitorizacion run-to-failure.

El sexto hito adapta el post-mortem y la memoria agentic a al perfil temporal sin crear un sistema paralelo. En lugar de definir un contrato independiente `RunToFailurePostMortem`, se extienden los episodios canonicos `DecisionEpisode` y los candidatos `MemoryCandidate`. El servicio `decision_memory.py` genera ahora episodios del `modeler` con la familia de modelo elegida, hipotesis, herramientas temporales, referencias de evidencia, objetivos de optimizacion y politica de alerta. Para `run_to_failure_degradation`, estos episodios incorporan lecciones sobre metricas temporales, F1 como metrica auxiliar y

necesidad de declarar la politica de aviso.

La memoria del `evaluator` tambien se especializa para este perfil. Las metricas de degradacion almacenadas en `MetricsReport.extra`, como `degradation_mean_lead_time_to_failure` o `degradation_mean_false_alarm_rate_nominal`, se trasladan a `after_metrics` del episodio y al contenido indexable del candidato de memoria. Ademas, una aprobacion temporal con limitaciones se marca como `partially_supported`, de forma que la memoria futura la trate como caso de frontera y no como ejemplo positivo simple. Los tags del candidato incluyen tradeoffs y guardarrails como `rul_not_estimated`, `sustained_alert_required` o `isolated_spike_not_failure`, facilitando su recuperacion vectorial por consultas futuras.

Finalmente, el grafo escribe artefactos de memoria del `modeler` cuando `PipelineMemoryConfig.g` esta activo: `modeler_decision_episode`, su informe legible, `modeler_memory_candidate` y su informe. Las ejecuciones sin memoria activa no cambian comportamiento. Con ello, el perfil temporal empieza a cerrar el ciclo de aprendizaje: las decisiones Qwen/LLM dejan una traza estructurada e indexable sobre estrategias de degradacion, no solo un resultado numerico final.

A partir de este punto se define una hoja de ruta especifica para elevar el sistema de memoria. El diagnostico distingue entre la memoria RAG local ya implementada y una base vectorial industrial completa: actualmente `LocalJsonVectorMemoryStore` persiste colecciones por agente en JSON, calcula similitud coseno y puede usar `qwen3-embedding:0.6b` via Ollama, con `LocalHashEmbeddingModel` como fallback reproducible. Esta solucion es suficiente como baseline auditable del TFM, pero no incorpora todavia Qdrant, Weaviate, Chroma, Milvus, `pgvector`, busqueda hibrida, reranking ni evaluacion sistematica del retrieval.

La hoja de ruta avanzada propone primero hacer visible la memoria en frontend: estado del recuerdo, ciclo candidato-indexado-recuperado-usado-auditado, consultas realizadas, similitudes, filtros, modelo de embeddings y citas usadas por cada decision. Solo despues se plantea mejorar el backend con Qdrant, reranking Qwen, busqueda hibrida y consolidacion/reflexion de recuerdos. Esta secuencia conserva el principio agentico del trabajo: la memoria informa a los agentes Qwen y aumenta su capacidad deliberativa, pero no sustituye su decision ni convierte la ejecucion en un conjunto de reglas deterministas.

El primer hito de esta linea transforma la vista de memoria persistida de la pestaña de agentes en un cockpit operativo. La interfaz muestra contadores por coleccion, recuerdos reutilizables, datasets, excluidos, distribucion por rol y origen, asi como un ciclo visual de candidatos, indexados, recuperados en runtime, usados por agentes y auditorias. Los identificadores de memoria citados por eventos agenticos pasan a ser navegables y el detalle del recuerdo incorpora veredicto, origen, resultado, modelo de embedding, dimension, `vector_id`, tags, metricas, contenido indexable y ruta de origen. Este hito no cambia la API ni el store JSON local; su funcion es hacer observable la memoria existente antes de introducir Qdrant, reranking o busqueda hibrida.

El segundo hito completa la observabilidad de retrieval. El grafo persiste ahora, para cada recuperacion de memoria dentro del flujo principal, un `memory_query.json` separado del `retrieved_memory_context.json`. La query conserva `query_id`, agente objetivo, texto de consulta, dataset, `run_id`, `decision_id`, contexto de decision, roles permitidos, `top_k` y `min_similarity`. El contexto recuperado conserva items, ranking, similitud, uso previsto, backend y modelo de embeddings.

La telemetria runtime distingue ahora fases del retrieval mediante `retrieval_event`:

consulta no disponible, consulta solicitada, contexto recuperado, memoria usada y memoria ignorada por el agente. En la interfaz, el cockpit muestra backend, embedding, parametros de consulta, extracto de query, items recuperados con score, rol, origen y dataset, ademas de recuerdos citados o ignorados. Esta trazabilidad permite reconstruir una decision con memoria desde la UI y prepara el siguiente paso: que el `modeler` consulte memoria antes de su primera decision de modelo.

El tercer hito lleva esa recuperacion a la decision inicial del modelador y anade gobierno operativo de recuerdos. `ModelingDecision` declara ahora `memory_context_id`, `used_memory_context`, `memory_record_ids`, `memory_usage_summary` y `memory_record_uses`. La consulta `build_modeler_memory_query` incluye dataset, objetivo, perfil de supervision, fuente de etiquetas, resumen de features y preguntas sobre familia de modelo, politica temporal, falsos avisos, lead time y uso auxiliar de F1. El prompt Qwen recibe los recuerdos recuperados y una guia derivada de warnings o casos frontera, pero la validacion impide citar recuerdos no recuperados y exige mitigacion explicita si se usa una advertencia.

La memoria se mantiene como capacidad transversal, no como herramienta exclusiva del modelador. El backend conserva colecciones por agente (`cleaner_memory`, `structurer_memory`, `modeler_memory`, `evaluator_memory`, `report_writer_memory` y memoria metodologica compartida), y el mismo contrato de recuperacion/cita puede ampliarse a cada nodo del grafo. Para facilitar benchmarks y pruebas exploratorias, la API y el cockpit permiten excluir un recuerdo del RAG sin perder trazabilidad, restaurarlo o borrarlo del indice local. Esta curacion manual prepara comparaciones con memoria completa, memoria filtrada o memoria desactivada, manteniendo el protagonismo de Qwen: la memoria informa su decision, pero no la sustituye por una politica determinista.

Para comprobar si la memoria produce algun efecto observable se ha implementado un benchmark offline sobre snapshots persistidos. El servicio `memory_effect_benchmark.py` y el script `benchmark_memory_effect.py` cargan una o varias runs, leen los artefactos `{agent}_retrieved_memory_context`, contrastan recuerdos recuperados frente a recuerdos citados y clasifican cada ejecucion como memoria no observada, consulta sin recuerdos, retrieval ignorado, memoria usada o uso invalido. Si se indica una run baseline, el informe anade deltas de configuracion y metricas sin atribuir causalidad automaticamente.

Como primera evidencia real se ha comparado `cwru-memory-transversal-fase4-001` con `cwru-memory-transversal-fase4-002-11m`. El informe generado en `codigo/reports/memory_b` concluye que la memoria fue recuperada y citada por `structurer` y `evaluator`, sin uso invalido. Las metricas binarias permanecen iguales respecto al baseline. Por tanto, en este par concreto la memoria si aporta deliberacion y trazabilidad agentica, pero no demuestra todavia una mejora numerica de rendimiento.

El siguiente refinamiento incorpora un quality gate determinista sin consolidar recuerdos. El servicio `memory_quality_gate.py` clasifica cada recuerdo recuperado como `pass`, `caution` o `exclude_candidate`. La decision se basa en similitud, coherencia de dataset y perfil, rol del recuerdo, veredicto humano, tags de exclusion manual y procedencia del recuerdo. Las cautelas no bloquean automaticamente al agente: sirven para dejar evidencia de que un caso frontera, warning o recuerdo parcial requiere mitigacion explicita.

El benchmark se ha ampliado ademas con variantes controladas como `memory_off`, `memory_full`, `memory_filtered`, `memory_conflict_excluded` y `retrieval_only`. Al

regenerar el smoke CWRU anterior, la run base queda etiquetada como `memory_off` y la run con Qwen como `memory_full`. Los recuerdos citados por `structurer` y `evaluator` pasan el gate sin advertencias. Por decision metodologica no se implementa todavia memoria consolidada o reflexiones globales, ya que podrian fijar prematuramente conclusiones y limitar futuras deliberaciones de los agentes.

Como primer paso hacia una memoria RAG con base vectorial externa se ha implementado `QdrantVectorMemoryStore`. Este backend mantiene el mismo contrato `VectorMemoryStore` que el indice JSON local, de forma que los agentes continuan trabajando con `AgentMemoryQuery` y `RetrievedMemoryContext` sin acoplarse al proveedor de almacenamiento. Cada recuerdo se guarda como punto Qdrant con embedding denso, metadatos filtrables y el `ReasoningMemoryRecord` completo en el payload. El identificador del punto se deriva de `memory_record_id` mediante UUID determinista, lo que conserva la trazabilidad de upserts y eliminaciones.

La seleccion de backend se centraliza en `get_default_vector_memory_store`: por defecto se mantiene `LocalJsonVectorMemoryStore` como baseline reproducible, mientras que `TFM_MEMORY_BACKEND=qdrant` activa Qdrant con host, clave, timeout y distancia configurables. Esta decision permite avanzar hacia un RAG industrial sin perder la capacidad de ejecutar pruebas locales estables ni comparar el efecto de la memoria contra una linea base. El hito se valida con tests mockeados de HTTP que cubren insercion, busqueda en coleccion de agente y memoria compartida, scroll, borrado y seleccion automatica de backend. Quedan pendientes la migracion JSON-Qdrant, un smoke test con servidor Qdrant real y la comparacion de retrieval entre ambos backends.

El siguiente hito completa esa primera validacion operacional. Se incorpora el servicio `memory_backend_migration.py`, que permite migrar registros entre dos implementaciones de `VectorMemoryStore` y comparar los IDs recuperados por ambos backends ante las mismas consultas. La migracion concreta `LocalJsonVectorMemoryStore -> QdrantVectorMemoryStore` lee los `ReasoningMemoryRecord` ya persistidos, los reindexa en Qdrant y genera consultas de verificacion por agente y dataset. Para que la comparacion sea metodologicamente justa, se crea un espejo local temporal reembebido con el mismo proveedor usado en Qdrant; asi se evita comparar un indice Qdrant nuevo con vectores JSON antiguos generados por otro modelo de embeddings.

Tambien se anade el script `codigo/scripts/migrate_memory_to_qdrant.py` y un servicio Docker opcional `qdrant` bajo el profile `memory-qdrant`. En el smoke real se levanta Qdrant en `http://127.0.0.1:6333` y se migra la memoria existente usando `local_hash_embedding` para aislar la infraestructura de la disponibilidad de Ollama. El resultado migra 20 recuerdos distribuidos en `evaluator_memory`, `modeler_memory` y `structurer_memory`. Las tres consultas de verificacion presentan solapamiento medio 1.0 y orden exacto 3/3 respecto al espejo local reembebido. Esto valida que Qdrant puede reproducir el retrieval del baseline bajo el mismo embedding, aunque todavia queda ejecutar una run agentica real con `TFM_MEMORY_BACKEND=qdrant` y repetir el flujo con `qwen3-embedding:0.6b`.

Como comprobacion posterior, los scripts de ejecucion con memoria se adaptan para construir el store mediante `get_default_vector_memory_store`. De este modo, una misma run puede usar memoria JSON o Qdrant modificando `TFM_MEMORY_BACKEND`, sin cambiar el contrato de los agentes ni el grafo. Se ejecuta un smoke CWRU con `TFM_MEMORY_BACKEND=qdrant`, `local_hash_embedding` y memoria de decision desactivada para aislar la infraestructura. La run `cwru-memory-qdrant-m4-4-smoke-001` finaliza aprobada y persiste contextos de memoria para `structurer`, `modeler` y `evaluator`,

todos con `retrieval_backend=qdrant_vector_memory_store`. El benchmark de efecto de memoria la clasifica como `retrieval_only`: Qdrant recupera memoria y no hay uso invalido, pero al ejecutarse sin LLM no se evalua todavia la cita ni la influencia deliberativa de Qwen. Esa validacion queda reservada para una run posterior con agentes LLM y, preferentemente, con el perfil `run_to_failure_degradation`.

5.34. Fase 9: cierre del perfil run-to-failure avanzado

La Fase 9 culmina la linea temporal iniciada en Fase 7 y agentificada en Fase 8. El objetivo no es crear un runner paralelo para run-to-failure, sino elevar el perfil `run_to_failure_degradation` reutilizando el runner comun, los contratos Pydantic, las herramientas agenticas, la memoria y el registro de runs ya existentes. De este modo, la mejora no desplaza a los agentes Qwen/LLM: les proporciona mas evidencia y mas familias de modelo dentro de fronteras estrictas.

El primer bloque implementa una suite canonica de modelos temporales. En `experiment_protocol` se define un plan comparable para PCA por error de reconstruccion, Isolation Forest, One-Class SVM y, posteriormente, `autoencoder_dense`. El script `run_run_to_failure_model_suite` materializa la suite usando el runner comun y persiste tabla de resultados, comparacion y artefactos por run. Con ello, una decision del `modeler` puede transformarse en un plan experimental auditable sin crear logica paralela.

El segundo bloque introduce politicas temporales versionadas. La politica `temporal_health_policy` diferencia primer pico, aviso sostenido, onset confirmado, episodios de alerta, picos aislados y racha maxima. Las metricas de evaluacion se centran en degradacion confirmada antes de fallo, `lead time` persistente, falsas alarmas nominales y tendencia Spearman del score. Las metricas binarias se conservan como apoyo cuando proceden de etiquetas proxy, pero dejan de ser el criterio principal del perfil.

El tercer bloque incorpora un Health Indicator avanzado. La politica `health_indicator_policy` genera un indicador causal de salud a partir del score temporal y calcula caida del indicador, monotonidad, robustez, volatilidad nominal, fuerza de tendencia y un score agregado de calidad. Estas metricas se anaden a `MetricsReport.extra`, a la tabla de comparacion de la suite y al contexto que reciben evaluador y redactor. El Health Indicator se presenta como evidencia auxiliar de degradacion, no como pronostico industrial completo.

El cuarto bloque implementa una herramienta de readiness para modelos temporales avanzados. `temporal_model_readiness_assessor` evalua numero de ventanas, ventanas nominales de entrenamiento, numero de features, trayectorias disponibles y riesgo de fuga temporal. Devuelve indicadores como `autoencoder_ready`, `rul_ready` y motivos de bloqueo o cautela. El `modeler` solo puede proponer modelos avanzados si cita esta herramienta y referencias `readiness:*`; asi se evita que Qwen use complejidad adicional sin evidencia suficiente.

El quinto bloque incorpora `autoencoder_dense` con PyTorch. La implementacion vive en el ejecutor de modelado y mantiene hiperparametros cerrados: capas ocultas, dimension latente, `learning_rate`, `batch_size`, `max_epochs`, paciencia, regularizacion, cuantil de umbral y dispositivo CPU por defecto. El agente no inventa arquitecturas ni ejecuta codigo. El ejecutor guarda `autoencoder_dense.pt`, `autoencoder_dense_scaler.joblib`, `autoencoder_dense_training_curve.json`, `predictions.csv` y `modeling_summary.json`. Si PyTorch no esta disponible, el error queda declarado de forma controlada.

El sexto bloque refuerza la robustez de las salidas LLM. La clase `OllamaJSONClient`

realiza ahora una reparacion JSON estructurada antes de declarar una respuesta irre-
cuperable. Ademias, `supervisor`, `evaluator`, `report_writer` y `report_verifier` rea-
lizan un reintento de contrato ante salidas validables pero incorrectas. Cuando el pro-
blema es una violacion del contrato, la traza se registra como `Guardrail correction`;
el texto `Fallback after LLM failure` se reserva para fallos tecnicos del proveedor,
conexion o JSON irrecuperable. Esta distincion mejora la evidencia de un proyecto
centrado en agentes: el sistema no oculta errores, pero tampoco presenta como fallo
del LLM una correccion metodologica esperada.

La fase queda cerrada sin implementar RUL completo ni LSTM autoencoder. Esa
decision es deliberada: el sistema ya puede separar anomalia, degradacion, Health In-
dicator, readiness y recomendacion agentica, pero no debe comunicar vida util restante
con falsa precision. RUL experimental y segundo dataset run-to-failure quedan como
trabajo futuro sobre una base temporal ya validada.

Capítulo 6

Experimentos

6.1. Protocolo experimental

La Fase 2 introduce un protocolo experimental local para pasar de una unica ejecucion valida a un conjunto pequeno de ejecuciones comparables. El protocolo se implementa en:

```
codigo/app/services/experiment_protocol.py
```

La unidad principal es un manifiesto ‘ExperimentPlan’, compuesto por varias entradas ‘ExperimentSpec’. Cada entrada fija un ‘run_id’, una descripcion y una configuracion de modelado validada mediante ‘ModelingConfig’. La ejecucion del plan reutiliza el wrapper persistido del pipeline y, al finalizar, invoca el registro consultable para comparar metricas entre runs.

Cada plan conserva tres artefactos agregados:

- ‘experiment_plan.json’, con el manifiesto experimental;
- ‘comparison.json’, con la comparacion estructurada;
- ‘results_table.md’, con la tabla resumida para documentacion.

Ademas, los modelos, predicciones, metricas e informes finales se escriben dentro de la carpeta de cada experimento para evitar sobrescrituras entre runs comparables.

6.2. Plan CWRU Isolation Forest Threshold

El primer plan experimental real se identifica como:

```
cwru_iforest_threshold_v1
```

El objetivo es estudiar una variacion controlada del umbral de decision del modelo base Isolation Forest. No se introduce ningun algoritmo nuevo, ya que la regla de la arquitectura exige que todo modelo disponible para un agente tenga un ejecutor determinista reproducible.

Los dos runs ejecutados han sido:

- ‘cwru_iforest_threshold_v1_baseline_threshold_099’;

- ‘cwru_iforest_threshold_v1_conservative_threshold_100’.

Ambos mantienen constante el dataset, la limpieza, la estructuracion temporal, el algoritmo, la semilla y ‘n_estimators=200’. La unica variable modificada es ‘threshold_quantile’: 0.99 para el baseline y 1.00 para la variante conservadora.

Tabla 6.1: Comparacion local de umbral con Isolation Forest sobre CWRU

Experimento	Umbral	Precision	Recall	F1	FPR
Baseline	0.99	0.9991	1.0000	0.9996	0.0513
Conservador	1.00	0.9996	1.0000	0.9998	0.0256

Los dos runs quedan aprobados por el agente evaluador. La variante conservadora reduce la tasa de falsos positivos de 0.0513 a 0.0256 y mejora ligeramente la precision y el F1-score, manteniendo recall 1.0000. En este benchmark, por tanto, el umbral conservador resulta preferible para reducir falsas alarmas sin penalizar la deteccion de fallos.

Los artefactos agregados del plan quedan en:

`codigo/experiments/cwru_local/cwru_iforest_threshold_v1/`

6.3. Comparacion Agentica de Ventanas CWRU

La Fase 3 incorpora un experimento donde el agente estructurador no solo elige una configuracion, sino que propone alternativas comparables. La ejecucion se lanzo con:

```
python -m codigo.scripts.run_cwru_agentic_window_comparison \
  --model qwen3.5:4b \
  --plan-id cwru-agentic-window-qwen-fase3
```

Qwen propuso tres configuraciones de ventana, todas con solape del 50 %: 2048 muestras como seleccion principal, 1024 muestras para aumentar resolucion temporal y 4096 muestras para aumentar contexto por ventana. El protocolo experimental materializo esas propuestas sin anadir configuraciones no decididas por el agente.

Tabla 6.2: Comparacion agentica de ventanas sobre CWRU

Ventana	Solape	Precision	Recall	F1	FPR
2048	0.50	0.9991	1.0000	0.9996	0.0513
1024	0.50	0.9996	0.9999	0.9998	0.0212
4096	0.50	0.9991	1.0000	0.9996	0.0517

La configuracion de 1024 muestras obtuvo la mejor precision, el mejor F1-score y la menor tasa de falsos positivos. La configuracion principal de 2048 muestras mantuvo recall 1.0000, pero produjo mas falsas alarmas. Este resultado apoya la idea central del sistema: los agentes formulan hipotesis tecnicas y la aplicacion las convierte en evidencia experimental trazable.

Los artefactos quedan en:

`codigo/experiments/cwru_local/cwru-agentic-window-qwen-fase3/`

6.4. Comparacion Agentica de Modelos CWRU

Tras seleccionar la ventana de 1024 muestras, se ejecuto una comparacion agentica de modelos. El modelador propuso Isolation Forest, PCA con error de reconstruccion y una variante conservadora de Isolation Forest:

```
python -m codigo.scripts.run_cwru_agentic_model_comparison \
--model qwen3.5:4b \
--plan-id cwru-agentic-model-qwen-fase3
```

Tabla 6.3: Comparacion agentica de modelos sobre CWRU

Modelo	Umbral	Precision	Recall	F1	FPR
Isolation Forest	0.99	0.9996	0.9999	0.9998	0.0212
PCA reconstruction	0.99	1.0000	0.9623	0.9808	0.0000
Isolation Forest cons.	1.00	0.9999	0.9994	0.9996	0.0085

PCA elimina falsos positivos en este protocolo, pero reduce el recall a 0.9623. Esta comparacion es relevante porque rompe el patron de resultados perfectos y muestra un trade-off mas creible entre sensibilidad y falsas alarmas.

6.5. Prueba Smoke NASA IMS con Agentes Qwen

Dentro de la Fase 3 se ha ejecutado una prueba smoke sobre una estructura NASA IMS sintetica preextraida. La ejecucion usa el grafo completo y agentes Qwen/Ollama, pero bloquea deliberadamente el modelado final porque el dataset run-to-failure aun no dispone de etiquetas por ventana ni de una politica de particion temporal validada.

```
python -m codigo.scripts.run_nasa_ims_qwen_smoke \
--model qwen3.5:4b \
--run-id nasa-ims-qwen-smoke-fase3-qwen
```

El snapshot queda persistido en:

```
codigo/reports/runs/nasa-ims-qwen-smoke-fase3-qwen/
```

La prueba registro 12 decisiones agenticas. Entre ellas, el limpiador selecciono ‘channel_1’, el estructurador eligio ventanas de 1024 muestras con solape del 50% y el modelador propuso ‘isolation_forest’. El resultado final fue ‘failed’, no por fallo de gestion o procesamiento, sino por guardarrail metodologico. Por tanto, esta ejecucion se interpreta como validacion de la adaptacion inicial de agentes y artefactos, no como experimento de rendimiento NASA IMS.

Para evitar presentar metricas supervisadas sin etiquetas defendibles, se ha anadido un diagnostico no supervisado de degradacion sobre las features NASA generadas:

```
python -m codigo.scripts.run_nasa_ims_degradation_diagnostics
```

El diagnostico no calcula recall ni F1. Sobre la prueba sintetica NASA IMS detecta una tendencia creciente: media inicial 0.3080, media final 0.6866, ratio final/inicial 2.2296 y pendiente temporal 0.1893.

6.6. Benchmark NASA IMS Sintetico Etiquetado

Para obtener una evaluacion supervisada multi-dataset sin usar etiquetas inventadas sobre NASA IMS real, se ejecuto un benchmark sintetico temporal. La estructura mantiene el formato NASA IMS Set 2, pero las etiquetas binarias son controladas por construccion.

```
python -m codigo.scripts.run_nasa_ims_synthetic_temporal_benchmark \
--model qwen3.5:4b \
--run-id nasa-ims-synth-agentic-qwen-fase3
```

Tabla 6.4: Run agentica sobre benchmark NASA IMS sintetico etiquetado

Modelo	Precision	Recall	F1	ROC-AUC	FPR
Isolation Forest	0.9130	0.6000	0.7241	0.8041	0.1429

La ejecucion quedo rechazada por el agente evaluador, ya que no cumple el recall minimo de 0.90 ni la tasa maxima de falsos positivos de 0.10. Este resultado es deliberadamente util: aporta un caso no perfecto sobre un dataset distinto de CWRU y muestra que el sistema no solo informa exitos, sino tambien fallos tecnicos que requieren una nueva iteracion de configuracion.

6.7. Iteracion Agentica sobre NASA IMS Sintetico

A partir de la run rechazada se ejecuto un bucle de reintento acotado, con dos intentos maximos. El modelador recibio un analisis de fallo con falsos negativos, falsos positivos y convencion del umbral. En ambos intentos Qwen interpreto que el problema principal era la perdida de anomalias y propuso bajar ‘threshold_quantile’.

Tabla 6.5: Reintentos agenticos sobre benchmark NASA IMS sintetico

Run	Umbral	Precision	Recall	F1	FPR
Base	0.99	0.9130	0.6000	0.7241	0.1429
Retry 1	0.95	0.8846	0.6571	0.7541	0.2143
Retry 2	0.50	0.7143	1.0000	0.8333	1.0000

El resultado ilustra el comportamiento buscado para el TFM. El agente aprende de los falsos negativos y actua en la direccion tecnica correcta: reducir el umbral aumenta el recall. Sin embargo, el segundo reintento muestra tambien el riesgo industrial de optimizar solo sensibilidad: se alcanza recall 1.0000, pero todos los normales del split de test pasan a clasificarse como anomalias. La ejecucion queda rechazada por FPR excesiva y el sistema detiene el bucle con ‘next_action = stop’ al agotarse el presupuesto de reintentos. Por tanto, la arquitectura permite deliberacion y correccion agentica, pero conserva un limite explicito contra busquedas indefinidas o aprobaciones engañosas.

Sobre estos reintentos se genero una auditoria de razonamiento. El primer caso se clasifico automaticamente como ‘partially_supported’, ya que la hipotesis mejoro recall y F1 pero mantuvo FPR por encima del umbral. El segundo se clasifico como ‘overcorrected’: la direccion del razonamiento era correcta, pero la magnitud del cambio

fue insegura. En ambos casos se creo una solicitud de revision humana para que el razonamiento pueda etiquetarse posteriormente como ejemplo positivo, negativo o caso frontera antes de incorporarlo a una memoria contextual.

6.8. Primer Experimento con Memoria Agentica

Como primer experimento de Fase 4 se activo la memoria RAG del modelador sobre la misma run NASA sintetica rechazada. Previamente se habia incorporado a ‘modeler_memory’ el segundo reintento de Fase 3 como ‘boundary_case’ revisado: la direccion de bajar el umbral era correcta, pero la magnitud habia producido FPR 1.0000.

```
python -m codigo.scripts.run_nasa_ims_agentic_retry \
  --model qwen3.5:4b \
  --source-run-id nasa-ims-synth-agentic-qwen-fase3 \
  --run-id-prefix nasa-ims-synth-agentic-qwen-fase4-memory \
  --max-attempts 1 \
  --use-memory \
  --memory-dir codigo/reports/reasoning_memory
```

El sistema recupero el recuerdo correcto y el agente declaro haberlo usado. La decision cito explicitamente el ‘memory_record_id’ del caso de sobrecorreccion. Sin embargo, Qwen volvio a proponer ‘threshold_quantile=0.50’, reproduciendo la misma sobrecorreccion que la memoria advertia.

Tabla 6.6: Reintento con memoria agentica sobre NASA IMS sintetico

Run	Umbral	Precision	Recall	F1	FPR
Base	0.99	0.9130	0.6000	0.7241	0.1429
Memoria	0.50	0.7143	1.0000	0.8333	1.0000

Este resultado es importante porque separa dos capacidades distintas. La infraestructura de memoria funciona: recupera el caso relevante, lo inyecta en el prompt y exige que el agente cite los recuerdos usados. No obstante, el LLM local no siempre convierte esa evidencia en una decision prudente. Para la siguiente iteracion sera necesario auditar el uso de memoria: el agente debera explicar como evita repetir fallos asociados a recuerdos ‘boundary_case’ o ‘warning’, no solo declarar que los ha leído.

Tras endurecer el contrato, se ejecuto una segunda variante con auditoria de uso de memoria:

```
python -m codigo.scripts.run_nasa_ims_agentic_retry \
  --model qwen3.5:4b \
  --source-run-id nasa-ims-synth-agentic-qwen-fase3 \
  --run-id-prefix nasa-ims-synth-agentic-qwen-fase4-memory-audit \
  --max-attempts 1 \
  --use-memory \
  --memory-dir codigo/reports/reasoning_memory
```

En esta segunda ejecucion, Qwen no solo cito la memoria, sino que declaro que la usaba como ‘adapted’, explico su influencia y propuso una mitigacion del riesgo de sobrecorreccion. Aun asi, volvio a elegir ‘threshold_quantile=0.50’ y el resultado mantuvo FPR 1.0000. La nueva auditoria clasifico el caso como repeticion de fallo

de frontera, proporcionando evidencia estructurada de que el agente habia leído la advertencia pero no la habia transformado en una accion suficientemente distinta.

La auditoria se incorporo despues al indice RAG como una nueva memoria de tipo 'warning', usando 'memory_usage_audit' como fuente. Al reconstruir la memoria con 'qwen3-embedding:0.6b', el indice del modelador contiene dos recuerdos: el caso frontera revisado por humano y la advertencia derivada de la auditoria. En una consulta de humo sobre repeticion de fallo por umbral, la advertencia se recupera antes que el caso frontera, con uso 'negative_warning'. Esto prepara la siguiente ejecucion: evaluar si el agente cambia su decision cuando no solo recibe el recuerdo original, sino tambien la evidencia de que ya lo interpreto mal una vez.

La ejecucion con memoria ampliada produjo un cambio cualitativo. Qwen recupero y cito los dos recuerdos, pero esta vez no repitio 'threshold_quantile=0.50'. Propuso un ajuste conservador a 'threshold_quantile=0.95', explicando que la direccion de bajar el umbral seguia siendo razonable pero que la magnitud debia limitarse por la advertencia de sobrecorreccion. El resultado mejoro recall '0.6000 ->0.6571' y F1 '0.7241 ->0.7541', pero aumento FPR '0.1429 ->0.2143'. Por ello, el evaluador rechazo la ejecucion. La auditoria de memoria clasifico el uso como 'memory_aligned': el agente aplico la leccion de no repetir el salto radical, aunque el ajuste resultante siguio siendo insuficiente.

Tras conectar los post-mortems con el contrato 'DecisionEpisode', se ejecuto una nueva run Qwen con el mismo protocolo. El resultado numerico fue equivalente al reintento anterior, pero la ejecucion genero automaticamente 'decision_episode.json' y 'memory_candidate.json'. El candidato quedo clasificado como 'boundary_case' para el modelador: documenta que la memoria evito repetir el cambio radical de umbral, pero que el ajuste moderado solo produjo una mejora parcial y aumento FPR. Al reconstruir la memoria canonica con embeddings Qwen, este candidato entro en 'modeler_memory' como recuerdo de tipo 'decision_episode'.

Una consulta de humo sobre mejora parcial de umbral y comparacion de familias de modelo recupero el nuevo episodio como 'boundary_context', junto con el caso humano revisado y las auditorias de uso de memoria. Esto confirma que la memoria ya contiene episodios completos de decision, no solo advertencias puntuales sobre umbrales.

Con esa memoria por episodios se ejecuto una nueva variante orientada a comprobar si el modelador dejaba de razonar exclusivamente sobre el umbral. El contrato de prompt incluyo 'source_type' y candidatos comparables. La run 'nasa-ims-synth-agentic-qwen-fase4-model-family-attempt-01' recupero cinco recuerdos, cito tres de ellos y anadio explicitamente 'pca_reconstruction_error_candidate' como alternativa. La configuracion ejecutada siguio siendo Isolation Forest con umbral 0.95, por lo que las metricas fueron equivalentes al caso anterior: precision 0.8846, recall 0.6571, F1 0.7541 y FPR 0.2143. El avance principal no fue metrico, sino de trazabilidad: el agente dejo evidencia de una comparacion entre familias de modelo dentro del contrato validado.

La misma ejecucion revelo una limitacion nueva. Aunque Qwen uso memoria y planteo PCA como alternativa, su explicacion incluyo una inconsistencia numerica: describio '0.95' como mayor que '0.99' y trato el cuantil '0.99' como si fuera bajo. Para convertir este fallo en evidencia reutilizable se amplio la auditoria de memoria y se ejecuto 'nasa-ims-synth-agentic-qwen-fase4-coherence-audit-attempt-01'. La auditoria clasifico el resultado como 'memory_reasoning_inconsistent' y guardo el motivo concreto en 'memory_usage_audit.json'. Tras reconstruir el indice con 'qwen3-embedding:0.6b', 'modeler_memory' contiene 9 registros: un caso humano revisado, cinco auditorias de

uso y tres episodios de decision.

El ultimo ajuste de esta serie consistio en convertir la recomendacion de cambiar de familia en una guia ejecutable dentro del prompt. Cuando la memoria indica que los ajustes de umbral en Isolation Forest ya han producido mejoras parciales rechazadas, la guia de reintento marca PCA como configuracion preferente o, alternatively, permite que el agente decida parar. Con esta guia, la run final de PCA ejecuto realmente el modelo por error de reconstruccion con umbral 0.99.

Tabla 6.7: Cambio de familia guiado por memoria sobre NASA IMS sintetico

Run	Modelo	Precision	Recall	F1	FPR
Base	IForest 0.99	0.9130	0.6000	0.7241	0.1429
IForest memoria	IForest 0.95	0.8846	0.6571	0.7541	0.2143
PCA memoria	PCA 0.99	0.9375	0.4286	0.5882	0.0714

El resultado muestra un trade-off claro: PCA redujo la tasa de falsos positivos por debajo del umbral operativo, pero perdio demasiadas anomalias, con recall 0.4286. La ejecucion fue rechazada por el evaluador, pero la memoria logro su objetivo metodologico: provocar un cambio real de familia de modelo y generar evidencia sobre por que esa alternativa concreta tampoco es suficiente en su configuracion conservadora.

6.9. Memoria Transversal en una Run Completa CW-RU

Despues de validar la memoria del modelador, se ejecuto el pipeline CWRU completo con memoria transversal activada para estructurador y evaluador. La primera run uso agentes deterministas y sirvio para generar los primeros episodios reutilizables de esos dos roles:

```
python -m codigo.scripts.run_cwru_pipeline_with_memory \
  --run-id cwru-memory-transversal-fase4-001 \
  --rebuild-index \
  --clear-existing \
  --embedding-provider ollama \
  --embedding-model qwen3-embedding:0.6b
```

La run fue aprobada con precision 0.9991, recall 1.0000, F1 0.9996 y FPR 0.0513. Tras reconstruir el indice, aparecieron por primera vez las memorias del estructurador y del evaluador, con un registro cada una.

La segunda run activo Qwen para comprobar si esos recuerdos eran recuperados y citados por agentes reales:

```
python -m codigo.scripts.run_cwru_pipeline_with_memory \
  --run-id cwru-memory-transversal-fase4-002-llm \
  --use-llm \
  --model qwen3.5:4b \
  --rebuild-index \
  --clear-existing \
  --embedding-provider ollama \
  --embedding-model qwen3-embedding:0.6b
```

El estructurador declaro ‘used_memory_context=true’ y cito el episodio de estructuracion de la run anterior. El evaluador hizo lo mismo con el episodio de evaluacion previo. Las metricas finales fueron equivalentes a la primera run y la ejecucion quedo aprobada. El indice reconstruido contiene veinte registros: dieciseis en ‘modeler_memory’, dos en ‘structurer_memory’ y dos en ‘evaluator_memory’.

La importancia de este experimento es cualitativa. La memoria ya no solo ayuda a un reintento de modelado, sino que se propaga por el proceso completo: estructura, evaluacion y modelado pueden dejar y recuperar experiencias separadas por rol. Al mismo tiempo, el evaluador sigue aprobando solo por las metricas actuales, no porque exista un recuerdo recuperado.

Como cierre del hito, se ejecuto la auditoria cualitativa transversal:

```
python -m codigo.scripts.audit_transversal_memory_run \
--run-id cwru-memory-transversal-fase4-002-11m
```

La auditoria clasifico el uso de memoria como ‘memory_aligned’ para el estructurador y para el evaluador. Ambos agentes citaron recuerdos que habian sido recuperados en su contexto RAG y declararon el uso como ‘adapted’. En el caso del evaluador, la aprobacion queda respaldada por las metricas actuales del run, no por la existencia de memoria previa: recall 1.0000 frente al umbral 0.9000 y FPR 0.0513 frente al maximo 0.1000. La auditoria recomienda revision humana si estos candidatos se quieren tratar como evidencia principal, aunque permite su reutilizacion como contexto trazable.

6.10. Validacion del Runner Comun Multi-dataset

Para comprobar que la capa comun de planificacion no modifica el comportamiento ya validado, se ejecuto CWRU mediante el runner multi-dataset:

```
python -m codigo.scripts.run_dataset_pipeline_with_memory \
--dataset-id cwru_bearing \
--adapter-id cwru_bearing \
--raw-path codigo/data/raw/cwru_bearing/mat \
--run-id cwru-runner-common-validation-fase4-001
```

La ejecucion completo el pipeline, fue aprobada y no produjo errores. Las metricas fueron precision 0.9991497803599263, recall 1.0000, F1 0.9995747093847462 y FPR 0.05128205128205128. Estos valores coinciden exactamente con las dos referencias CWRU previas usadas como baseline, por lo que el runner comun no introduce regresion en el benchmark principal.

Tambien se ejecuto NASA IMS en modo diagnostico:

```
python -m codigo.scripts.run_dataset_pipeline_with_memory \
--dataset-id nasa_ims_bearing \
--adapter-id nasa_ims_bearing \
--raw-path codigo/data/raw/nasa_ims_bearing/preextracted_synthetic_qwen \
--run-id nasa-runner-common-diagnostic-fase4-001 \
--execution-mode diagnostic
```

La ejecucion termino sin errores, pero sin metricas ni evaluacion. Solo se ejecutaron manifesto, perfilado, limpieza y estructuracion; los artefactos generados fueron manifesto, perfil, senales limpias, log, features, tensores y splits. Esto confirma que el modo diagnostico no entra en modelado supervisado.

Por ultimo, se comprobo que NASA IMS en modo completo sigue bloqueado en la fase de planificacion, con ‘can_execute_requested_stages=false’ y bloqueos en ‘modeling’ y ‘evaluation’. La causa declarada es la ausencia de una politica temporal y de etiquetas por ventana defendible para NASA IMS real.

Una vez implementada la primera politica temporal versionada, se repitio la planificacion y ejecucion completa declarando:

```
python -m codigo.scripts.run_dataset_pipeline_with_memory \
  --dataset-id nasa_ims_bearing \
  --adapter-id nasa_ims_bearing \
  --raw-path codigo/data/raw/nasa_ims_bearing/preextracted_synthetic_qwen \
  --run-id nasa-runner-temporal-policy-v1-completed-low-metrics-002 \
  --dataset-policy-id nasa_ims_temporal_v1
```

Con la politica temporal v1, el runner genero un manifiesto derivado, etiqueto dos snapshots como ‘normal’ y uno como ‘degradation’, y la estructuracion uso los hints de split declarados en ‘metadata_json’: un snapshot normal para entrenamiento, un snapshot normal para test y un snapshot de degradacion para test. La ejecucion alcanzo modelado y evaluacion sin errores de infraestructura. Sobre el split de test se obtuvieron precision 0.5000, recall 1.0000, F1 0.6667 y FPR 1.0000. El evaluador marco la run como ‘approved=false’ por exceso de falsos positivos, pero el supervisor la completo y genero informe final. La conclusion experimental es que la politica temporal desbloquea una ejecucion completa y auditable, aunque las metricas siguen siendo proxy metodologicas y requieren un protocolo mas robusto antes de usarse como resultado principal.

6.11. Experimento de cierre de Fase 9

El cierre de Fase 9 se valida con dos tipos de prueba: una suite comparable de modelos run-to-failure y una run agentica libre con Qwen. La primera comprueba que las familias PCA, Isolation Forest, One-Class SVM y autoencoder PyTorch se ejecutan bajo el mismo protocolo temporal. La segunda comprueba que el agente `modeler` mantiene capacidad de decision y no queda sustituido por una regla que fuerce el autoencoder.

La suite ligera se ejecuto sobre `synthetic_autoencoder_readiness_lite`, un conjunto NASA IMS-like de 24 snapshots sinteticos preparado para no convertir el cierre de fase en un benchmark nuevo. La herramienta de readiness declaro `autoencoder_ready=true`, `readiness_level=caution` y `n_train_nominal_windows=35`. RUL se mantuvo bloqueado por existir una sola trayectoria completa, lo que valida que readiness permite autoencoder pero no abre automaticamente pronostico de vida util.

```
python -m codigo.scripts.run_run_to_failure_model_suite \
  --plan-id fase9-hito4b-ae-readiness-lite-001 \
  --raw-path codigo/data/raw/nasa_ims_bearing/synthetic_autoencoder_readiness_lite
```

La segunda prueba ejecuto el pipeline comun con agentes Qwen/Ollama reales:

```
python -m codigo.scripts.run_dataset_pipeline_with_memory \
  --dataset-id nasa_ims_bearing \
  --adapter-id nasa_ims_bearing \
  --dataset-policy-id nasa_ims_temporal_v1 \
  --run-id fase9-agentic-modeler-readiness-lite-llm-guardrail-001 \
  --raw-path codigo/data/raw/nasa_ims_bearing/synthetic_autoencoder_readiness_lite \
  --use-llm
```

Esta run finalizo como `completed` y `approved=true`. El `modeler` eligio `pca_reconstruction_err` no el autoencoder, priorizando interpretabilidad temporal y robustez ante etiquetas proxy. Al mismo tiempo, mantuvo `autoencoder_dense_temporal` como candidato comparable condicionado a readiness suficiente. Este resultado es metodologicamente importante: el autoencoder existe y puede ejecutarse, pero no quita protagonismo al agente.

Por ultimo, se valido el hardening de fallbacks visibles. La misma linea de experimentos mostro que `supervisor`, `evaluator` y `report_writer` inicial responden con LLM real y confianza alta; las salidas de contrato incorrectas se registran como `Guardrail correction` y no como fallos opacos de LLM. La verificacion completa de codigo tras el cierre produjo:

```
python -m unittest discover codigo/tests
# Ran 352 tests
# OK (skipped=1)
```

6.12. Validacion operativa de la aplicacion local

La Fase 5 no introduce un nuevo benchmark numerico, sino una validacion operativa de la aplicacion que envuelve el pipeline. El criterio experimental pasa a ser comprobar que el flujo completo puede ejecutarse desde una interfaz web local sin duplicar la logica de planificacion ni leer artefactos directamente desde el navegador.

La validacion se organizo alrededor de los siguientes puntos:

- compilacion del frontend React/Vite mediante ‘`npm run build`’;
- comprobacion de ‘`GET /health`’ a traves del proxy local del frontend;
- comprobacion de ‘`GET /llm/status`’ para verificar disponibilidad de Ollama y del modelo ‘`qwen3.5:4b`’;
- preflight de CWRU mediante ‘`POST /runs`’ con ‘`dry_run=true`’;
- ejecucion en segundo plano con polling de job local;
- consulta de snapshot, metricas, informe, artefactos, memoria, observabilidad agentica y visualizacion;
- preflight de NASA IMS conservando los bloqueos o advertencias metodologicas declaradas por la politica de dataset.

Esta validacion no sustituye las metricas de los experimentos anteriores. Su valor es demostrar que la arquitectura completa puede operarse como una aplicacion local, manteniendo FastAPI como frontera canonica, los snapshots persistidos como fuente de verdad y los ejecutores deterministas como unica capa que transforma datos.

6.13. Experimentos posteriores

Experimentos previstos:

- validacion de la misma metodologia sobre un segundo dataset run-to-failure real, preferentemente XJTU-SY o PRONOSTIA;

- extension del RUL experimental con intervalos e incertidumbre, sin presentarlo como prediccion industrial validada;
- comparacion entre features temporales, frecuenciales y representaciones aprendidas cuando existan suficientes trayectorias completas;
- validacion leave-one-run-out o por condicion operativa para reducir riesgo de fuga temporal;
- prueba Docker completa tras incorporar PyTorch como dependencia del entorno local.

Capítulo 7

Resultados

7.1. Resultados cuantitativos

La validacion cuantitativa se ha organizado en dos niveles. Primero se conserva el benchmark CWRU como referencia de regresion del MVP. Despues se incorporan experimentos de Fase 3 donde los agentes locales proponen alternativas de ventana, modelo y reintento sobre evidencia persistida.

7.1.1. Baseline CWRU de Fase 2

La Fase 2 incorpora una comparacion local entre dos ejecuciones persistidas del baseline Isolation Forest sobre CWRU. Ambas ejecuciones mantienen constante el dataset, la limpieza, la estructuracion temporal, la semilla y ‘n_estimators=200’; la unica variable modificada es el cuantil usado para fijar el umbral de anomalia.

Experimento	Umbral	Precision	Recall	F1	FPR
Baseline	0.99	0.9991	1.0000	0.9996	0.0513
Conservador	1.00	0.9996	1.0000	0.9998	0.0256

Tabla 7.1: Comparacion de umbral con Isolation Forest en el split de test CWRU.

La variante conservadora reduce los falsos positivos sin penalizar el recall. En el baseline, la matriz de confusion del split de test contiene 111 verdaderos negativos, 6 falsos positivos, 0 falsos negativos y 7051 verdaderos positivos. En la variante con umbral 1.00, los falsos positivos bajan a 3, con 114 verdaderos negativos, 0 falsos negativos y 7051 verdaderos positivos.

7.1.2. Comparacion agentica de ventanas

En Fase 3 el agente estructurador Qwen no recibe una unica configuracion fija, sino que puede proponer alternativas comparables. Sobre CWRU propuso tres ventanas: 2048 muestras como opcion principal, 1024 muestras para mayor resolucion temporal y 4096 muestras para mayor contexto por ventana. El protocolo experimental materializo solo las alternativas propuestas por el agente y persistio cada run.

La alternativa de 1024 muestras obtuvo el mejor F1 y la menor tasa de falsos positivos. Este resultado es relevante porque la mejora no procede de una busqueda

Ventana	Precision	Recall	F1	FPR
1024	0.9996	0.9999	0.9998	0.0212
2048	0.9991	1.0000	0.9996	0.0513
4096	0.9991	1.0000	0.9996	0.0517

Tabla 7.2: Comparacion agentica de tamanos de ventana en CWRU con 50 % de solape.

exhaustiva codificada, sino de una hipotesis propuesta por el agente y despues evaluada por ejecutores deterministas.

7.1.3. Comparacion agentica de modelos

La Fase 3 tambien amplio el espacio de decision del modelador con PCA basado en error de reconstruccion. Qwen propuso comparar Isolation Forest, PCA y una variante conservadora de Isolation Forest usando la ventana de 1024 muestras seleccionada en el experimento anterior.

Modelo	Umbral	Precision	Recall	F1	FPR
Isolation Forest	0.99	0.9996	0.9999	0.9998	0.0212
PCA reconstruction	0.99	1.0000	0.9623	0.9808	0.0000
Isolation Forest conservador	1.00	0.9999	0.9994	0.9996	0.0085

Tabla 7.3: Comparacion agentica de modelos sobre CWRU con ventana de 1024 muestras.

PCA ofrece un intercambio distinto: elimina falsos positivos en este protocolo, pero reduce el recall. Este resultado evita que toda la evidencia de la fase dependa de metricas perfectas y muestra que el agente puede comparar alternativas con ventajas y limitaciones tecnicas diferentes.

7.1.4. Benchmark NASA sintetico y reintentos

NASA IMS real se mantiene bloqueado para metricas supervisadas hasta definir una politica temporal y de etiquetado defendible. Para obtener un caso no trivial sin atribuir etiquetas artificiales al dataset real, se genero un benchmark sintetico con forma de entrada NASA IMS y etiquetas conocidas por construccion.

La primera ejecucion agentica obtuvo precision 0.9130, recall 0.6000, F1 0.7241, ROC-AUC 0.8041, PR-AUC 0.8842 y FPR 0.1429. El evaluador rechazo la run al detectar que las metricas no cumplian los umbrales locales. Despues se activo un bucle de reintento acotado donde el modelador recibia analisis de error y decidia si modificar la configuracion o detenerse.

El primer reintento mejora recall y F1, aunque aumenta la tasa de falsos positivos. El segundo reintento recupera todas las anomalias, pero clasifica tambien todos los normales de test como anomalos. El evaluador rechaza esta sobrecorreccion y el bucle se detiene por limite de intentos. El resultado es valioso porque muestra aprendizaje local de errores, pero tambien control ante una mejora aparente que seria inaceptable operativamente.

Run	Umbral	Precision	Recall	F1	FPR
Base	0.99	0.9130	0.6000	0.7241	0.1429
Reintento 1	0.95	0.8846	0.6571	0.7541	0.2143
Reintento 2	0.50	0.7143	1.0000	0.8333	1.0000

Tabla 7.4: Reintentos agenticos sobre benchmark NASA sintetico etiquetado.

7.1.5. Primer resultado con memoria agentica

En Fase 4 se realizo una primera prueba con memoria vectorial activa. El modelador recupero un recuerdo revisado del segundo reintento de Fase 3, etiquetado como ‘boundary_case’ por sobrecorreccion del umbral. El agente declaro que habia usado memoria y cito el identificador del recuerdo recuperado. Por tanto, la trazabilidad del RAG funciono correctamente.

Run	Umbral	Precision	Recall	F1	FPR
Base sin memoria	0.99	0.9130	0.6000	0.7241	0.1429
Retry con memoria	0.50	0.7143	1.0000	0.8333	1.0000

Tabla 7.5: Primera comparacion con memoria agentica sobre NASA sintetico.

La conclusion es mixta. La recuperacion semantica funciona, pero el agente interpreto el recuerdo de frontera como apoyo para repetir la accion que habia generado la sobrecorreccion. El resultado demuestra una limitacion importante: un RAG operativo no garantiza aprendizaje correcto si el contrato no exige razonar sobre el tipo de recuerdo recuperado. Esta evidencia orienta el siguiente ajuste de Fase 4: auditar si la memoria fue seguida, ignorada o contradicha, especialmente en casos ‘boundary_case’ y ‘warning’.

Tras incorporar esta auditoria, se repitio la prueba. El agente declaro que usaba el recuerdo como ‘adapted’, explico que la memoria advertia contra bajar el umbral sin control y propuso una mitigacion. Sin embargo, la configuracion final siguio siendo ‘threshold_quantile=0.50’ y las metricas repitieron FPR 1.0000. El nuevo artefacto de auditoria clasifico el resultado como repeticion de fallo de frontera. Esto aporta una evidencia cualitativa mas fina: no solo se sabe que el agente recupero memoria, sino tambien que su explicacion no fue suficiente para evitar repetir el fallo.

El resultado se ha convertido a su vez en memoria reutilizable de tipo ‘warning’. La consulta posterior al indice recupera primero esta advertencia y despues el caso frontera original, lo que muestra que el sistema ya no solo almacena decisiones post-mortem, sino tambien evidencia sobre como los agentes usan o malinterpretan recuerdos previos. Este punto es relevante para la defensa del TFM: la memoria agentica se comporta como un mecanismo de retroalimentacion trazable, no como una lista estatica de ejemplos.

Al ejecutar de nuevo con esta memoria ampliada, el comportamiento cambio. El modelador cito los dos recuerdos y propuso ‘threshold_quantile=0.95’ en lugar de repetir ‘0.50’. Esta decision mejoro recall y F1 frente a la base, pero tambien aumento FPR hasta 0.2143, por lo que la ejecucion siguio rechazada. El resultado es positivo desde el punto de vista agentic, porque demuestra que la memoria puede modificar la estrategia del LLM local, pero tambien muestra que la mejora de razonamiento no

implica automaticamente aprobar metricas. La aplicacion conserva asi la separacion entre deliberacion generativa y validacion experimental.

La ejecucion posterior con generacion automatica de episodios no cambio las metricas, pero si cambio la calidad de la evidencia persistida. El sistema produjo ‘decision_episode.json’ y ‘memory_candidate.json’, y este ultimo se incorporo a la memoria vectorial como ‘boundary_case’. Una consulta semantica sobre mejora parcial de umbral y necesidad de comparar familias de modelo recupero este episodio junto a los recuerdos anteriores. Por tanto, el sistema ya no solo aprende que un valor de umbral fue problematico, sino que conserva una experiencia completa: contexto, accion, resultado, trade-off, rechazo del evaluador y condiciones de reutilizacion.

La siguiente iteracion confirma esta interpretacion. Al recuperar el episodio anterior, el modelador ya no recibio solo una advertencia sobre el umbral, sino una memoria que sugeria comparar familia de modelo tras una mejora parcial. La decision generada por Qwen incluyo ‘pca_reconstruction_error_candidate’ como alternativa comparable, aunque la configuracion ejecutada siguio siendo Isolation Forest con ‘threshold_quantile=0.95’. El resultado numerico no mejoro respecto a la run anterior, pero la evidencia cualitativa si: el sistema puede registrar que el agente considero PCA y por que no lo selecciono.

Tambien se obtuvo una evidencia negativa valiosa. La auditoria detecto que el razonamiento del agente contenia una comparacion numerica incoherente (‘0.95’ mayor que ‘0.99’). El nuevo resultado ‘memory_reasoning_inconsistent’ muestra que el proyecto no solo mide metricas del modelo, sino tambien la calidad de las explicaciones agenticas. Esta capa es importante para un sistema con LLM locales: permite reutilizar fallos de razonamiento como advertencias, sin convertirlos en ejemplos positivos.

Finalmente, se incorporo una guia derivada de memoria al prompt de reintento. Cuando los recuerdos recuperados indicaban que los ajustes de umbral sobre Isolation Forest ya habian producido mejoras parciales rechazadas, la guia marcaba PCA por error de reconstruccion como configuracion preferente, sin ejecutarla de forma determinista. Con esta condicion, Qwen eligio realmente PCA como configuracion de reintento. El resultado fue precision 0.9375, recall 0.4286, F1 0.5882 y FPR 0.0714. La ejecucion fue rechazada por bajo recall, pero aporta una evidencia relevante: la memoria no solo cambia textos o explicaciones del agente, sino que puede desplazar la accion ejecutada hacia una nueva familia de modelo. Al mismo tiempo, el evaluador conserva el control experimental y evita aceptar una alternativa que reduce falsos positivos a costa de perder demasiadas anomalias.

7.1.6. Resultado con memoria transversal

La extension de memoria a estructurador y evaluador se valido con dos runs CWRU completas. La primera genero episodios y candidatos para ambos agentes; la segunda, ejecutada con Qwen, recupero esos recuerdos. El estructurador y el evaluador declararon ‘used_memory_context=true’ y citaron los ‘memory_record_id’ correspondientes de la run anterior.

El resultado no pretende demostrar mejora metrica, porque CWRU ya estaba casi resuelto por el pipeline base. Su valor es arquitectonico: confirma que la memoria agentica puede operar por rol dentro de una ejecucion completa. Tras la segunda run, el indice contiene 20 registros: 16 en ‘modeler_memory’, 2 en ‘structurer_memory’ y 2 en ‘evaluator_memory’. Esto demuestra que la memoria supervisada empieza a

Run	Precision	Recall	F1	FPR	Memorias
Transversal 1	0.9991	1.0000	0.9996	0.0513	2
Transversal 2 Qwen	0.9991	1.0000	0.9996	0.0513	4

Tabla 7.6: Primer ciclo de memoria transversal en CWRU.

representar aprendizaje de proceso, no solo ajustes de hiperparametros.

La auditoria posterior del uso transversal clasifico ambas decisiones como ‘memory_aligned’. No se detectaron recuerdos citados fuera del contexto recuperado ni declaraciones de uso ausentes. El evaluador mantuvo respaldo metrico independiente de la memoria: recall 1.0000 y FPR 0.0513 bajo los umbrales exigidos. Por tanto, los nuevos candidatos quedan aceptados como contexto reutilizable, pero no como evidencia academica principal sin revision humana.

7.2. Resultados cualitativos

La prueba smoke con agentes Qwen sobre una carpeta NASA IMS sintetica preextraida confirma que el patron de expediente experto y decision estructurada es viable con LLM locales. El limpiador selecciono un canal a partir del perfil, el estructurador eligio una configuracion temporal soportada y el modelador propuso un modelo ejecutable. Sin embargo, la run termino en estado ‘failed’ por guardarrail metodologico: NASA IMS no debe evaluarse como problema supervisado hasta definir etiquetas por ventana y particion temporal sin fuga.

En Fase 4 se ha dado el primer paso para retirar ese bloqueo de forma controlada: la politica ‘nasa_ims_temporal_v1’ etiqueta el tramo inicial de cada secuencia como normalidad proxy, el tramo posterior como degradacion proxy y declara esos metadatos en el manifiesto derivado. La run ‘nasa-runner-temporal-policy-v1-completed-low-metrics-002’ alcanzo modelado, evaluacion e informe final. Termino como ‘completed’, pero con ‘approved=false’ por FPR 1.0000. Este resultado es util precisamente porque separa dos conceptos: la infraestructura ya puede ejecutar NASA IMS con politica declarada, pero las metricas aun no son evidencia aprobada.

La Fase 3 tambien introduce auditoria del razonamiento. Cada reintento puede generar un post-mortem que compara la hipotesis declarada por el agente con el resultado real. En el benchmark NASA sintetico, el primer reintento queda como ‘partially_supported’ y el segundo como ‘overcorrected’. Con el trigger de revision humana se genera ademas una plantilla para que una persona etiquete si el razonamiento debe reutilizarse, descartarse o conservarse como caso frontera.

Este punto refuerza la aportacion central del TFM: el sistema no solo registra metricas finales, sino tambien por que un agente cambio una configuracion, que efecto esperaba y si la evidencia posterior respalda ese razonamiento.

7.2.1. Resultado operativo de Fase 5

El resultado principal de Fase 5 es cualitativo y arquitectonico: el sistema pasa de ser un conjunto de scripts, servicios y endpoints a una aplicacion web local capaz de operar el pipeline completo. Desde la interfaz se puede configurar una run, planificarla, detectar bloqueos metodologicos, ejecutarla en segundo plano, seguir el job, abrir el

snapshot, revisar metricas, informe y artefactos, comparar ejecuciones, observar eventos agenticos, consultar memoria persistida y visualizar una proyeccion PCA cuando existen artefactos compatibles.

La activacion de `'use_llm=true'` deja de ser un modo bloqueado y pasa a estar condicionada por una comprobacion explicita de Ollama y del modelo `'qwen3.5:4b'`. Esto confirma que la aplicacion web puede invocar agentes Qwen reales sin relajar los contratos JSON/Pydantic ni permitir ejecucion de codigo generado. Al mismo tiempo, la memoria se mantiene visible y consultable desde la interfaz, pero su activacion como retrieval durante ejecuciones web queda fuera del cierre de Fase 5 para no mezclar dos cambios metodologicos independientes.

El rediseño final del frontend mejora tambien la interpretabilidad operacional: se separan configuracion, preflight, ejecucion, historico, agentes, memoria y visualizacion; se ocultan rutas locales de ficheros en la pagina web; y los detalles tecnicos quedan disponibles sin dominar la lectura principal. Por tanto, Fase 5 aporta una capa de producto local que hace defendible el sistema ante una demo, aunque todavia no resuelve empaquetado, despliegue ni multiusuario.

7.3. Resultados de Fase 9: run-to-failure avanzado

La Fase 9 aporta el resultado mas completo del perfil `run_to_failure_degradation`. La suite ligera con readiness confirma que el autoencoder PyTorch ya no es solo una propuesta futura: se entrena, genera artefactos reproducibles y se compara con PCA, Isolation Forest y One-Class SVM bajo la misma politica temporal. La Tabla 7.7 resume la lectura principal.

Modelo	Aprobado	Onset	Lead persistente	FAR nominal	F1 proxy
PCA	Si	1.0000	7200.7680	0.0952	0.8602
Isolation Forest	No	0.0000	0.0000	0.0159	0.2051
One-Class SVM	Si	1.0000	9600.7168	0.1905	0.9366
Autoencoder denso	Si	1.0000	7200.8192	0.0476	0.8854

Tabla 7.7: Suite run-to-failure ligera con readiness para autoencoder.

El autoencoder denso lidero metricas asociadas al Health Indicator, como caida del indicador, monotonidad y score global de salud, mientras que One-Class SVM lidero `lead time` persistente y tendencia del score. La conclusion no es que el autoencoder deba sustituir a PCA, sino que ya es una alternativa defendible para que el agente la considere cuando readiness lo permite.

La run agentica final confirma esta interpretacion. En `fase9-agentic-modeler-readiness-lite` Qwen eligio `pca_reconstruction_error` como modelo principal por interpretabilidad temporal y robustez ante etiquetas proxy. El autoencoder permanecio como candidato comparable, condicionado a readiness suficiente. Las metricas principales de la run fueron: onset confirmado 1.0000, `lead time` persistente 7200.7680, falsas alarmas nominales 0.0952, caida de Health Indicator 76.4453, monotonidad 0.5214, tendencia Spearman 0.7967 y F1 proxy 0.8602. La ejecucion quedo aprobada.

Tambien se obtiene un resultado cualitativo relevante sobre robustez agentica. Tras reforzar la reparacion JSON y los reintentos de contrato, las decisiones principales de `modeler`, `evaluator` y `report_writer` inicial aparecen como respuestas LLM reales

con confianza alta. Las correcciones de contrato quedan registradas como **Guardrail correction**, no como caídas genericas a fallback. Esto mejora la presentación metodológica del sistema: los guardarrailes se muestran como control de seguridad, no como sustitución silenciosa del agente.

Con estos resultados, Fase 9 cierra el paso de una detección temporal básica a un flujo PHM local: comparación de modelos, políticas temporales versionadas, Health Indicator, readiness para modelos avanzados, autoencoder PyTorch y recomendación agentica trazable. RUL completo queda correctamente fuera de las conclusiones, ya que el sistema solo declara que no está estimado cuando no hay evidencia suficiente.

7.4. Limitaciones observadas

Los resultados sobre CWRU deben interpretarse como validación del pipeline y del protocolo experimental, no como garantía de generalización industrial. CWRU es un benchmark controlado, con condiciones experimentales limpias y diferencias claras entre normalidad y fallo.

NASA IMS real aporta mayor interés industrial. La política temporal v1 permite métricas supervisadas proxy, pero estas no deben presentarse como etiquetas oficiales ni como resultado principal sin más validación. La versión sintética permite estudiar el comportamiento multiagente ante métricas no perfectas, y la política temporal permite estudiar el flujo run-to-failure, pero ambos casos requieren cautela metodológica.

Otra limitación es que los agentes locales siguen dependiendo de la calidad del expediente experto que reciben. El sistema evita delegar conocimiento de nicho en la memoria paramétrica del LLM, pero debe seguir ampliando diagnósticos, modelos soportados y revisiones humanas para que la autonomía agentica sea defendible en escenarios más variados.

Capítulo 8

Conclusiones

8.1. Conclusiones principales

El trabajo desarrollado demuestra que una arquitectura multiagente puede ser viable para procesamiento de datos industriales y detección de anomalías si se separa con claridad la decisión generativa de la ejecución determinista. Los agentes proponen configuraciones, comparan alternativas y justifican cambios; los ejecutores validan contratos, aplican transformaciones reproducibles y persisten artefactos, métricas y limitaciones.

La primera aportación es una arquitectura local trazable basada en LangGraph, contratos Pydantic y persistencia por run. Esta base permite reproducir el flujo completo sobre CWRU, consultar ejecuciones anteriores y comparar métricas sin mezclar razonamiento del agente con manipulación arbitraria de datos.

La segunda aportación es la generalización inicial a datasets heterogéneos. NASA IMS se ha incorporado mediante contratos comunes, adaptadores de manifiesto, perfilado enriquecido, selección controlada de canal y estructuración temporal. La evaluación supervisada sobre NASA IMS real se ha bloqueado de forma deliberada hasta definir etiquetas temporales y particiones sin fuga, lo que refuerza la validez metodológica del sistema.

La tercera aportación es el aumento de protagonismo agéntico sin eliminar guardarrailes. Los agentes Qwen locales han usado expedientes expertos para elegir canales, ventanas, modelos y reintentos. También han generado alternativas comparables y razonamientos que después se han auditado frente a los resultados reales. El sistema muestra que un LLM local no necesita conocer de memoria datasets industriales de nicho si recibe evidencia estructurada, opciones soportadas y límites claros.

La cuarta aportación es experimental. La comparación agéntica de ventanas y modelos sobre CWRU muestra mejoras y trade-offs medibles. En particular, PCA con error de reconstrucción reduce falsos positivos a costa de recall, mientras que Isolation Forest mantiene mayor sensibilidad. El benchmark NASA sintético introduce un caso no perfecto donde el agente aprende de falsos negativos, prueba una hipótesis y se detiene cuando la mejora de recall provoca una tasa de falsos positivos inaceptable.

La quinta aportación, iniciada en Fase 4, es una memoria agéntica supervisada y transversal. La memoria ya no se plantea como una optimización puntual del modelador, sino como un registro de experiencias reutilizables por agente: estructurador, modelador, evaluador y, posteriormente, otros roles. Esta orientación permite que el sistema aprenda gradualmente de ejecuciones reales, sin forzar errores artificiales y

manteniendo la separacion entre contexto recuperado, decision agentica y validacion determinista.

La sexta aportacion corresponde a la Fase 5: una aplicacion web local para operar el pipeline multiagente. La interfaz permite planificar, ejecutar, monitorizar, comparar, observar agentes, consultar memoria y visualizar resultados sin duplicar la logica del backend ni leer ficheros directamente desde el navegador. Esta capa convierte la arquitectura en una herramienta demostrable y no solo en un conjunto de scripts reproducibles.

La septima aportacion, cerrada en Fase 9, es la consolidacion del perfil run-to-failure como caso PHM agentico local. El sistema compara PCA, Isolation Forest, One-Class SVM y autoencoder PyTorch bajo una misma politica temporal; calcula onset confirmado, `lead time` persistente, falsas alarmas nominales, tendencia del score y Health Indicator; usa readiness para decidir si procede un modelo avanzado; y permite que Qwen elija sin imponerle el autoencoder. Ademias, las salidas LLM invalidas se reparan o se registran como correcciones de guardarrail, evitando que el sistema parezca depender de fallbacks opacos.

8.2. Limitaciones

Las metricas de CWRU no deben interpretarse como garantia de rendimiento en planta industrial. CWRU es un benchmark controlado y muy separable. NASA IMS es mas representativo de degradacion temporal. La politica `nasa_ims_temporal_v1` y las pruebas sinteticas permiten ejecutar y evaluar trayectorias run-to-failure de forma auditable, pero las etiquetas por ventana siguen siendo proxy y no deben presentarse como ground truth oficial. La validacion industrial requiere mas trayectorias, otros datasets run-to-failure y particiones por activo o condicion operativa.

El sistema tambien depende de la calidad de los expedientes expertos que reciben los agentes. Esta decision es intencionada: se evita pedir a un LLM local que invente conocimiento de dominio, pero obliga a mantener diagnosticos, contratos, registros de modelos y revisiones humanas bien documentados.

Con el cierre operativo de la Fase 6 ya existen imagenes Docker iniciales para backend y frontend, ademias de un ‘docker-compose’ minimo que captura puertos, volúmenes y variables principales del stack local. Las pruebas de humo automatizadas y la secuencia de demo final se difieren hasta estabilizar el siguiente bloque funcional de la aplicacion, y Ollama se mantiene como dependencia externa documentada. Ademias, los jobs de la API siguen siendo locales y en memoria, lo que es suficiente para la demo local pero no para una plataforma productiva.

La incorporacion de PyTorch para el autoencoder denso aumenta la capacidad tecnica del sistema, pero tambien obliga a validar reproducibilidad Docker y coste local. En la fase cerrada se ha usado CPU por defecto y arquitectura cerrada; quedan fuera LSTM autoencoder, senal cruda y RUL industrial.

8.3. Trabajo futuro

Las lineas principales son:

- validar la metodologia temporal sobre un segundo dataset run-to-failure real, como XJTU-SY o PRONOSTIA;

- incorporar RUL experimental con incertidumbre y criterios explicitos de no comunicacion de falsa precision;
- estudiar modelos secuenciales, como LSTM autoencoder, solo cuando existan suficientes trayectorias completas y tests reproducibles;
- ampliar la experimentacion multi-dataset con planes acotados, particiones por activo y comparacion frente a politicas deterministas;
- pulir la aplicacion web y backend tras disponer ya de un minimo Docker reproducible, ampliando funcionalidad, usabilidad y fronteras de consulta sin duplicar contratos;
- reconstruir y validar Docker tras la incorporacion de PyTorch;
- convertir las comprobaciones manuales de Docker en pruebas de humo automatizadas cuando la superficie funcional de la aplicacion este mas estabilizada;
- preparar una demo final repetible que levante la aplicacion, verifique Ollama/Qwen, ejecute CWRU desde la interfaz y muestre el bloqueo o la politica declarada de NASA IMS;
- evolucionar Human Review y memoria desde vistas locales hacia flujos mas robustos, manteniendo las pruebas automatizadas con intervencion humana apagada por defecto;
- preparar SLURM, checkpoints persistentes o despliegue cloud solo cuando el empaquetado local y la politica de ejecucion esten estabilizados.

Bibliografía

- [1] Case Western Reserve University Bearing Data Center. 12k drive end bearing fault data, 2026. Disponible en: <https://engineering.case.edu/bearingdatacenter/12k-drive-end-bearing-fault-data>.
- [2] Case Western Reserve University Bearing Data Center. Bearing data center, 2026. Disponible en: <https://engineering.case.edu/bearingdatacenter/welcome>.
- [3] Case Western Reserve University Bearing Data Center. Normal baseline data, 2026. Disponible en: <https://engineering.case.edu/bearingdatacenter/normal-baseline-data>.
- [4] Data.gov. Ims bearings, 2026. Disponible en: <https://catalog.data.gov/dataset/ims-bearings>.
- [5] IEEE PHM 2012 Data Challenge. Ieee phm 2012 prognostic challenge details, 2012. Disponible en: <https://raw.githubusercontent.com/wkzs111/phm-ieee-2012-data-challenge-dataset/master/IEEEPHM2012-Challenge-Details.pdf>.
- [6] NASA. Metrics library for prognostics performance evaluation, 2026. Disponible en: <https://software.nasa.gov/software/ARC-17898-1>.
- [7] NASA Prognostics Center of Excellence. Data set repository, 2026. Disponible en: <https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/>.
- [8] NASA Prognostics Center of Excellence. Ims bearings dataset, 2026. Disponible en: <https://phm-datasets.s3.amazonaws.com/NASA/4.+Bearings.zip>.
- [9] Xi'an Jiaotong University. Xjtu-sy bearing datasets, 2026. Disponible en: <https://github.com/WangBiaoXJTU/xjtu-sy-bearing-datasets>.